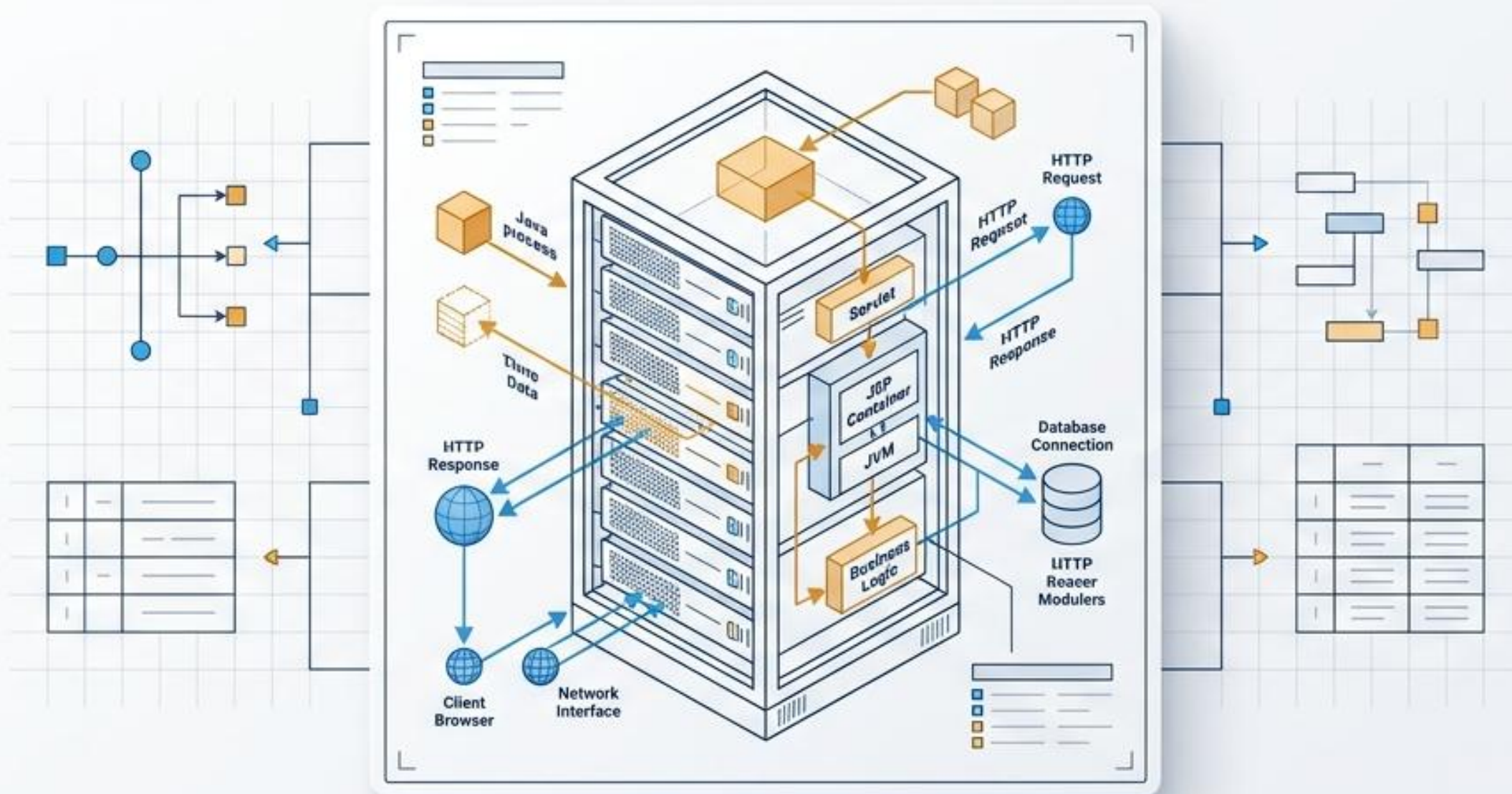


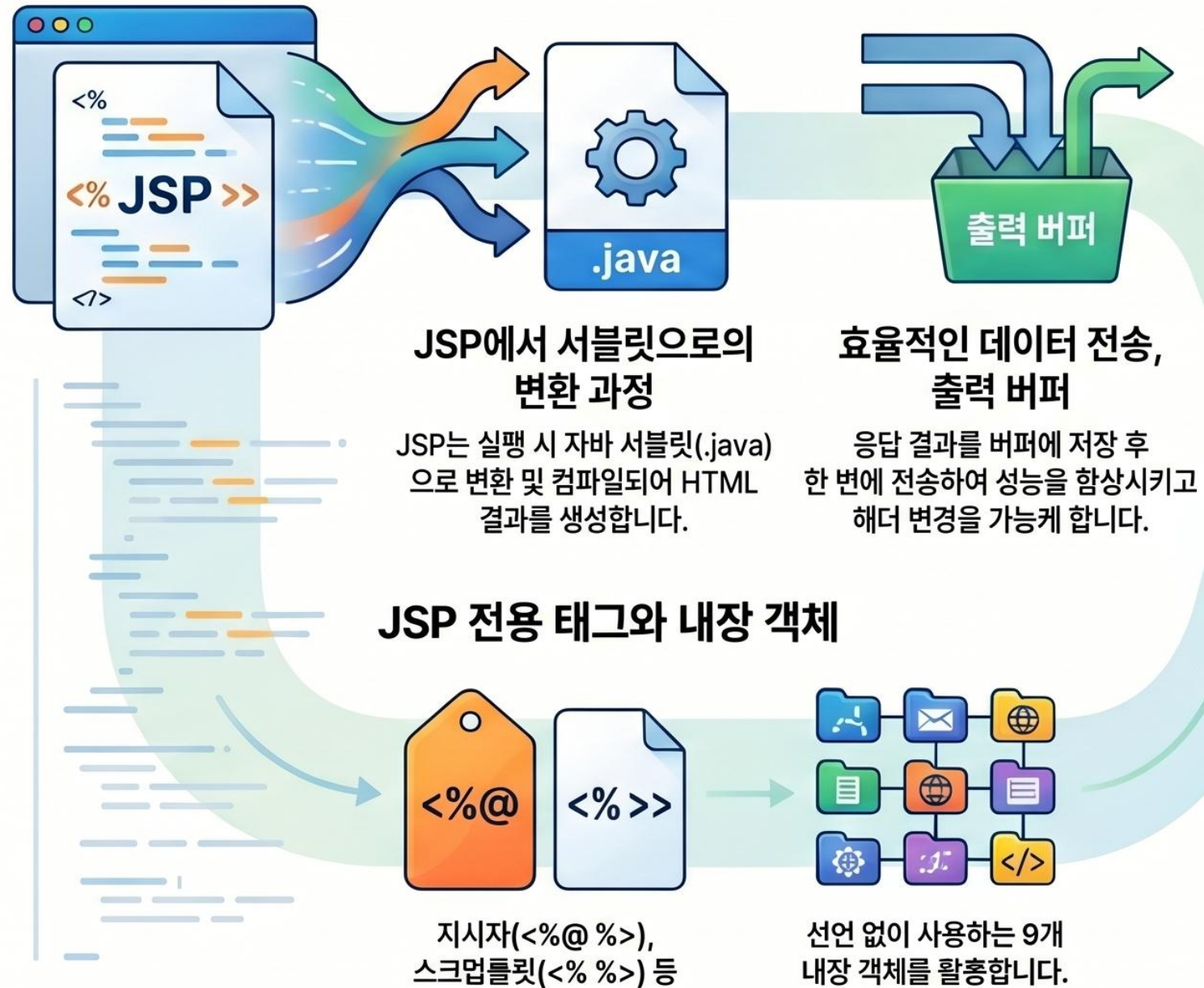
Java 웹 프로그래밍 설계도: JSP와 서블릿(Servlet) 아키텍처

HTTP 요청부터 MVC 패턴까지, 동적 웹 어플리케이션을 구동하는 데이터 처리 파이프라인 완벽 가이드



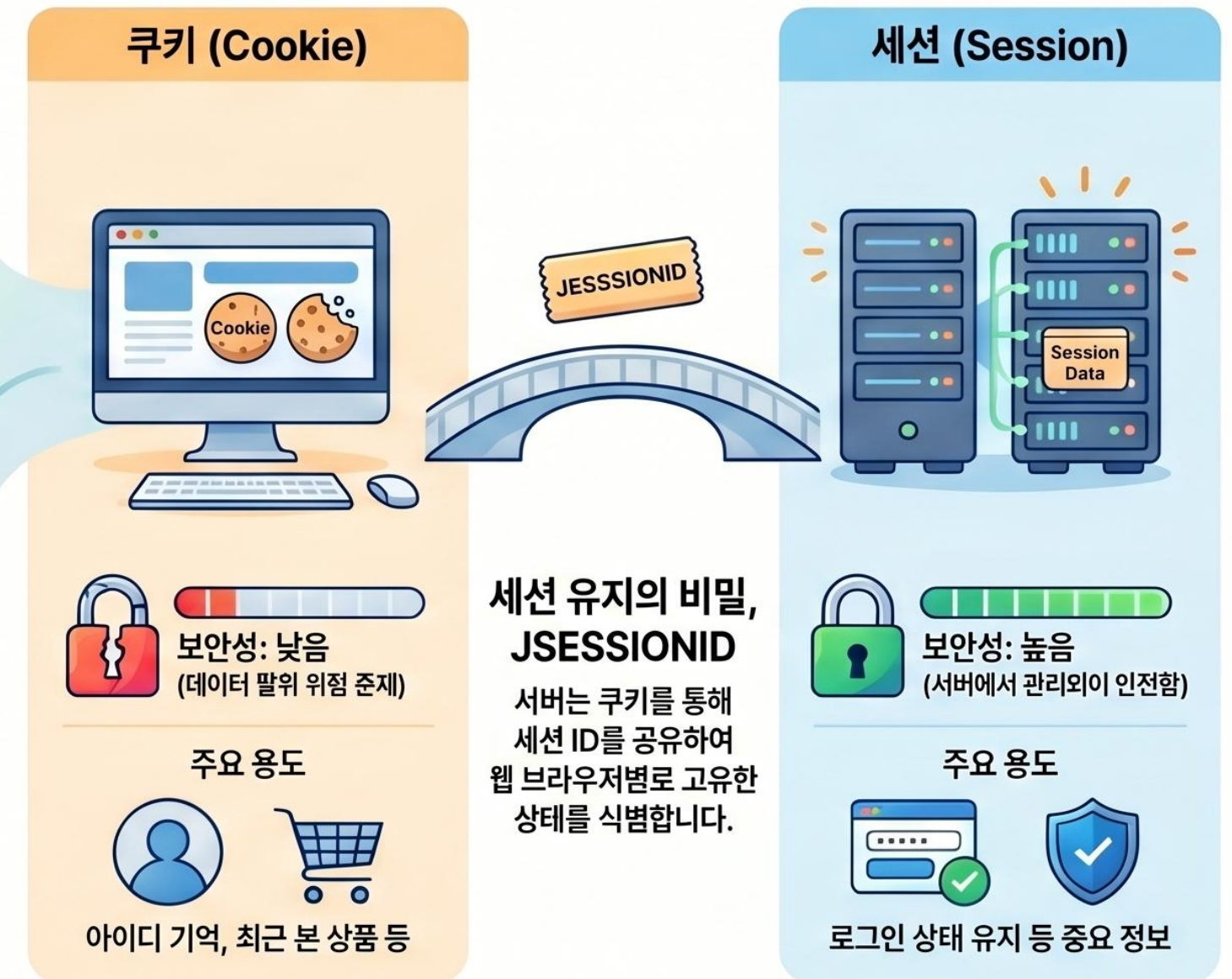
JSP 웹 프로그래밍 핵심 가이드: 동작 원리부터 상태 관리까지

JSP의 동작 원리와 구성



상태 관리의 핵심: 쿠키(Cookie) vs 세션(Session)

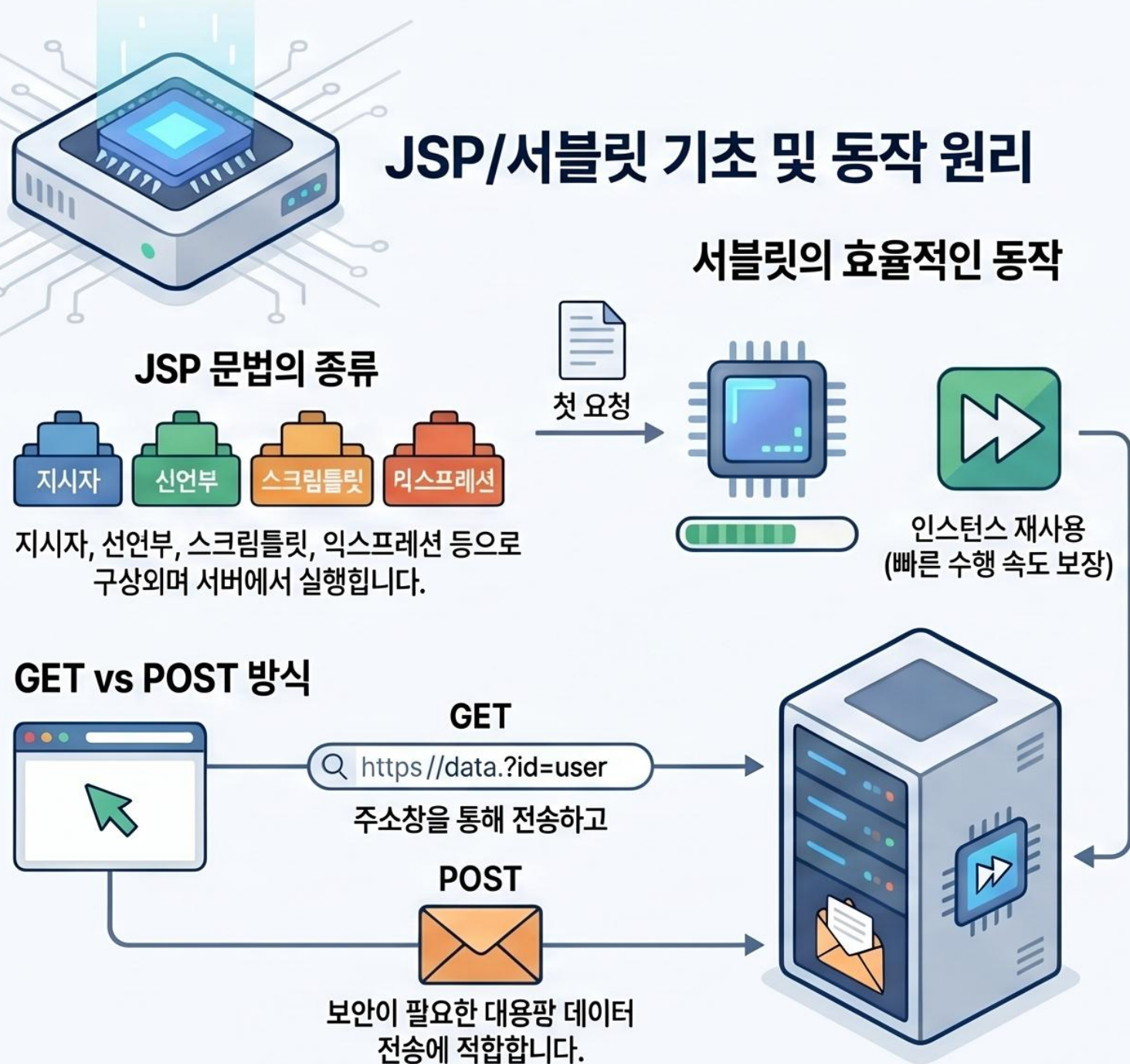
저장 위치와 보안의 차이



한눈에 보는 JSP & 서블릿 핵심 요약 가이드

JSP와 서블릿의 기본 개념, 동작 원리, 그리고 상태 관리 기술(쿠키/세션)을 요약하여 전달합니다.

JSP/서블릿 기초 및 동작 원리



상태 유지 기술: 쿠키와 세션

쿠키(Cookie) - 클라이언트 저장

브라우저에 데이터를 저장하며, 유효 시간 설정을 통해 데이터 삭제 시점을 조절합니다.

세션(Session) - 서버 저장

서버 영역에 데이터를 저장하고 브라우저에는 세션 ID만 전달하여 보안성이 높습니다.

세션 ID



핵심 차이점 비교 (저장 위치 & 보안 특성)

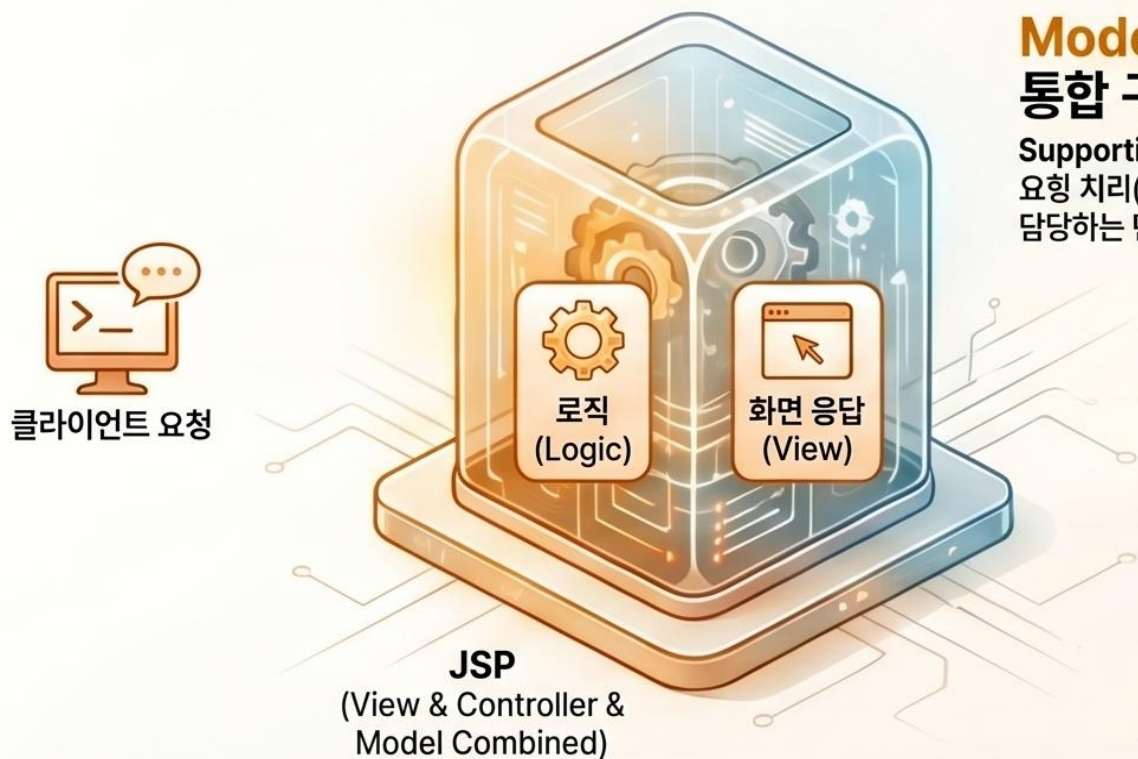
구분	쿠키 (Cookie)	세션 (Session)
저장 위치	웹 브라우저 (클라이언트)	웹 서버
데이터 전송	오징 시아다 URL과 함께 전송	세션 ID만 브라우저로 전송
보안성	상대적으로 낮음	상대적으로 높음

JSP 웹 개발 아키텍처: Model 1 vs. Model 2 비교 가이드

Context Summary: JSP(Java Server Page)는 HTML에 자바 코드를 삽입하여 동적 페이지를 생성하는 언어입니다. 웹 애플리케이션 구축 시 로직과 로직과 화면을 하나로 합치느냐(Model 1), 혹은 MVC 패턴에 따라 분리하느냐(Model 2)에 따라 개발 효율과 유지보수성이 결정됩니다.

Model 1: JSP 중심의 통합 구조

Supporting Detail: JSP가 클라이언트 요청 처리(로직)와 화면 응답(뷰)을 모두 담당하는 단순한 구조입니다.



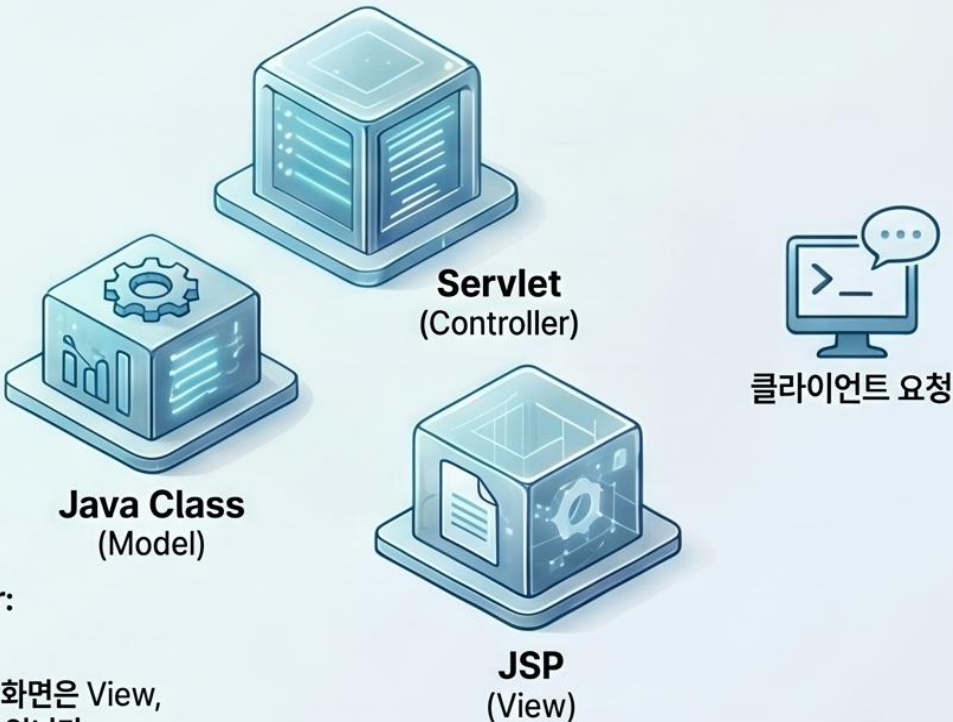
Model 2: MVC 기반의 역할 분리

Supporting Detail: Servlet(컨트롤러), Java Class(모델), JSP(뷰)가 각자의 역할을 엄격히 분담합니다.

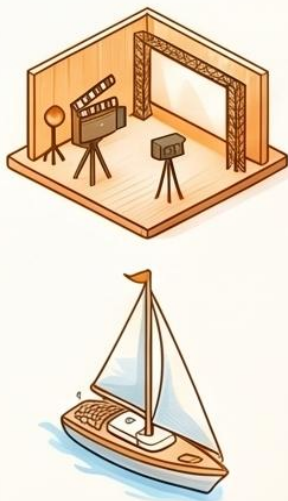
MVC 구성 요소의 이해



Supporting Detail: 데이터 처리는 Model, 화면은 View, 흐름 제어는 Controller가 담당하여 효율을 높입니다.



상황별 최적의 모델 선택 기준 - Model 1



Model 1 추천 - 단발성 프로토타입

Supporting Detail: 영화 홍보 페이지처럼 개발 기간이 짧고 굵병 사용을 중단할 페이지에 적합합니다.

프로젝트 규모와 유지보수성 고려

Supporting Detail: Model 1은 소규모에 유리하며, Model 2는 대규모 프로젝트의 확장성에 필수적입니다.

핵심 장단점 비교

Model 1		Model 2	
 빠름 (단순한 구조)	개발 속도	 느림 (초기 설정 복잡)	
 어려움 (코드 혼재)	유지보수	 용이함 (역할 분리)	
 소규모, 프로토타입	적합한 프로젝트	 대규모, 기업형 서비스	

상황별 최적의 모델 선택 기준 - Model 2

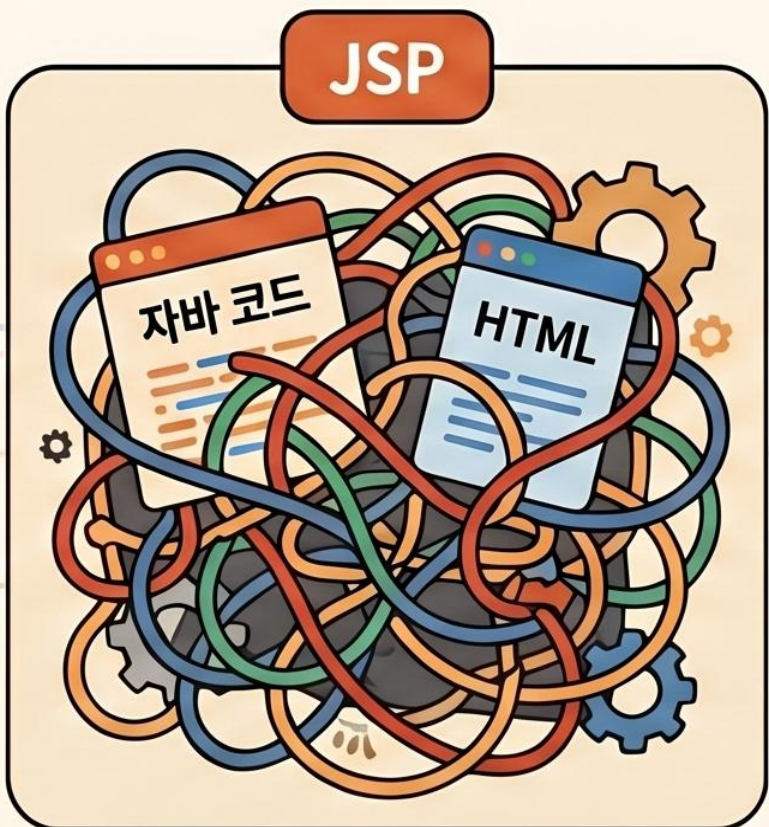
Model 2 추천 - 장기 운영 서비스

Supporting Detail: 지속적인 기능 추가와 유지보수가 필요한 대규모 시스템 구축 시 유리합니다.



웹 개발 아키텍처의 진화: 모델 1 vs. 모델 2 (MVC)

모델 1: 빠르지만 복잡한 통합 구조



● **JSP 중심의 단일 설계 방식**
JSP 마블 하나가 컨트롤러와 뷰 역할을 모두 수행하며 자바 코드와 HTML이 문체됩니다.



● **낮은 학습 곡선과 빠른 초기 개발**
구조가 단순하여 숙련도가 낮아도 빠르게 적응 가능하지만, 비즈니스 로직이 섞여 유지보수는 어렵습니다.



● **협업의 어려움 발생**
로직과 디자인 코드가 뒤섞여 개발자와 디자이너 간의 작업 분리가 불가능해집니다.

모델 2 (MVC): 역할 분리를 통한 체계적 설계

Model
(Business Logic, Data)

Controller
(Servlet)

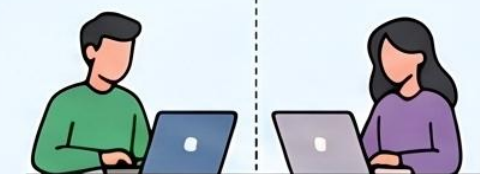
View
(JSP, Screen Output)

Model-View-Controller
3단 분리

● **Controller 3단 분리**
비즈니스 로직(Model), 화면 출력, 흐름 제어(Controller)로 역할을 명확히 나눕니다.

● **서블릿 기반의 흐름 제어**
컨트롤러인 서블릿이 요청을 받아 모델을 업데이트하고, 결과에 맞는 뷰(JSP)를 선택해 응답합니다.

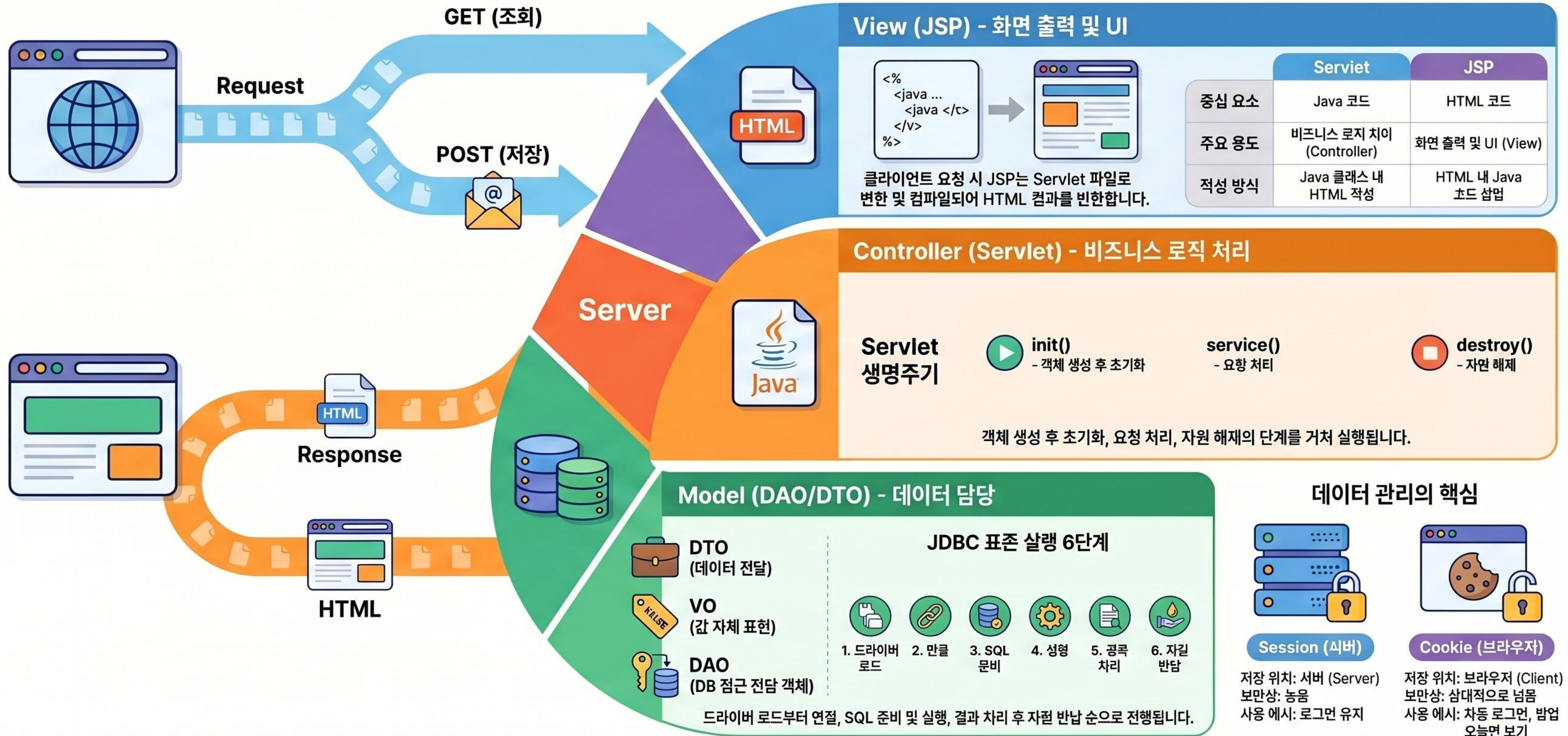
● **유지보수와 협업의 최적화**
로직이 명확해져 확장성이 뛰어나며, 개발자와 디자이너가 독립적으로 작업할 수 있습니다.



모델 1과 모델 2 핵심 차이점 비교

비교 항목	모델 1 (Model 1)	모델 2 (MVC)
핵심 구성	JSP + JavaBean	Servlet + JSP + JavaBean
주요 장점		
주요 단점		

한눈에 정리하는 Java 웹 개발 핵심 (JSP, Servlet, JDBC)



개발자를 위한 소프트웨어 디자인 패턴 가이드

디자인 패턴의 3대 분류(생성, 구조, 행동)와 각 핵심 패턴의 목적을 한눈에 파악하도록 돕습니다.

생성 및 구조 패턴 (Creational & Structural)

생성 패턴: 객체 생성의 유연성 확보

객체 생성 로직을 캡슐화하여 코드의 결합도를 낮추고 유연성을 높입니다.

구조 패턴: 더 큰 구조를 위한 조합

클래스나 객체를 조합하여 복잡한 시스템을 모듈적인 구조로 구성합니다.

어댑터(Adapter) 패턴

USB-C와 HDMI 변환기처럼 호환되지 않는 인터페이스를 연결합니다.



생성 (Creational):
싱글톤 (Singleton)
- 인스턴스를 단 하나만 생성 및 공유



구조 (Structural):
퍼사드 (Facade)
- 복잡한 시스템에 단순한 인터페이스 제공

행동 패턴 (Behavioral Patterns)

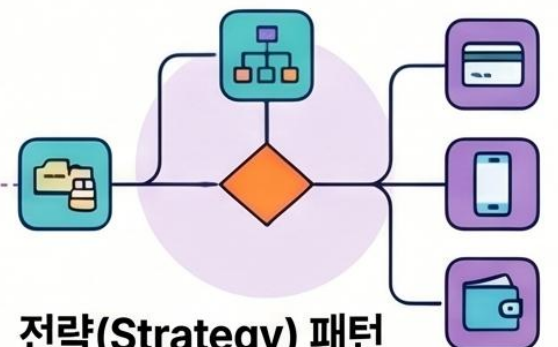
객체 간의 책임 분배와 상호작용

객체 사이의 알고리즘이나 통신 방식, 책임 할당을 정의하는 데 집중합니다.



옵저버(Observer) 패턴

유튜브 구독 알림처럼 상태 변화를 의존 객체들에게 자동으로 알립니다.



전략(Strategy) 패턴

결제 방식 선택처럼 런타임에 알고리즘을 자유롭게 교체할 수 있습니다.



행동 (Behavioral):
핸들러 메서드
- 알고리즘의 골격을 정의하고 단계별 제정의



디자인 패턴은 소프트웨어 설계 시 반복되는 문제들을 해결하기 위한 재사용 가능한 설계 방법론입니다. 객체의 생성 방식, 클래스 간의 조립 구조, 그리고 객체 간의 상호작용 방식에 따라 크게 세 가지 범주로 나뉩니다.

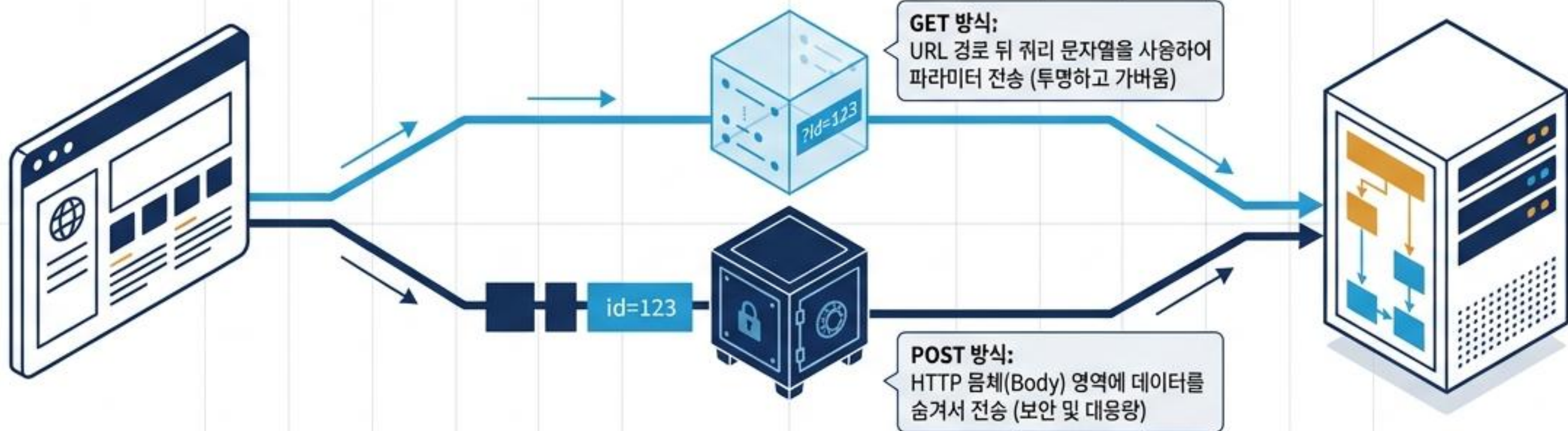
http://localhost:8080/hospitalWeb/memberform.jsp?id=123

프로토콜 (Protocol):
웹 브라우저와 웹 서버 간의
HTTP 데이터 통신 규칙

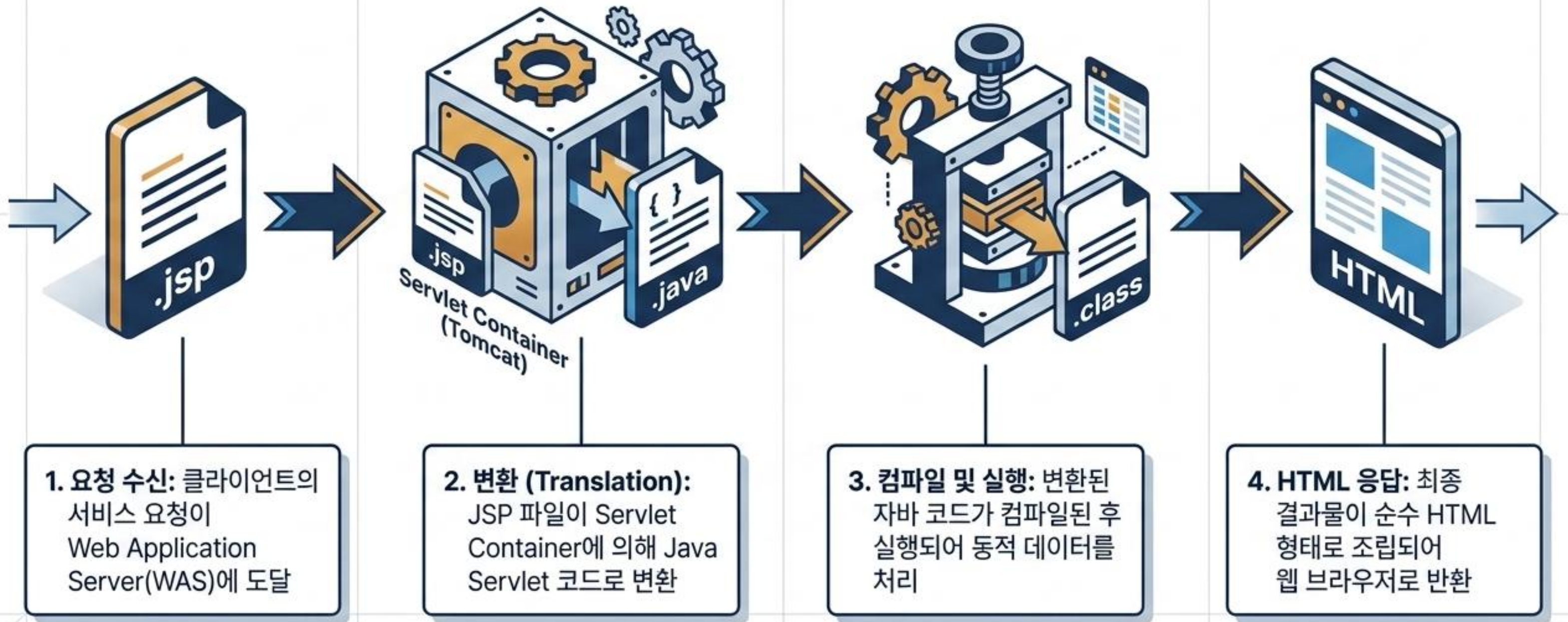
서버 이름 및 포트:
요청을 수신할 서버의 주소와
프로그램 식별자

경로 (Path):
실행할 JSP 페이지의
물리적 위치

쿼리 문자열 (Query String):
클라이언트가 서버로
전달하는 데이터



JSP 실행 파이프라인: 파일 형태의 변환 과정



서블릿(Servlet)과 JSP의 구조적 역전



서블릿 (Servlet)

- 자바 코드 안에 HTML 포함
- 자바 클래스 내에서 `out.println` 방식 사용
- 수정 시 재컴파일 필요



JSP

- HTML 문서 안에 자바 코드 포함
- 웹 디자이너와 협업 용이
- Tomcat(WAS)이 수정 후 재배포를 자동 처리



출력 버퍼(Output Buffer)와 데이터 전송 제어

JSP는 응답 결과를 즉시 브라우저로 보내지 않고 출력 버퍼에 모아 한 번에 전송합니다.

1. 성능 향상: 바구니에 물건을 모아 한 번에 옮기듯, 네트워크 I/O 횟수를 줄여 데이터 전송 성능을 극대화.

2. 버퍼 초기화 (포워딩):
버퍼가 다 차기 전이라면,
<jsp:forward>를 사용해 기존 데이터를 비우고 완전히 새로운 페이지로 요청 흐름을 이동 가능.

Clear

Flush

3. 응답 헤더(Header) 수정:
HTTP 규칙상 헤더가 먼저 전송되어야 함. 버퍼가 꽉 차서 전송되기 전까지는 자유롭게 응답 헤더 정보 수정 가능.

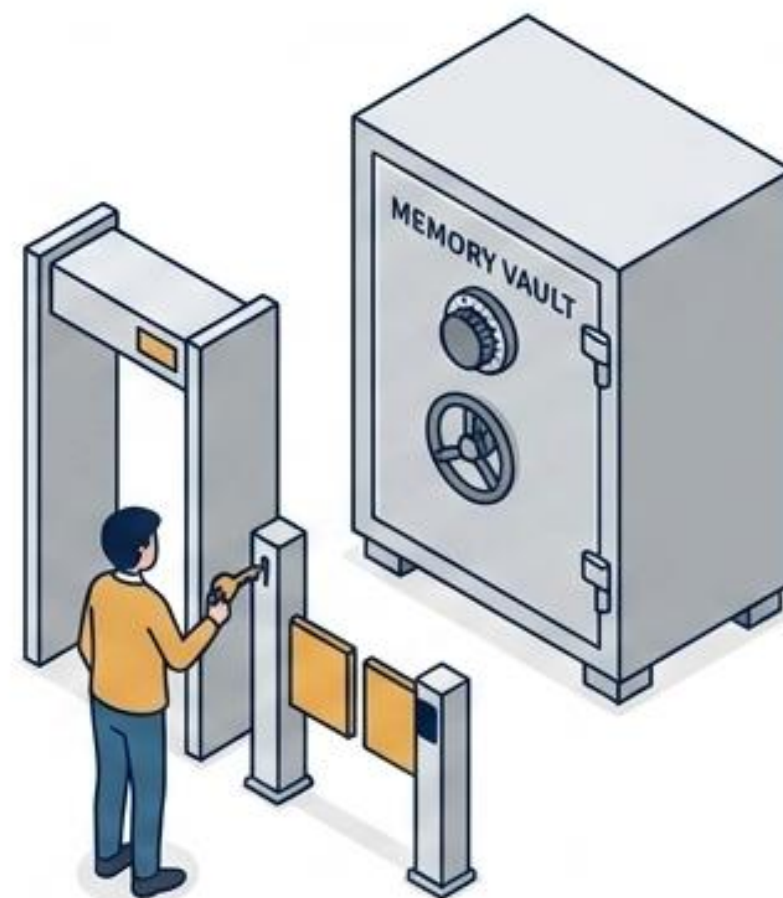
HTTP의 무상태성(Stateless) 극복하기

HTTP 통신은 기본적으로 요청/응답이 끝나면 상태를 기억하지 않습니다.
따라서 클라이언트의 상태 유지를 위해 두 가지 전략을 사용합니다.



쿠키 (Cookie)

데이터 자체를 웹 브라우저(클라이언트)가 보관.
서버에 요청할 때마다 데이터를 지참하여 통과.



세션 (Session)

데이터는 서버의 안전한 저장 공간(웹 컨테이너)에 보관.
클라이언트는 이 공간을 열 수 있는 식별키(JSESSIONID)만 쿠키로 소지.

쿠키(Cookie): 클라이언트 측 데이터 보관소

Set-Cookie: 쿠키이름=쿠키값; Domain=도메인값; Path=경로값; Expires=만료일시;

이름과 값: 데이터 구별자
및 실제 데이터

도메인: 쿠키를 전송할
서버 범위 지정

경로: 쿠키를 공유할
기준 URL 경로 지정

유효시간: 미지정 시 브라우저
종료 시 삭제. setMaxAge(0)은
쿠키 즉시 삭제

특징 및 활용 (Use Cases & Limitations)

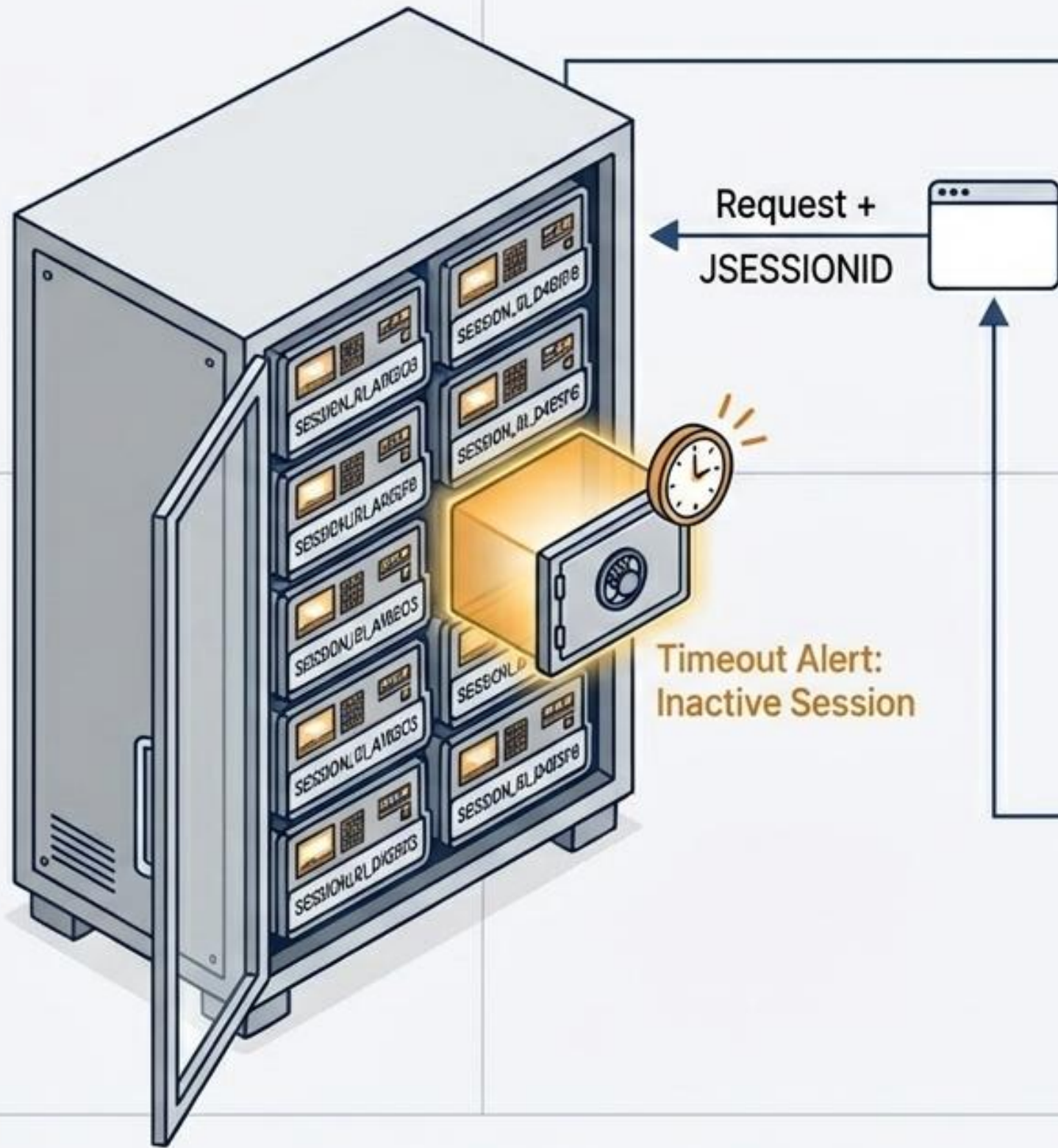
특징 및 활용 (Use Cases)

- 보안이 중요하지 않은 가벼운 데이터 저장
- 아이디 기억하기, 최근 본 상품 내역
- 팝업창 (일주일 간 보이지않기) 제어

한계점 (Limitations)

- 브라우저 개발자 도구를 통해 값 변조 가능성 존재
- 네트워크 전송 중 탈취 위험

세션(Session): 서버 영역의 안전한 저장 공간



작동 원리 (Session Mechanics)

- 웹 브라우저 최초 접속 시 서버는 고유한 세션 객체를 메모리에 생성.
- 서버는 세션 ID를 생성하고, 이를 JSESSIONID 쿠키를 통해 브라우저와 공유하여 사용자를 식별.
- 사용자마다 독립적인 저장 공간 제공.

생명주기 및 메모리 보호 (Lifecycle Management)

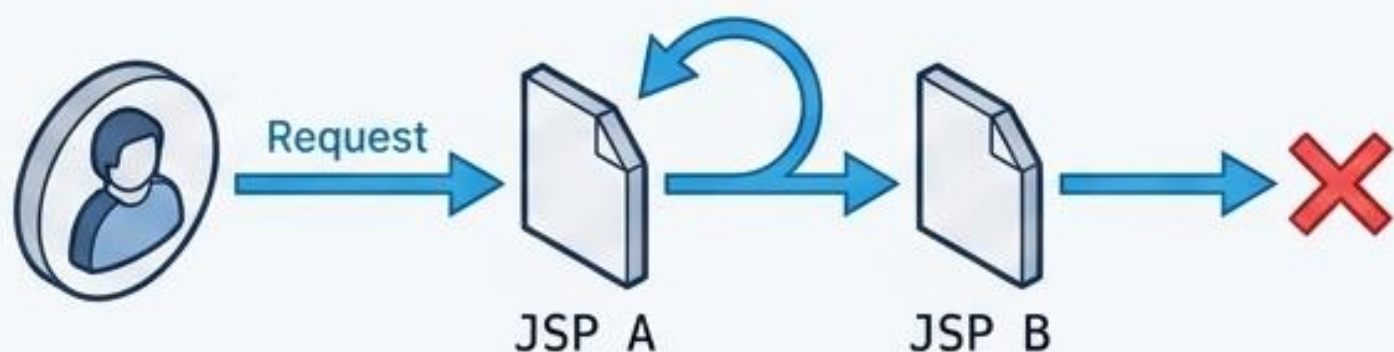
- `session.invalidate()`: 세션 즉시 종료 및 속성 데이터 완전 삭제 (로그아웃 시 사용).
- Timeout 주의: 유효시간을 지정하지 않거나 0/음수로 설정하면 객체가 메모리에 영구 체류하여 메모리 부족(OOM) 현상 발생.
- 시간 설정: `web.xml`의 `<session-timeout>` (분 단위) 또는 `setMaxInactiveInterval()` (초 단위) 사용.

상태 관리 기술 비교 매트릭스

	쿠키 (Cookie)	세션 (Session)
저장 위치	웹 브라우저 (클라이언트)	웹 서버 메모리 (컨테이너)
보안성	낮음 (네트워크 탈취 및 클라이언트 변조 가능) ⚠	높음 (데이터가 서버 내부에 안전하게 존재) ✓
브라우저 의존성	브라우저가 쿠키를 차단하면 사용 절대 불가 ⚠	쿠키 차단 시 URL 재작성 방식으로 우회 가능 ✓
서버 간 공유	도메인 설정을 통해 여러 서버 간 데이터 공유 가능 ✓	서로 다른 웹 어플리케이션(컨텍스트) 간 공유 절대 불가 ⚠
적합한 용도	사용자 편의성 (자동 로그인, 팝업 제어, 장바구니)	민감한 정보 유지 (로그인 인증 상태, 보안 데이터)

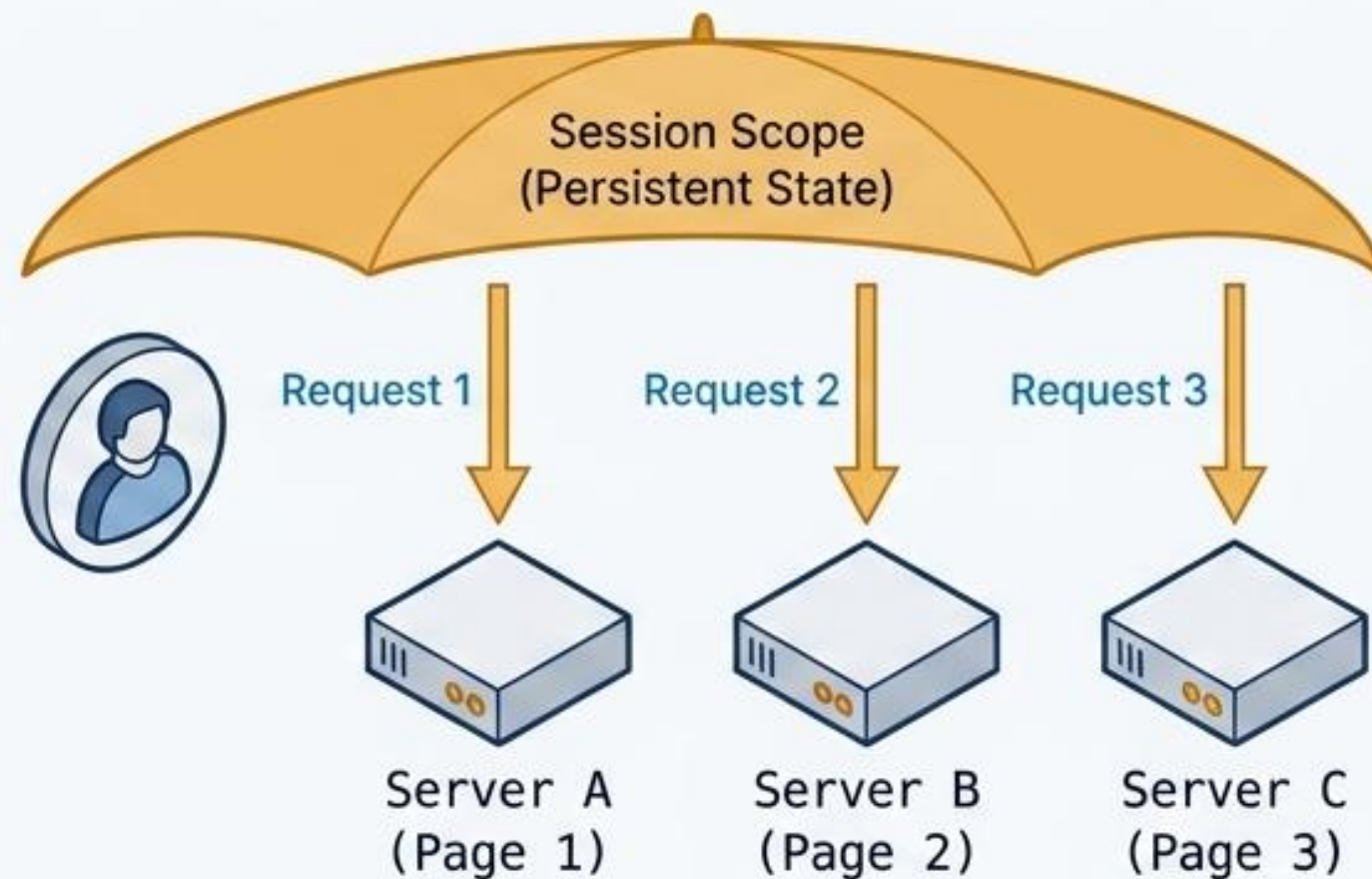
데이터의 생명주기: Request 객체 vs Session 객체

Request Scope (단일 요청)



- 클라이언트가 전송한 하나의 요청을 처리하는 동안에만 생존.
- <jsp:forward> 등을 이용해 페이지가 이동(포워딩)될 때, 여러 JSP 페이지 사이에서 동일한 Request 객체를 공유하여 데이터 전달 가능.

Session Scope (다중 요청)



- 웹 브라우저의 전체 생명주기와 동기화.
- 동일한 브라우저에서 발생하는 여러 번의 개별 요청들 사이에서 데이터를 지속적으로 공유.
- 예 : 페이지를 이동해도 계속 유지되는 로그인 사용자 정보.

JSP 전용 태그: 서버 코드가 HTML로 렌더링되는 방식

JSP 전용 태그는 서블릿 생성 시 특정한 자바 코드로 변환되어 동적 웹페이지를 완성합니다.

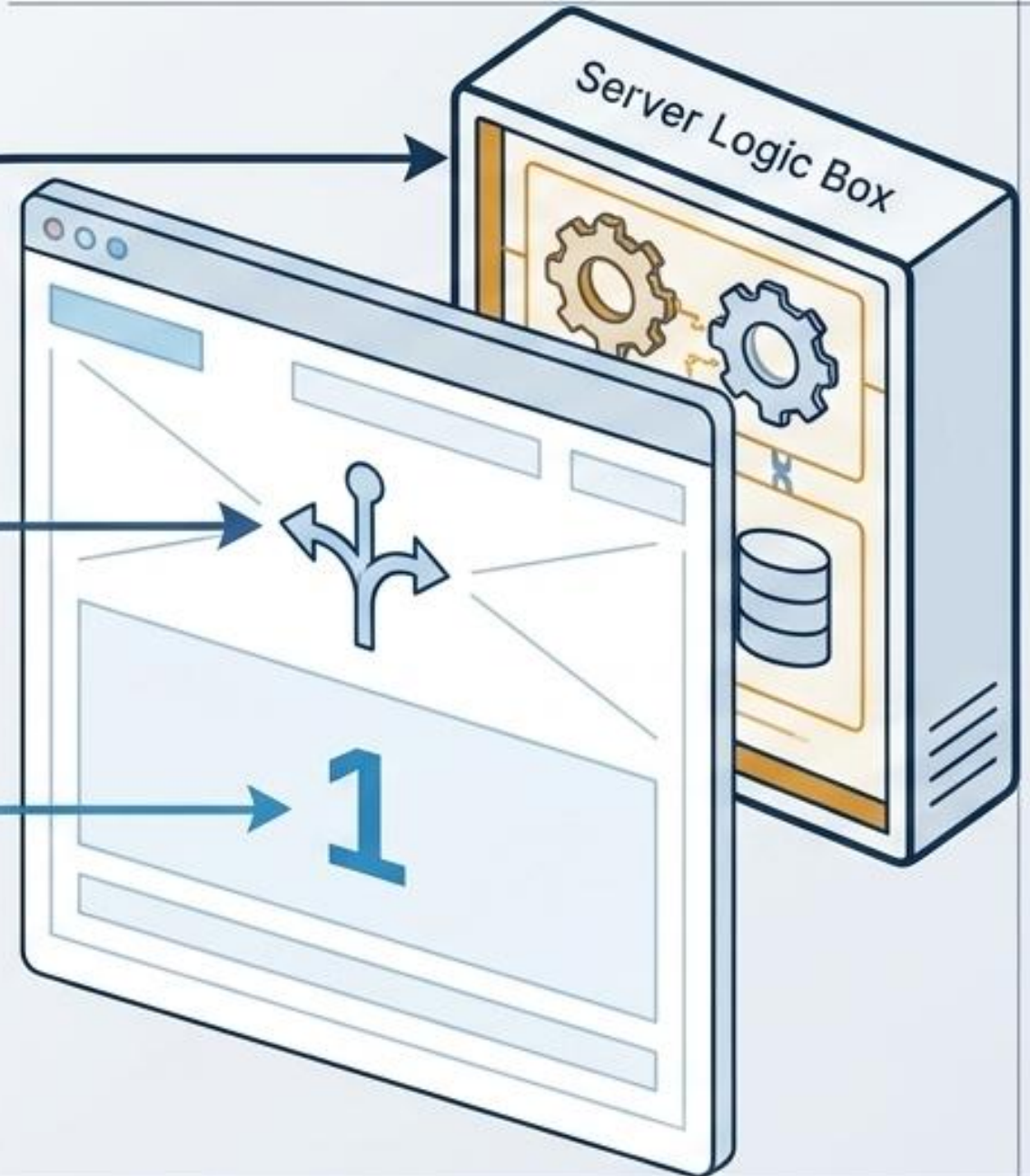
Code-to-Visual Mapping

```
<%!  
    int age = 0; %>  
%>  
  
<%  
    if(i>0) { %>  
%>  
  
<%= ++age %>
```

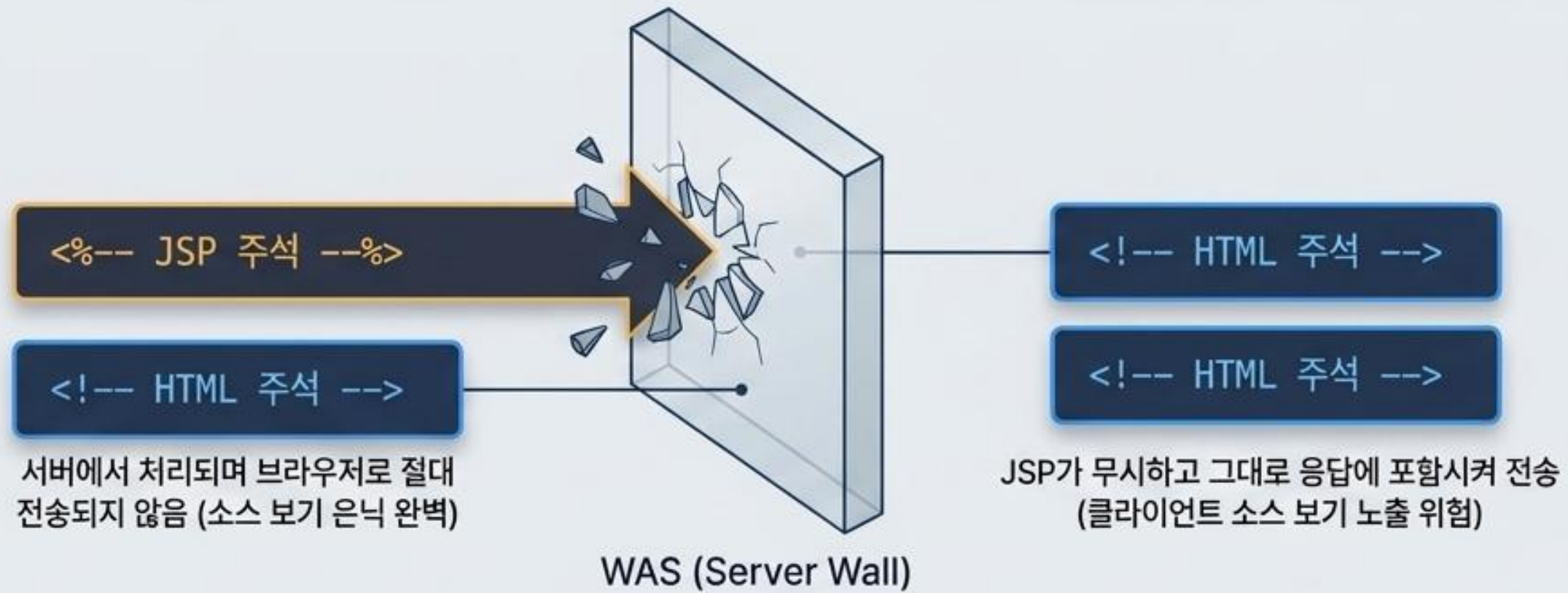
선언문 (Declarations): 서블릿 클래스의 전역 멤버 정의. 화면에 노출되지 않음.

스크립틀릿 (Scriptlet): 실제 로직을 수행하고 흐름을 제어하는 자바 코드 블록.

표현식 (Expressions): 결과값을 문자열로 반환하여 HTML 문서 내에 직접 출력.



지시자(Directives)와 주석(Comments)의 동작 원리



Page 지시자 (Page Directives) `<%@ page %>`

contentType



출력 데이터의 MIME 타입과
응답 결과의 최종 인코딩 지정.
예: text/html; charset=UTF-8

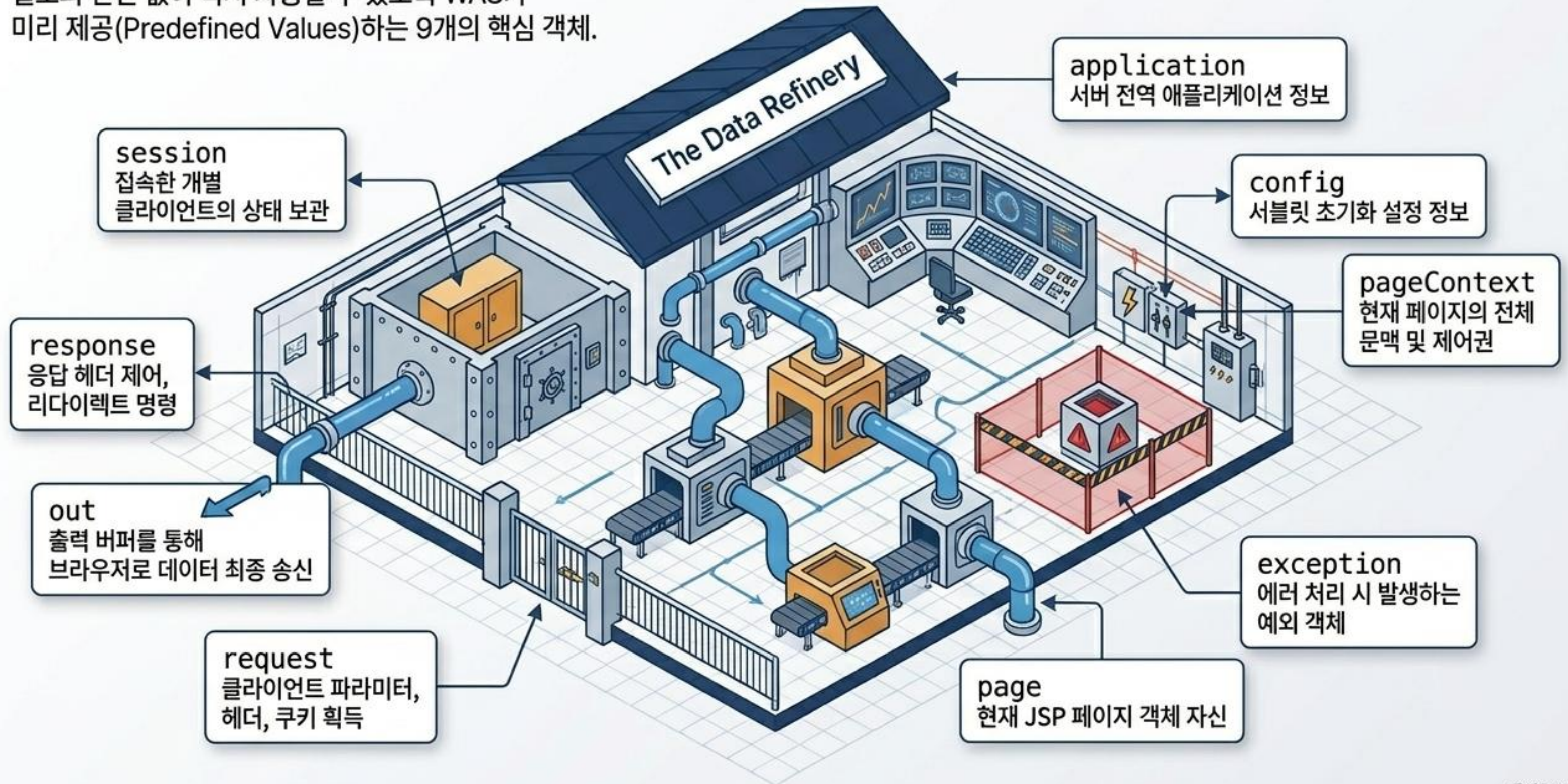
pageEncoding



JSP 파일 자체를 작성할 때 에디터에서
사용한 원본 문자 인코딩 설정.

JSP 내장 객체(Implicit Objects) 지형도

별도의 선언 없이 즉시 사용할 수 있도록 WAS가
미리 제공(Predefined Values)하는 9개의 핵심 객체.



코드의 진화: Action Tag와 표현 언어(EL)

자바 코드(Scriptlet)와 HTML이 뒤섞이는 스파게티 코드를 해결하고, 유지보수성을 높이기 위해 선언적 방식이 도입되었습니다.

과거: Scriptlet 혼용 방식 (가독성 저하)

```
<%  
    String name = memberDto.getUserName();  
%>  
  
<html>  
<body>  
    <h1>Welcome, <%= memberDto.getUserName() %>!</h1>  
</body>  
</html>
```

Diagram illustrating the spaghetti code structure where Java code (Scriptlet) is mixed with HTML. A callout box highlights the repeated use of `<%= memberDto.getUserName() %>` in both the scriptlet and the HTML output, showing the lack of abstraction and readability.

1 Action Tag: `<jsp:forward>`, `<jsp:useBean>` 등 자바 인스턴스 관리 및 흐름 제어를 태그 형태로 깔끔하게 처리.

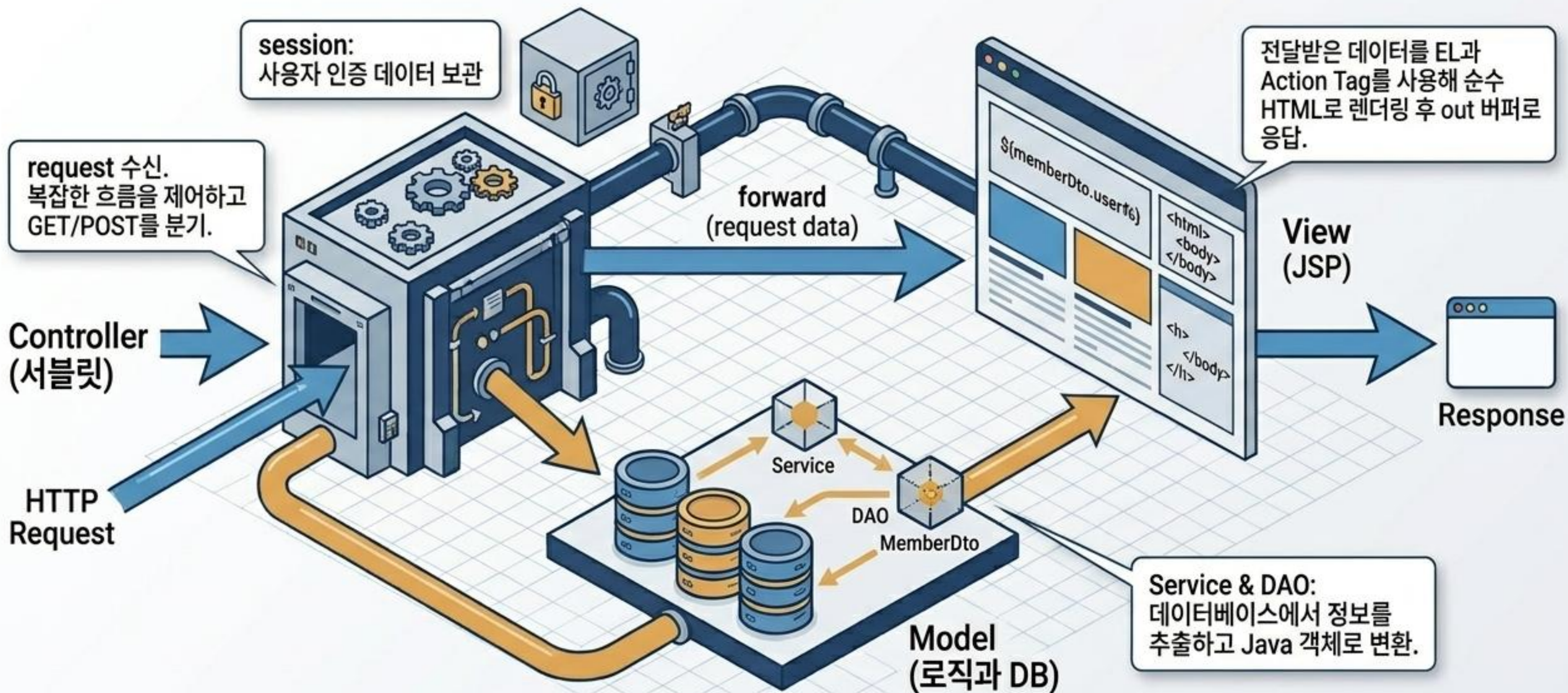
현대: Action Tag & EL (직관적이고 선언적인 구조)

```
<jsp:useBean id="memberDto" class="com.example.MemberDto" />  
<jsp:setProperty name="memberDto" property="*" />
```

```
<html>  
  <body>  
    <h1>Welcome, ${memberDto.userName}!</h1>  
  </body>  
</html>
```

2 EL - Expression Language: `${memberDto.userName}` 문법을 통해 Java Beans 데이터에 직관적으로 접근. 세미콜론과 복잡한 메서드 호출 제거.

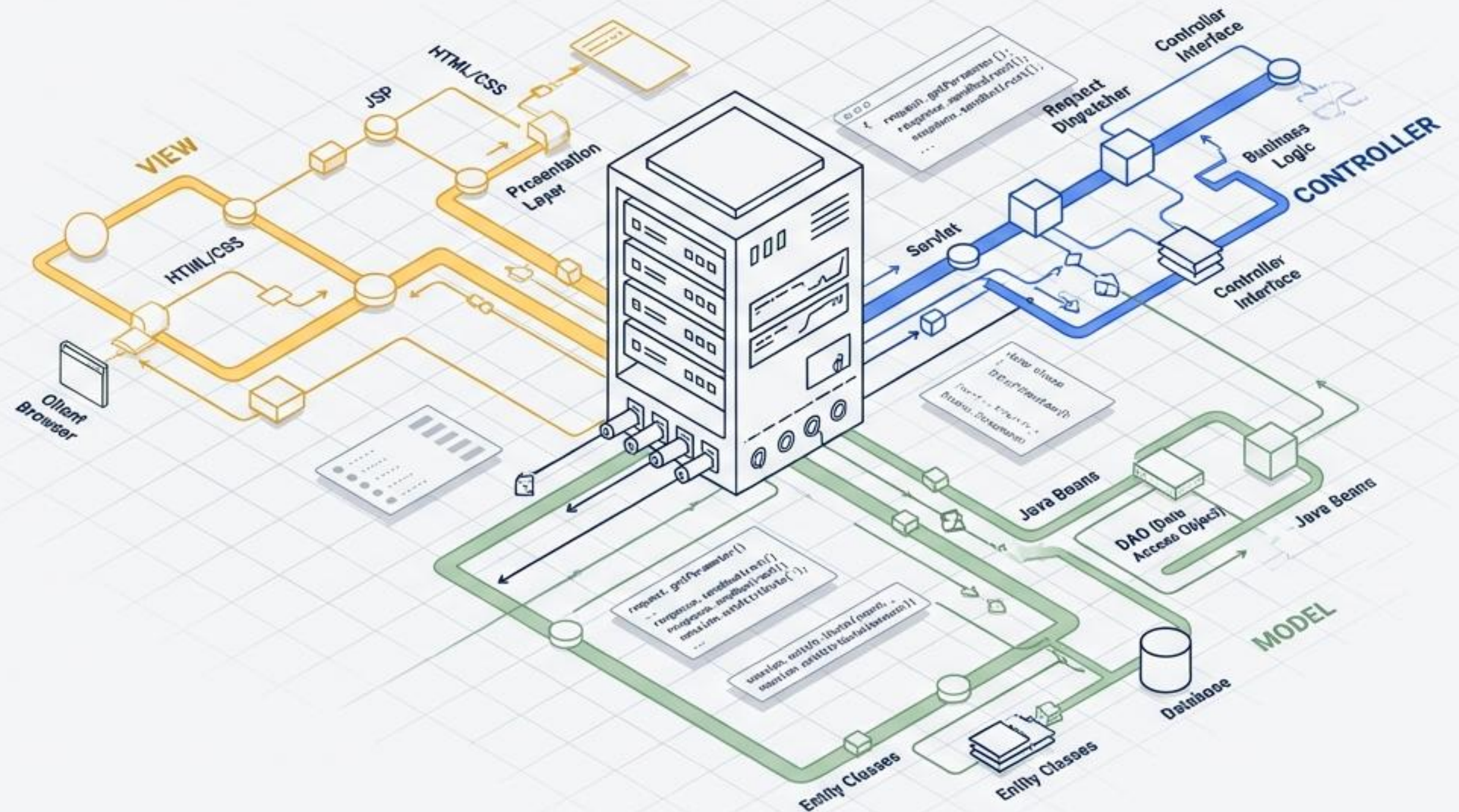
데이터 정제소의 완성: MVC (Model-View-Controller) 아키텍처



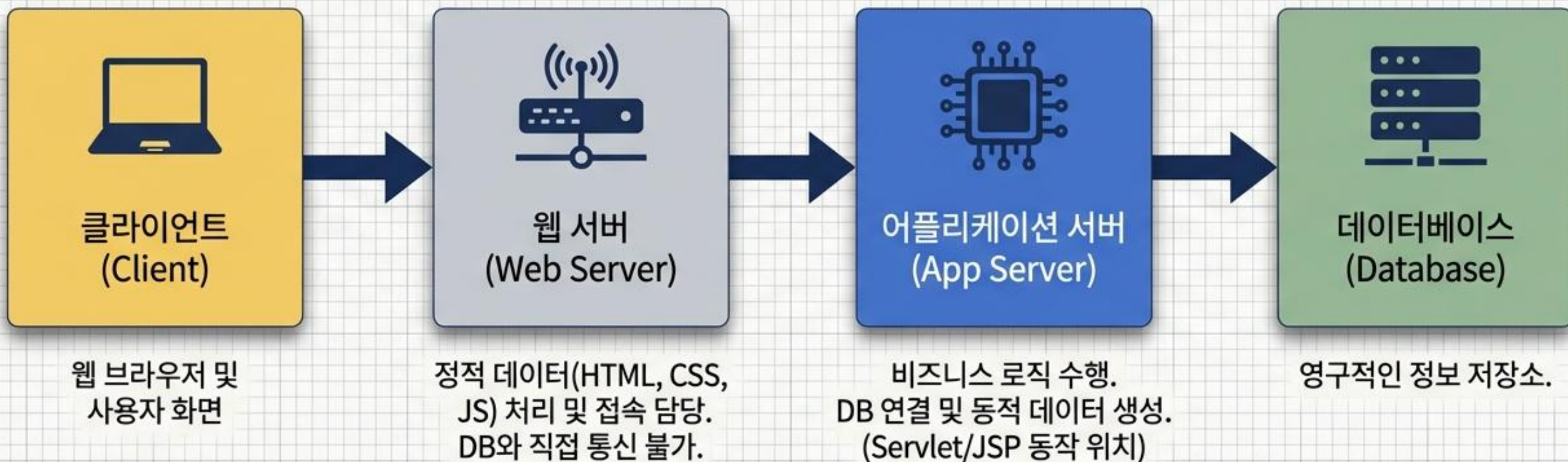
최종 요약: 서블릿의 강력한 로직 제어력과 JSP의 직관적인 UI 렌더링 능력을 분리 및 결합하는 것이 완벽한 웹 어플리케이션 설계의 핵심입니다.

Java 웹 아키텍처 설계도

Servlet과 JSP의 기초부터 MVC 패턴의 진화까지



웹 아키텍처의 맥락: 프론트엔드와 DB의 간극



프론트엔드는 DB와 직접 통신할 수 없습니다. 백엔드(Java Servlet/JSP)가 이 둘을 연결하는 핵심 토피바퀴 역할을 합니다.

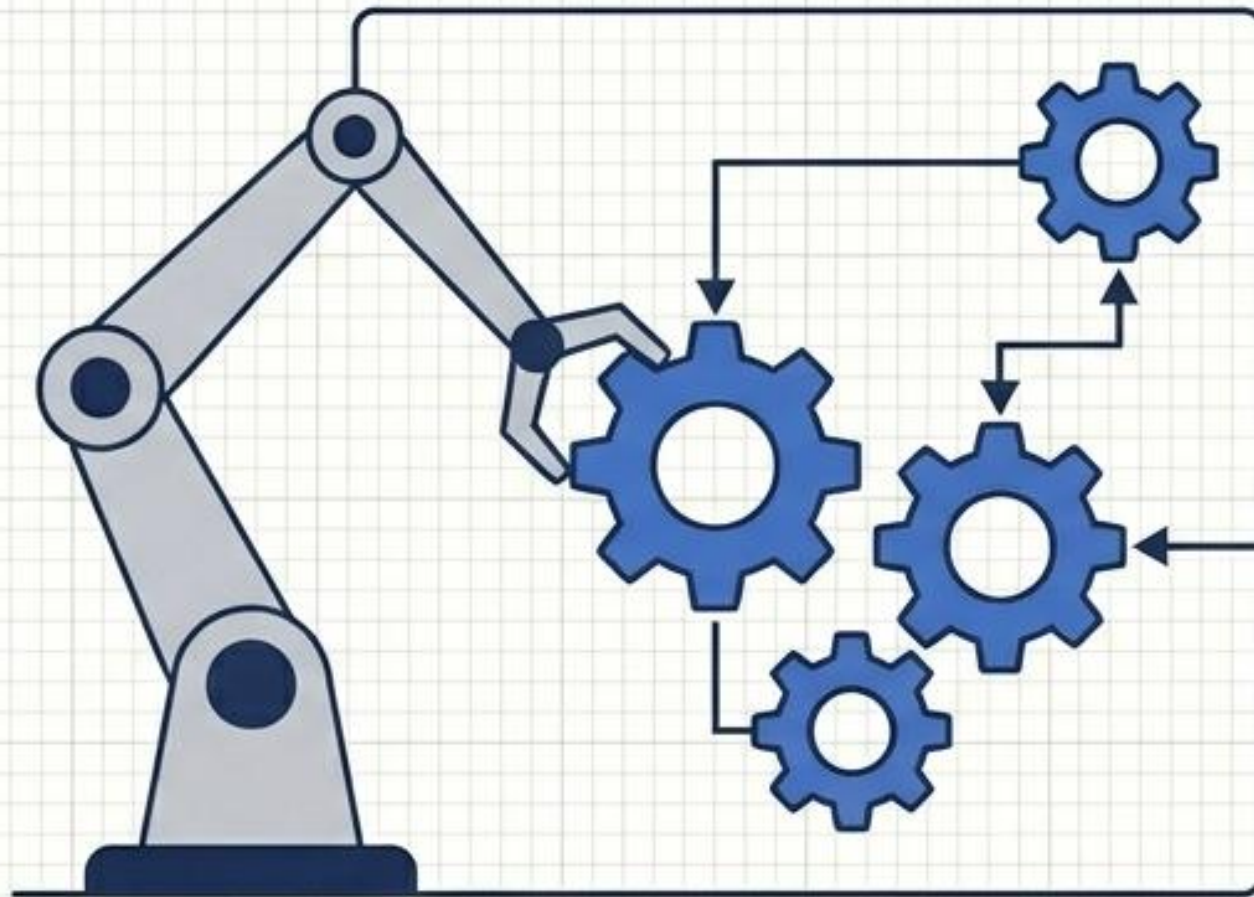
Servlet: 자바 속에 들어간 HTML

자바를 사용하여 동적인 웹페이지를 생성하는 서버측 프로그램.
자바 코드 안에 HTML이 포함된 구조를 가집니다.

Code Editor

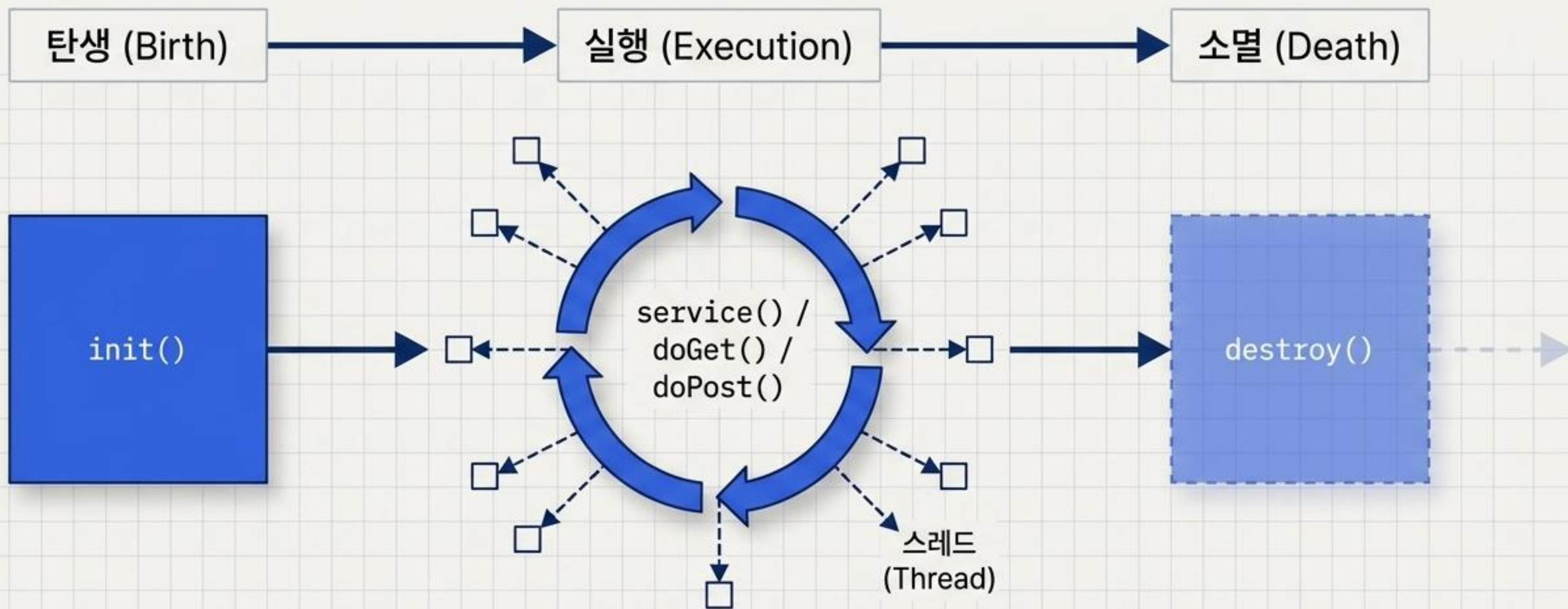
```
out.print("<html>");  
out.print("<body>");  
out.print("  
out.print("<h1>Hello Servlet!</h1>");  
out.print("</body>");  
out.print("</html>");
```

서블릿 컨테이너 (Servlet Container)



일반적인 Java와 달리 main() 메서드가 없습니다.
객체의 생성, 초기화, 소멸을 개발자가 아닌
'서블릿 컨테이너'가 전적으로 통제합니다.

Servlet 생명주기 (Lifecycle Flowchart)



객체 생성 및 초기화.
(서버 실행 후 최초 1회만 발생)

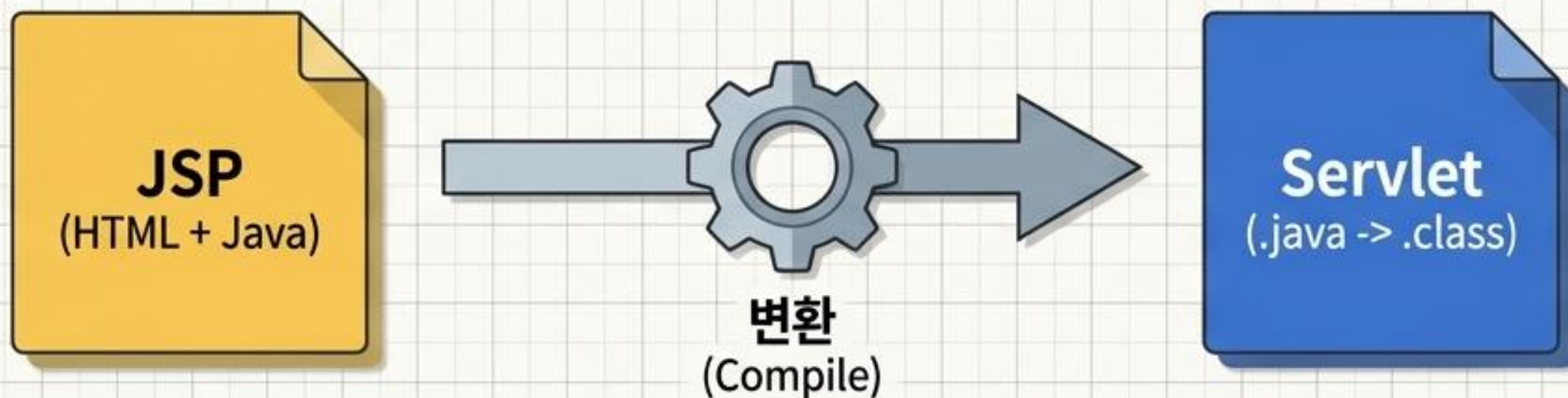
클라이언트 요청마다 반복.
컨테이너가 자동으로 스레드를 생성하여
병렬 처리하므로 속도 저하를 방지합니다.

객체가 더 이상 필요 없을 때
메모리에서 제거. (1회 발생)

단일 객체 인스턴스를 유지하며 스레드만 생성하여 요청을 처리하는 것이 Servlet의 핵심 성능 비결입니다.

JSP: HTML 속에 들어간 자바

JSP(Java Server Page)는 HTML 내에 자바 코드를 삽입하는 언어입니다.
실행 시 자동으로 자바 서블릿으로 변환되어 동작합니다.



디자인 편의성

HTML 표준을 따르므로 프론트엔드 작업과 웹 디자인이 훨씬 수월합니다.



JSTL / 커스텀 태그

복잡한 자바 코드 없이 태그만으로 로직(반복, 조건) 기술이 가능합니다.



생산성 향상

HTML 텍스트 문자열을 일일이 자바 out.print()로 감쌀 필요가 없습니다.

JSP 문법 해부도 (Annotated Code Anatomy)

Code Editor

`<%@ page language="java" contentType="text/html" %>`

`<%-- 사용자 데이터 출력 페이지 --%>`

`<%! int userCount = 0; %>`

`<%
 userCount++;
 request.setAttribute("count", userCount);
%>`

`<h2>총 방문자: <%= userCount %></h2>`

지시자 (Directive): 컨테이너에게 페이지 처리 방법, include, taglib 등의 정보를 제공.

주석 (Comment): 브라우저에 노출되지 않는 JSP 전용 주석.

선언 (Declaration): 멤버 변수나 메서드를 선언.

스크립트릿 (Scriptlet): 매 요청마다 호출되는 영역. 변환 시 service() 내부에 배치됨.

표현식 (Expression): 브라우저에 출력할 데이터. (세미콜론 생략 필수)

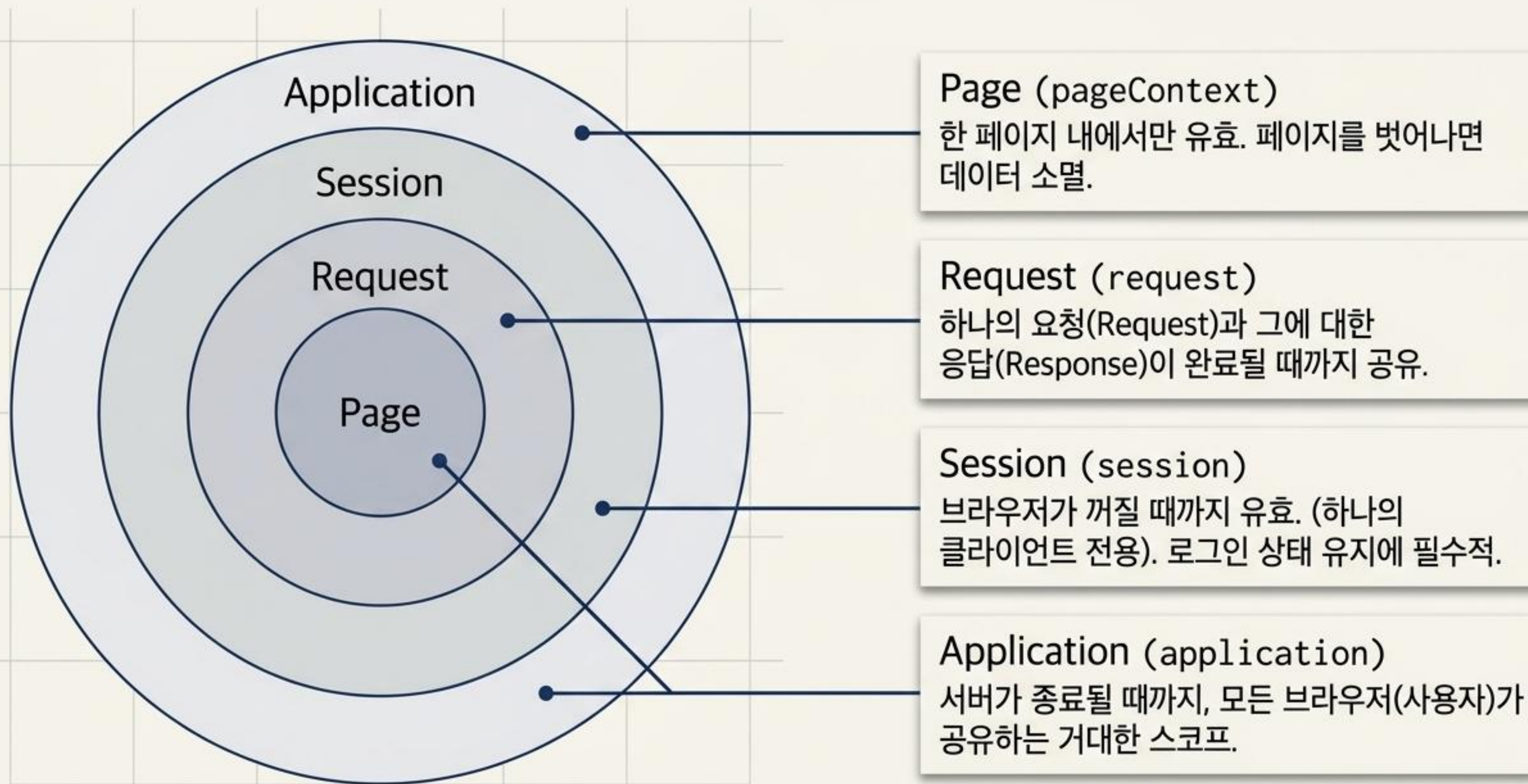
JSP 기본 객체 (9 Implicit Objects)

new 선언 없이 즉시 사용 가능한 도구들

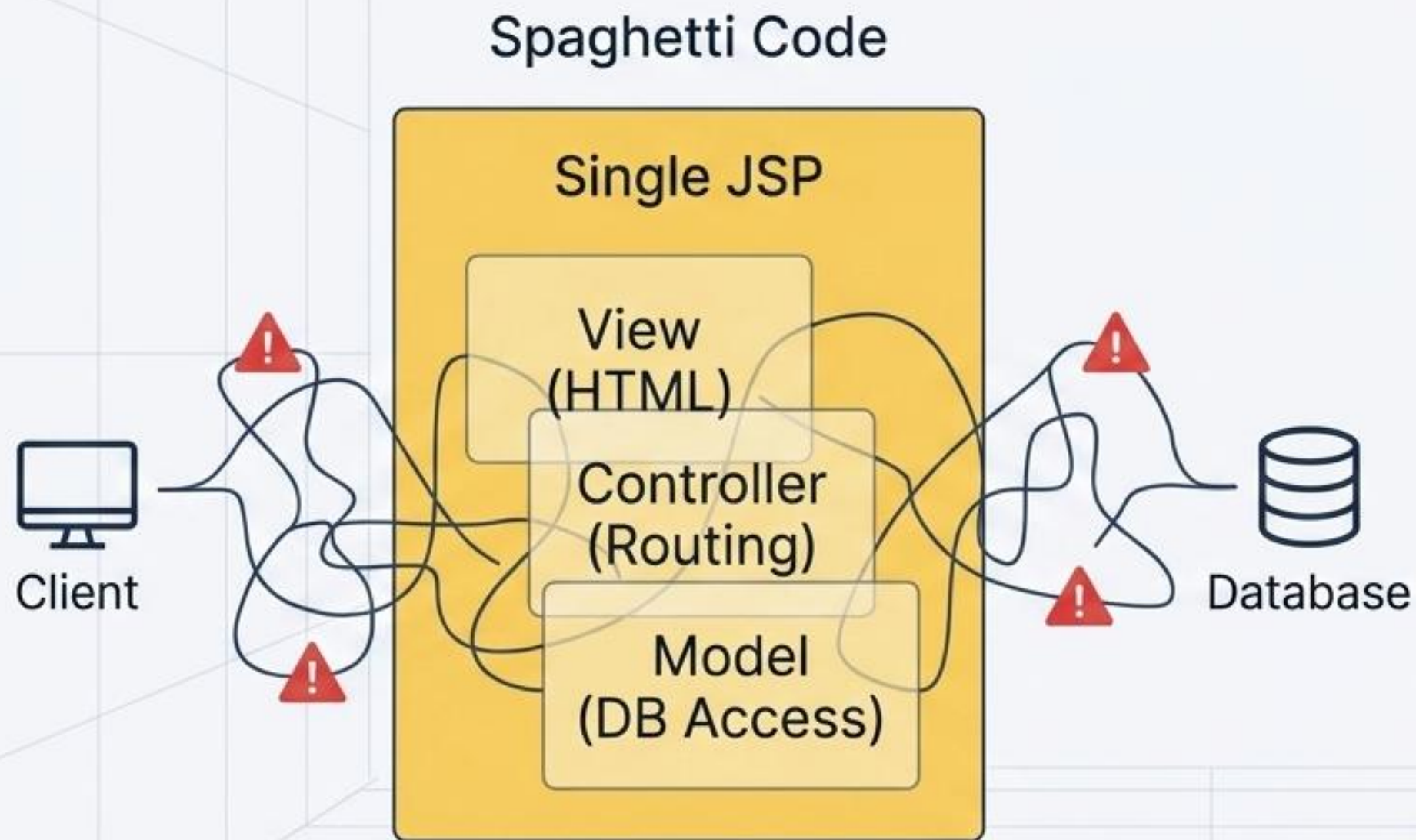
입출력 및 제어 (I/O & Control)		
request1	response2	out3
사용자 입력 및 브라우저 요청 정보 읽기	사용자 요청에 대한 응답 반환 로직 처리	브라우저로 데이터를 전송하는 출력 스트림
데이터 스코프 (Context & Scope)		
session4	application5	pageContext6
브라우저 당 1개 할당 (예: 로그인 유지)	서버 전체 공유 자원 (예: 누적 방문자 수)	기본 객체 접근 및 페이지 포워드/인클루드
환경 및 오류 (Config & Error)		
config7	exception8	9
현재 JSP의 초기화 환경 처리	오류 발생 시 정보 전달 (isErrorPage='true' 필요)	

page: 현재 JSP 페이지 참조 (this)

데이터 생명주기: 4가지 스코프 (Concentric Scopes)



아키텍처의 한계: Model 1의 딜레마



혼돈의 코드

프론트엔드 코드(View)와 백엔드 자바 로직이 뒤섞여 복잡도 극대화.

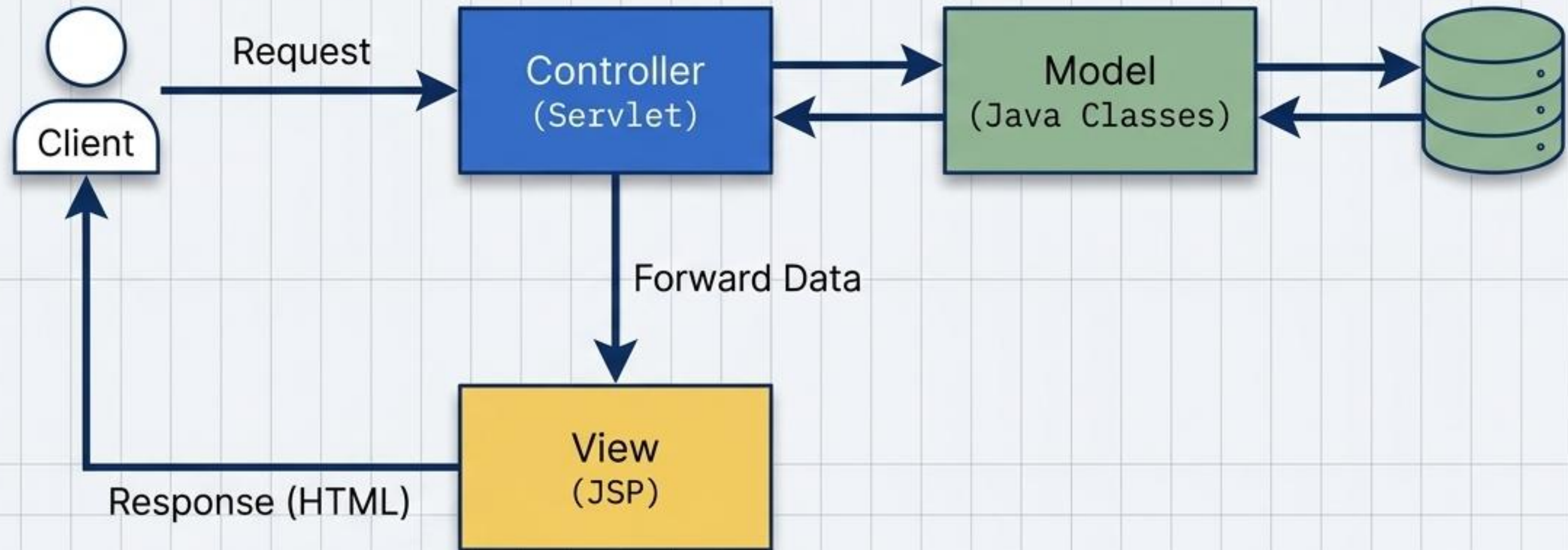
분업 불가

디자이너와 백엔드 개발자가 동일한 파일을 수정해야 하므로 협업이 극도로 어려움.

유지보수 재앙

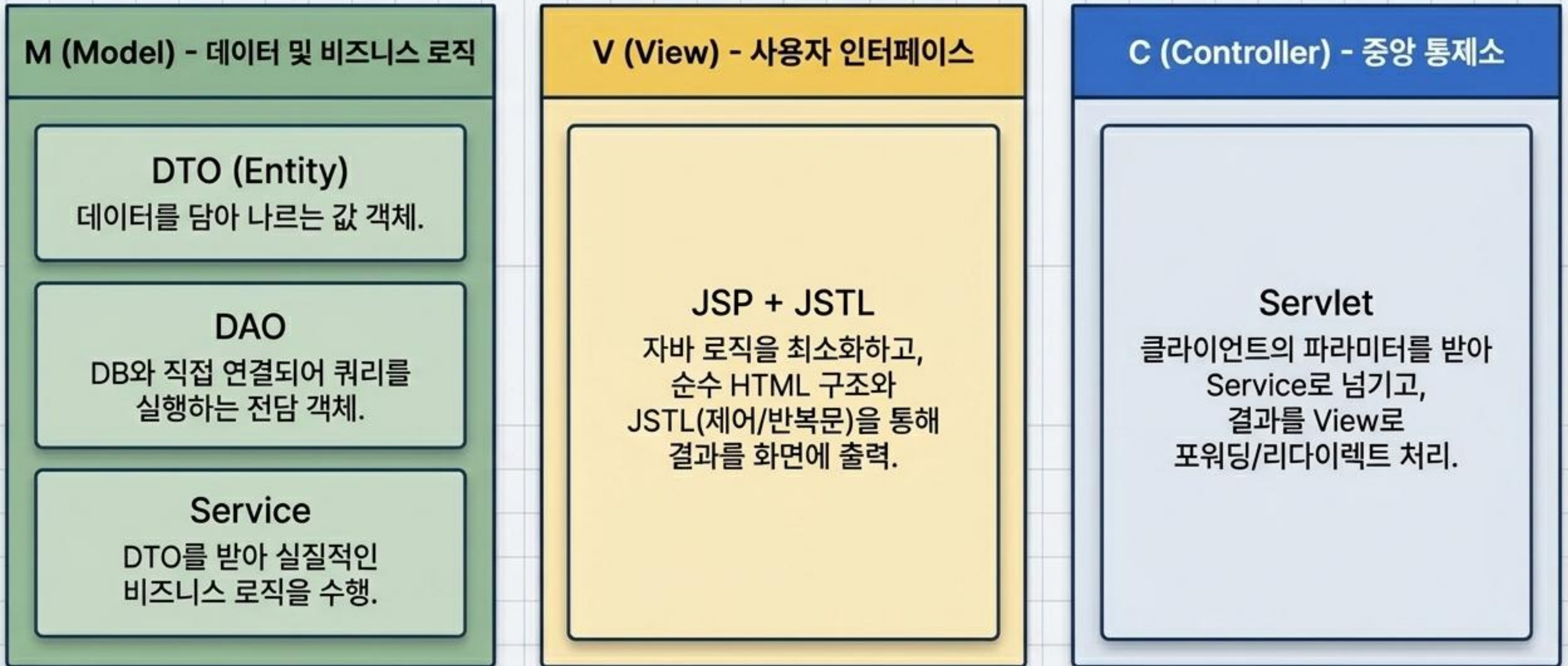
프로젝트 규모가 커질수록 수정이 어렵고, 프레임워크 등 신기술 도입이 불가.

질서의 부여: Model 2 (MVC 패턴)



클라이언트 요청 처리는 **Servlet**이, 로직 처리는 **Java Class**가,
화면 출력은 **JSP**가 전담하는 완벽한 역할 분담 구조.

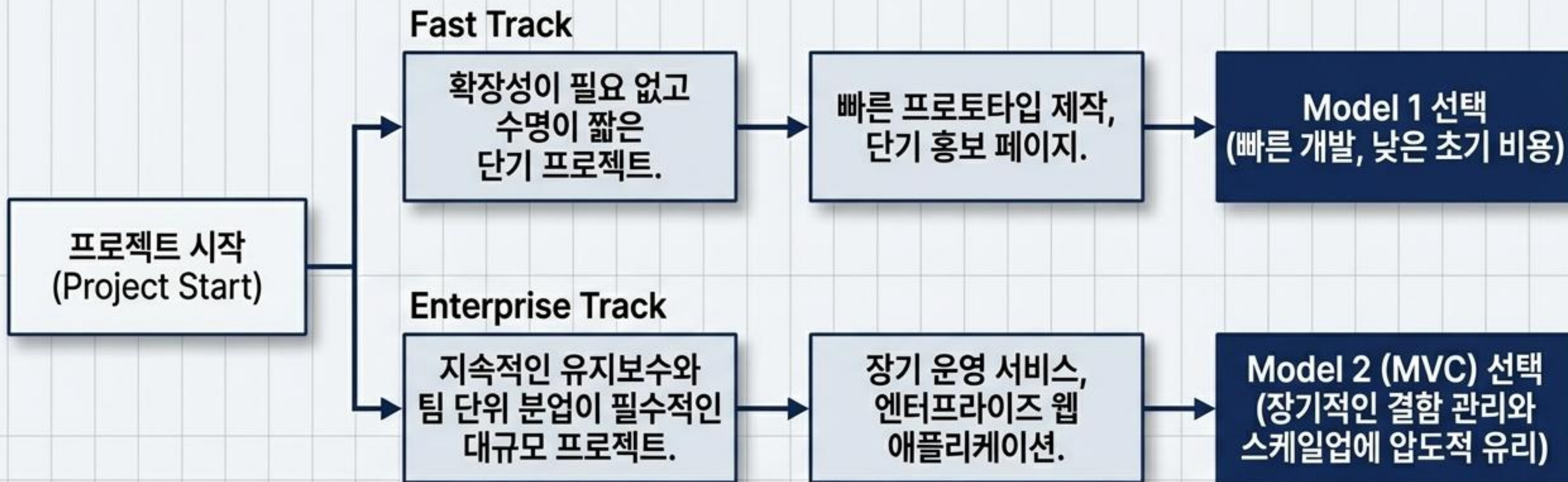
MVC 컴포넌트 해부도 (The Synthesis Map)



아키텍처 진단 매트릭스: Model 1 vs MVC (Model 2)

비교 항목 (Dimension)	Model 1	MVC (Model 2)
구조 (Core Mechanism)	JSP 단독 처리 (뷰+로직 혼재)	역할 완벽 분리 (M, V, C)
코드 복잡도 (Code Complexity)	매우 높음 (스파게티 코드)	낮음 (구조화되고 깔끔함)
유지보수 및 확장성 (Maintenance & Scaling)	나쁨	매우 뛰어남
개발 속도 및 비용 (Dev Speed/Cost)	초기 개발 빠름, 비용 적음	초기 진입장벽 높음, 비용 증가
협업 (Team Collaboration)	프론트/백엔드 분업 매우 어려움	화면단과 로직단 완전 분리로 분업 용이

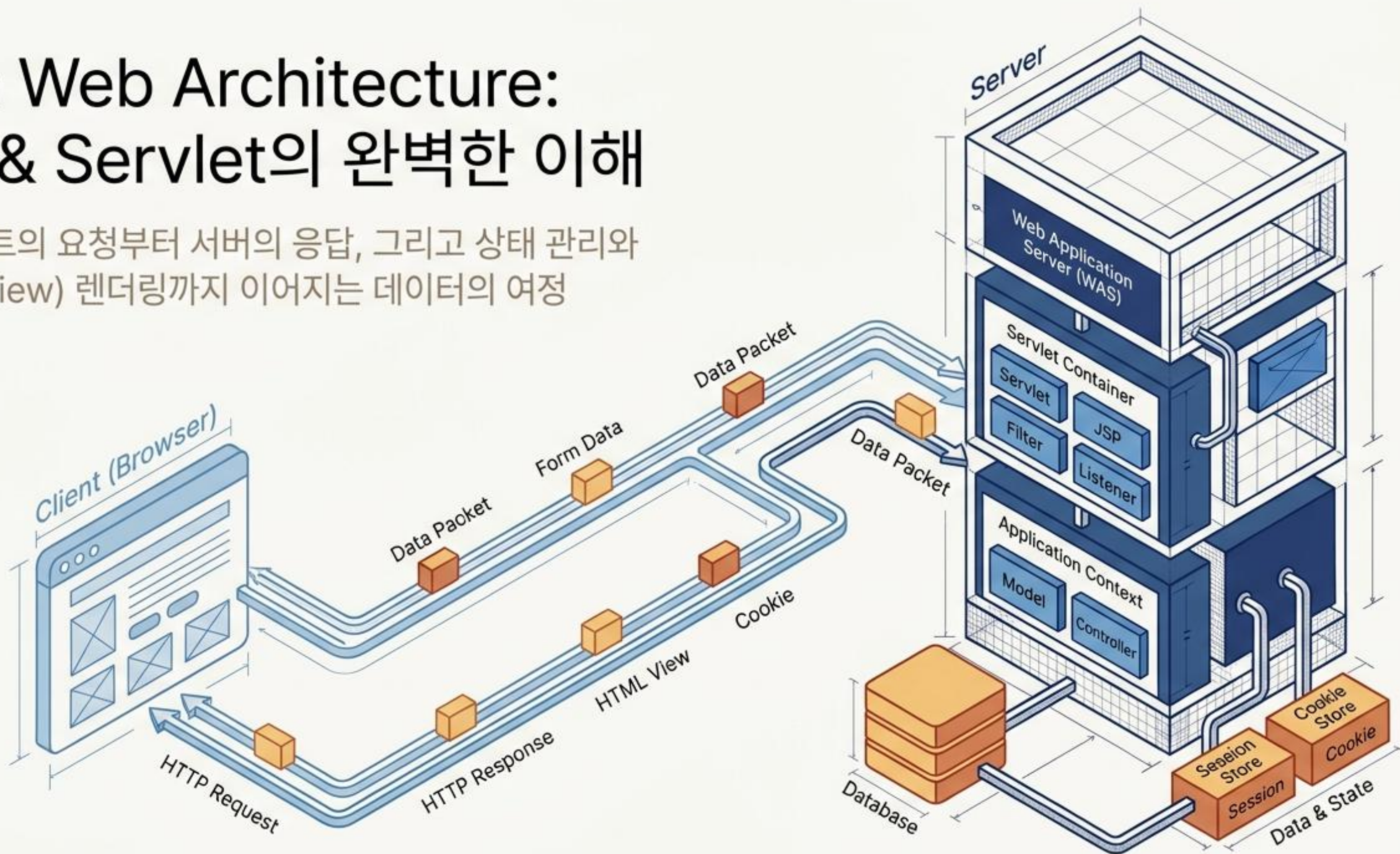
실전 아키텍처 선택 가이드 (Decision Framework)



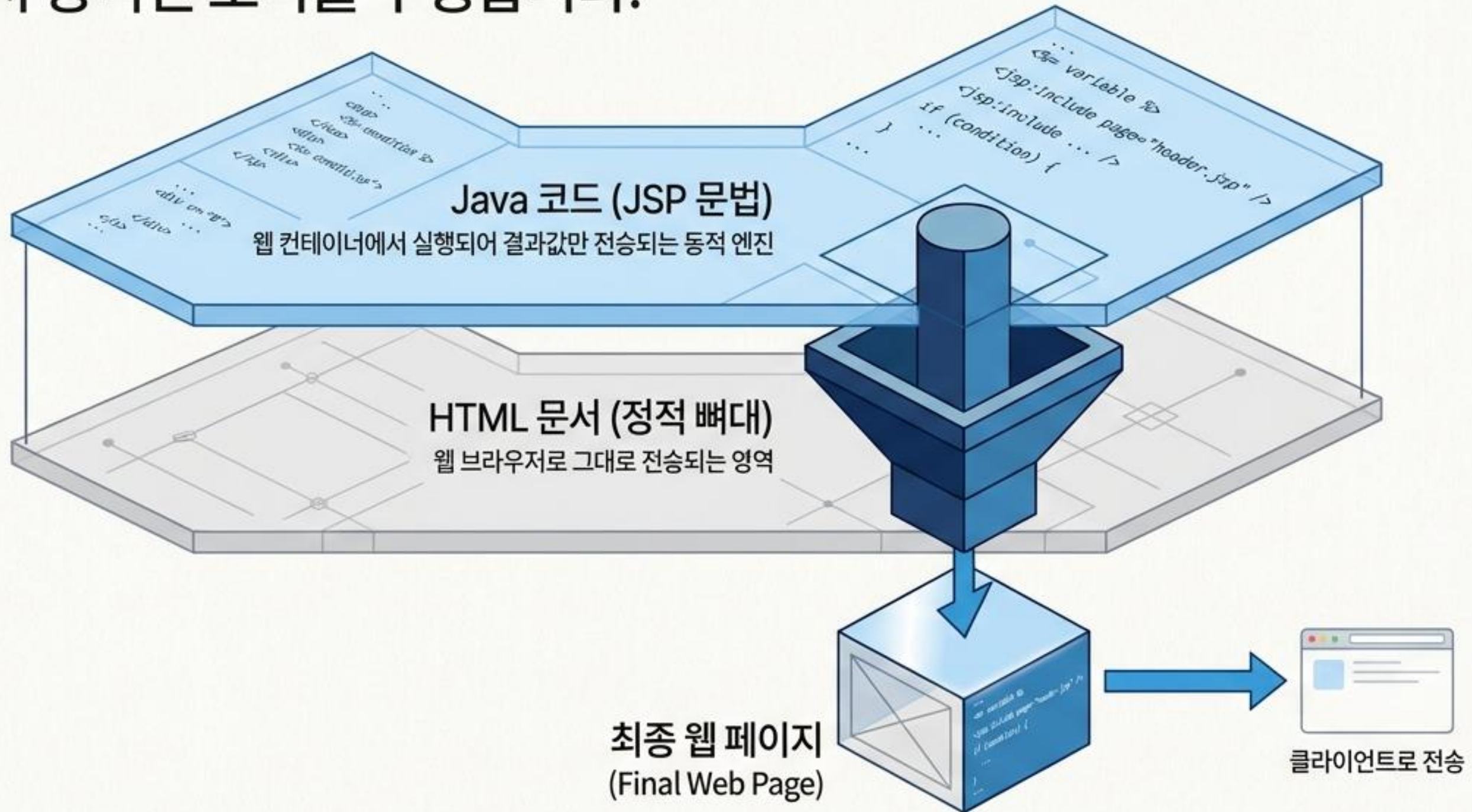
완벽한 하나의 모델은 없습니다. 상황과 규모에 맞는
상황과 규모에 맞는 아키텍처를 선택하는 것이 백엔드 설계의 진정한 시작입니다.

Java Web Architecture: JSP & Servlet의 완벽한 이해

클라이언트의 요청부터 서버의 응답, 그리고 상태 관리와
모던 뷰(View) 렌더링까지 이어지는 데이터의 여정



웹 애플리케이션 구현 시, HTML 문서 사이에 JSP 코드가 삽입되어 서버 측에서 동적인 로직을 수행합니다.



목적에 따라 다르게 조립하는 JSP 문법의 해부학

```
<%@ page language="java" %>
```

```
<%! int count = 0; %>
```

```
<% count++; out.println("Hello"); %>
```

```
<%= count %>
```

지시자 (Directive)

<%@ %>

웹 컨테이너가 JSP 페이지를 서블릿 클래스로 변환할 때 필요한 설정 정보 기술

선언부 (Declaration)

<%! %>

전역 변수나 메서드를 선언하는 공간

스크립틀릿 (Scriptlet)

<% %>

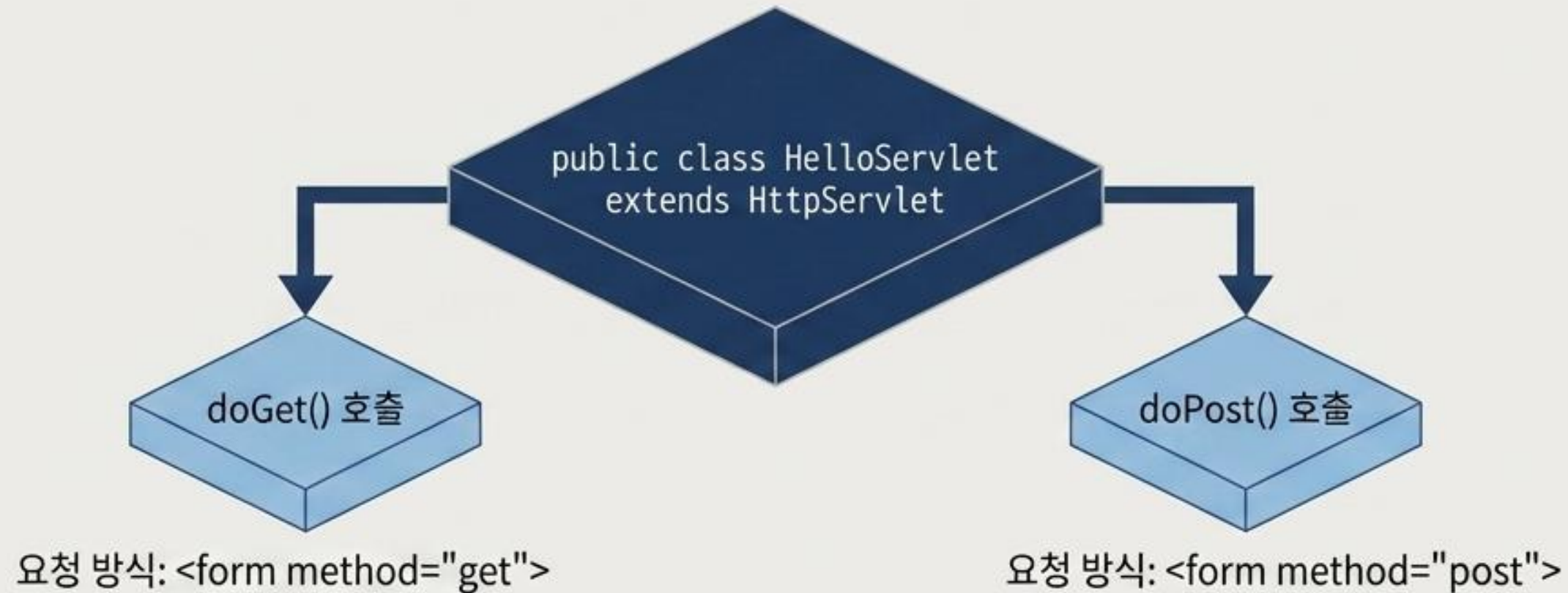
실제 실행될 자바 명령문이 위치하는 핵심 로직 영역

익스프레션 (Expression)

<%= %>

연산식, 상수, 변수 등의 결과값을 화면에 즉시 출력

모든 비즈니스 로직을 제어하는 핵심 엔진: Servlet



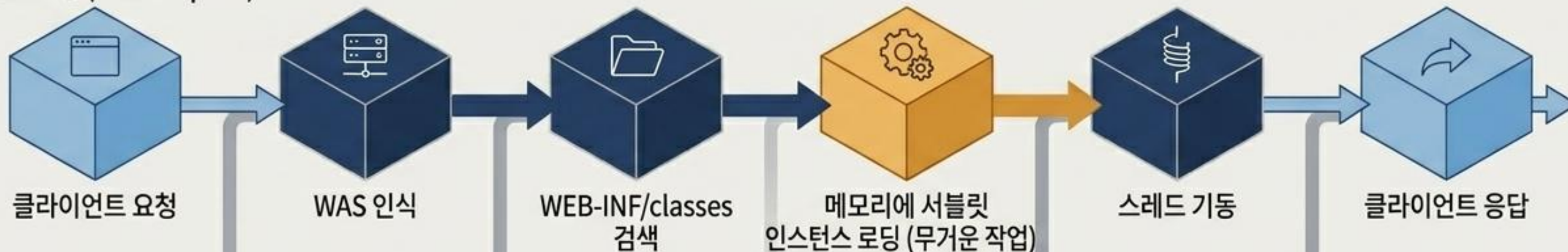
GET 방식	POST 방식
주소창을 통한 전송	HTML Header를 통한 암호화 전송
보안 취약	보안 우수
255자 이하 소용량 데이터에 적합	대용량 데이터 전송에 적합

응답 방식 결정: `response.setContentType("text/html;charset=utf-8");`

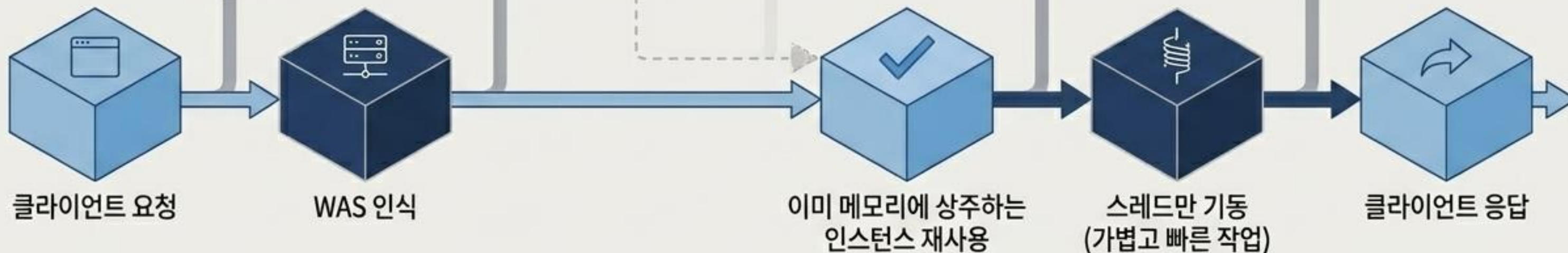
서블릿은 왜 빠를까? 스레드 기반의 메모리 로딩 메커니즘

서블릿은 요청될 때마다 객체를 새로 생성하지 않습니다.
컨테이너가 스레드만 분기하여 기존 인스턴스를 활용하므로 속도가 압도적으로 빠릅니다.

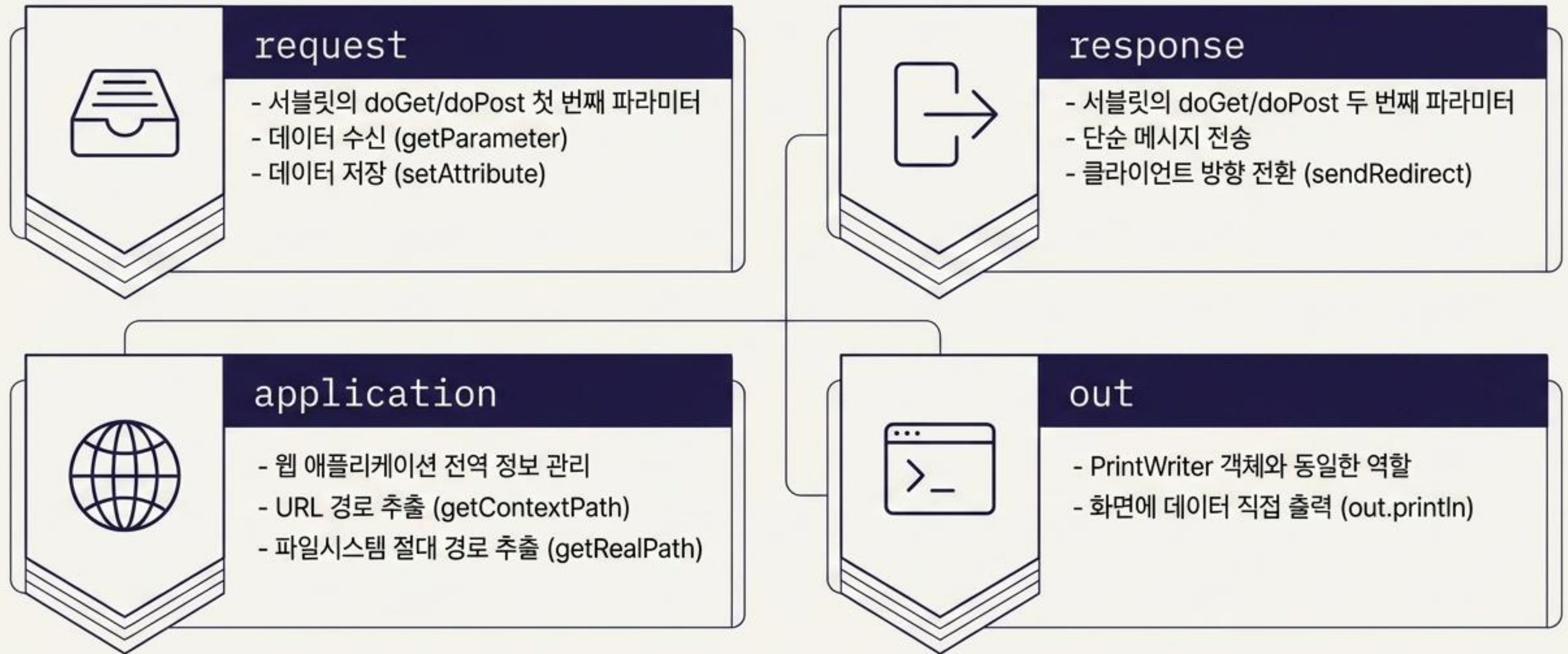
최초 요청 (First Request)



이후 요청 (Subsequent Requests)



별도의 선언 없이 즉시 호출하는 데이터 통로: 내장 객체



HTTP의 치명적 한계(Stateless)와 상태 관리의 필요성



웹 브라우저는 서버로부터
응답을 받고 나면 연결을 즉시
끊어버립니다.

(서버 과부하 방지 목적)



문제점 (Stateless):

- 사용자의 로그인 상태나
장바구니 데이터를 기억할 수
없음.



해결책:

이 끊어진 기억을 이어주는 두 가지 마
스터키 가 바로 Cookie와 Session입니다.

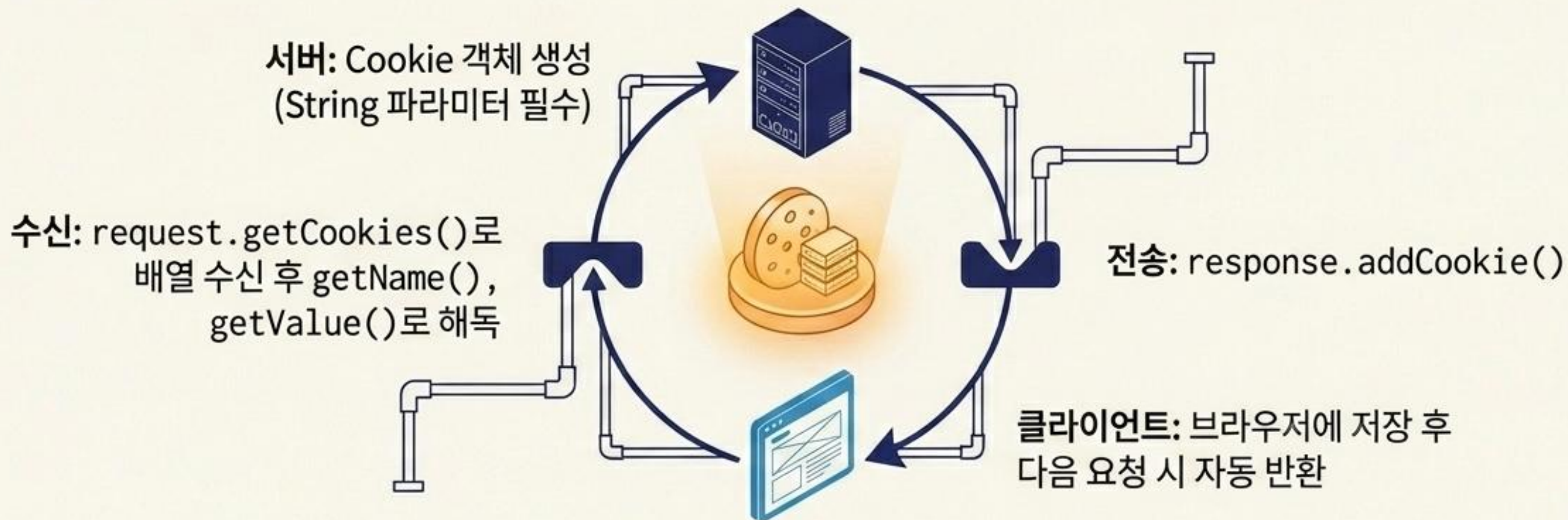


Cookie

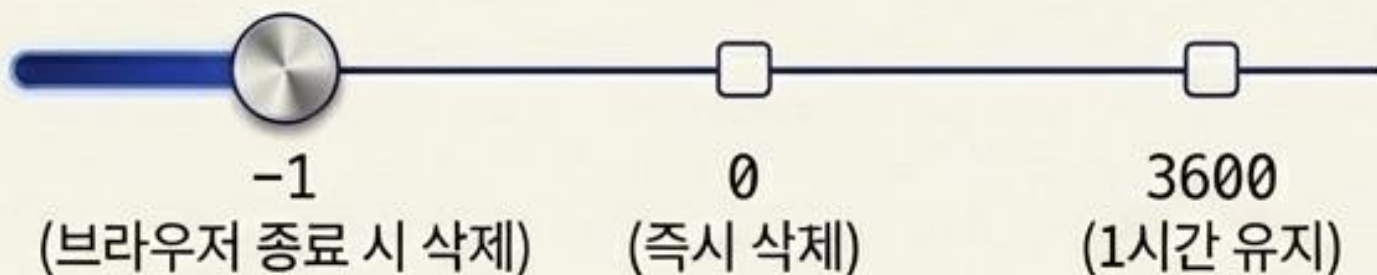


Session

클라이언트의 브라우저에 저장되는 기억 장치: 쿠키 (Cookie)



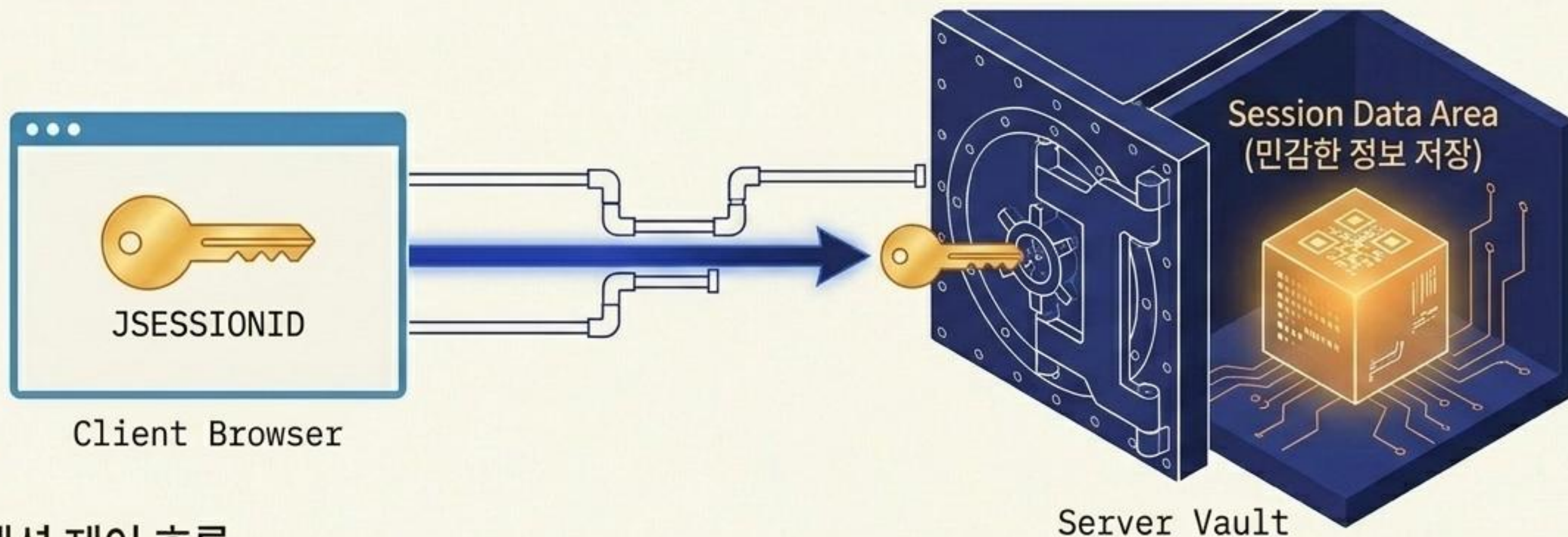
수명 제어 (setMaxAge)



보안 및 범위 제어

setPath	/JSP/cookie01/ (특정 경로로만 전송)
setDomain	.cookie.co.kr (특정 도메인 공유)

서버 내부의 안전한 금고와 단 하나의 열쇠: 세션 (Session)



서블릿 세션 제어 흐름

Open (획득)	Store (저장)	Retrieve (호출)	Destroy (종료)
<code>request.getSession()</code>	<code>session.setAttribute("ID", "chaeng")</code>	<code>(String)session.getAttribute("ID")</code>	<code>session.invalidate()</code>
세션 객체 생성 및 획득	데이터 저장 및 수정	데이터 호출 (반드시 캐스팅 필요)	전체 세션 강제 종료 (로그아웃) *특정 삭제: <code>session.removeAttribute()</code>

최적의 상태 관리 전략 선택: Cookie vs. Session 진단표

비교 기준	Cookie	Session
저장 위치 (Location)	클라이언트 웹 브라우저	웹 서버의 메모리 영역 (안전 금고)
보안성 (Security)	낮음 (위변조 위험 존재)	높음 (서버에 저장, 클라이언트는 ID만 보유)
데이터 형태 (Data Type)	오직 String (문자열 데이터만 가능)	모든 Object (자바 객체 저장 가능)
서버 부하 (Server Load)	없음 (브라우저가 감당)	있음 (사용자가 많을수록 서버 메모리 차지)
식별 방식 (Identifier)	getName()으로 배열에서 직접 검색	쿠키로 전달된 JSESSIONID로 자동 매칭

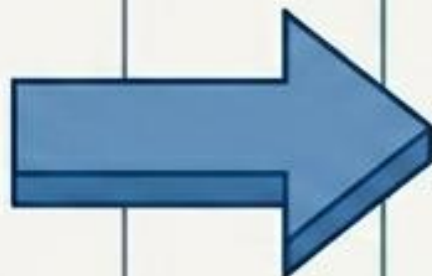
복잡한 Java 코드를 숨기고 표현을 극대화하다: EL (Expression Language)

Before (익스프레션 문법)

```
1 <%=  
2 (String)session.getAttribute("cnt")  
3 + 1 %>  
4  
5
```



문제점: 자바 변수 캐스팅이 필요하며,
코드가 길고 가독성이 심각하게 저하됨.



After (EL 문법)

```
1 ${cnt + 1}  
2  
3
```



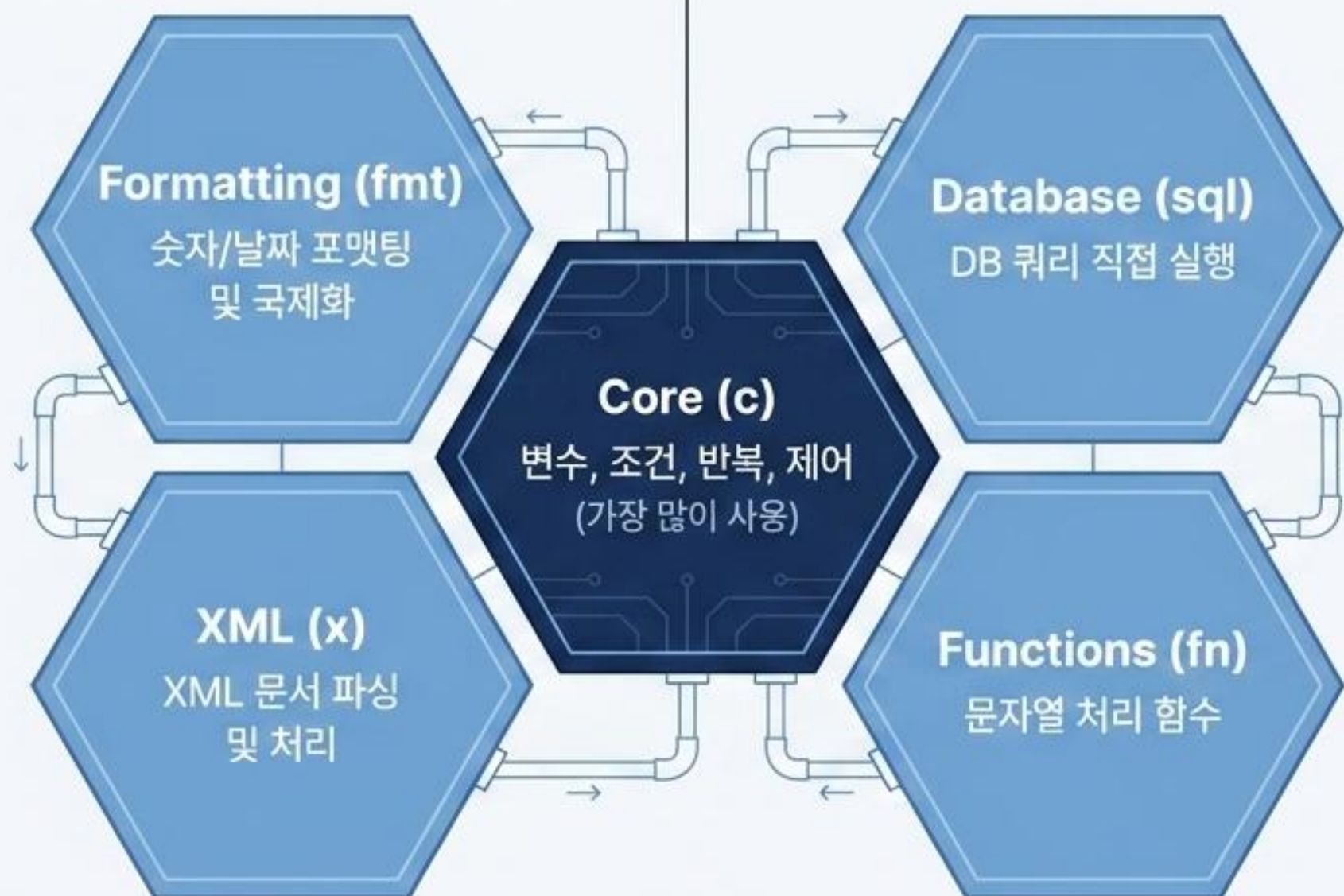
장점: 데이터 영역(Scope)에서 애트리뷰트
이름을 자동으로 찾아 계산 및 즉시 출력.
캐스팅 불필요.

핵심 기능: EL은 `${result}`, `${m:sqrt(100)}` 처럼 단일 출력부터 자바의 정적 메서드 호출까지,
식을 계산해 결과를 즉시 출력하는 데 특화된 언어입니다.

우아한 뷰(View) 로직을 위한 표준 태그 라이브러리: JSTL

```
<%@taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```

← 지시자를 통한
라이브러리 연결 필수



JSTL Core ①: 변수 선언과 조건 분기의 태그화

1. 변수 선언 (<c:set>)

```
<c:set var="num" value="100"/>
```

(Java의 `int num = 100;` 과 동일)

특징: `scope="request"` 속성으로 데이터 영역 직접 지정 가능. `value`에 EL식 사용 가능.

2. 단일 조건문 (<c:if>)

```
<c:if test="${num1 > num2}">
```

(Java의 `if` 문과 동일)

3. 다중 조건 분기 (<c:choose>)

```
<c:choose>
  <c:when test="..."> (Java의 case)
  <c:otherwise> (Java의 default)
</c:choose>
```

`<c:choose>` 내부에 분기 조건을 배치하여 Java의
`switch-case`와 완벽하게 동일한 흐름 제어.

JSTL Core ②: 강력한 반복 순회와 페이지 제어 이동

데이터 순회 (<c:forEach>)

단순 반복:

```
<c:forEach var="cnt" begin="1" end="10">  
(1부터 10까지 순차적으로 HTML 렌더링)
```

컬렉션 순회:

```
<c:forEach var="str" items="${arr}">  
(배열이나 리스트의 데이터를 순서대로 출력)
```

페이지 제어 (<c:redirect> & <c:import>)

```
<c:redirect url="result.jsp"/>
```

완전히 새로운 페이지로 이동.

```
<c:import url="result.jsp">
```

다른 웹 자원을 현재 페이지에 포함 (<jsp:include>와 유사). 내부에 <c:param>을 사용해 파라미터 전달 가능.

Note: 데이터의 안전한 출력은 <c:out value="\${str}" default="No data"/> 활용

글로벌 규격에 맞는 완벽한 데이터 출력: JSTL fmt

Terminal/Console Output

날짜 포매팅 <fmt:formatDate>

입력: "java.util.Date" 객체

- type="time" -> "시각만 출력"
- pattern="(a) hh:mm:ss" -> "지정된 커스텀 패턴으로 출력"

숫자 포매팅 <fmt:formatNumber>

입력값 기준 출력 예시:

- groupingUsed="true" -> "100,000" (천 단위 콤마)
- pattern="#.##" -> "3.14" (소수점 제어)
- type="percent" -> "50%" (0.5 입력 시)
- type="currency" currencySymbol="\\" -> "\\100,000" (화폐 기호 적용)

강력한 서블릿 엔진 위에서 동작하는 JSP는, 세션을 통한 상태 관리와 EL/JSTL의 우아한 문법을 통해 현대적이고 견고한 웹 뷰(View)를 완성합니다.

모던 웹 애플리케이션의 뼈대: 3계층 아키텍처와 MVC 설계의 이해

JSP, Servlet, JDBC부터 완전한 백엔드 시스템 조립까지 — 기술 면접과 실무를 위한 시각적 해설서



이 문서는 단일 페이지로 읽을 수 있도록 최적화된 독립적 기술 가이드입니다. 각 컴포넌트의 역할과 최종적인 데이터 흐름을 추적합니다.

파편화된 기술들은 하나의 완벽한 시스템으로 조립됩니다.



요청의 지휘자

Servlet & Controller

@WebServlet

doGet/doPost

클라이언트의 요청을 분석하고
분기하는 로직의 중심.



화면의 렌더러

JSP & View

HTML

<% %>

사용자에게 보여질 동적 HTML을
생성하는 프레젠테이션 엔진.



데이터의 저장소

JDBC & Model

DAO

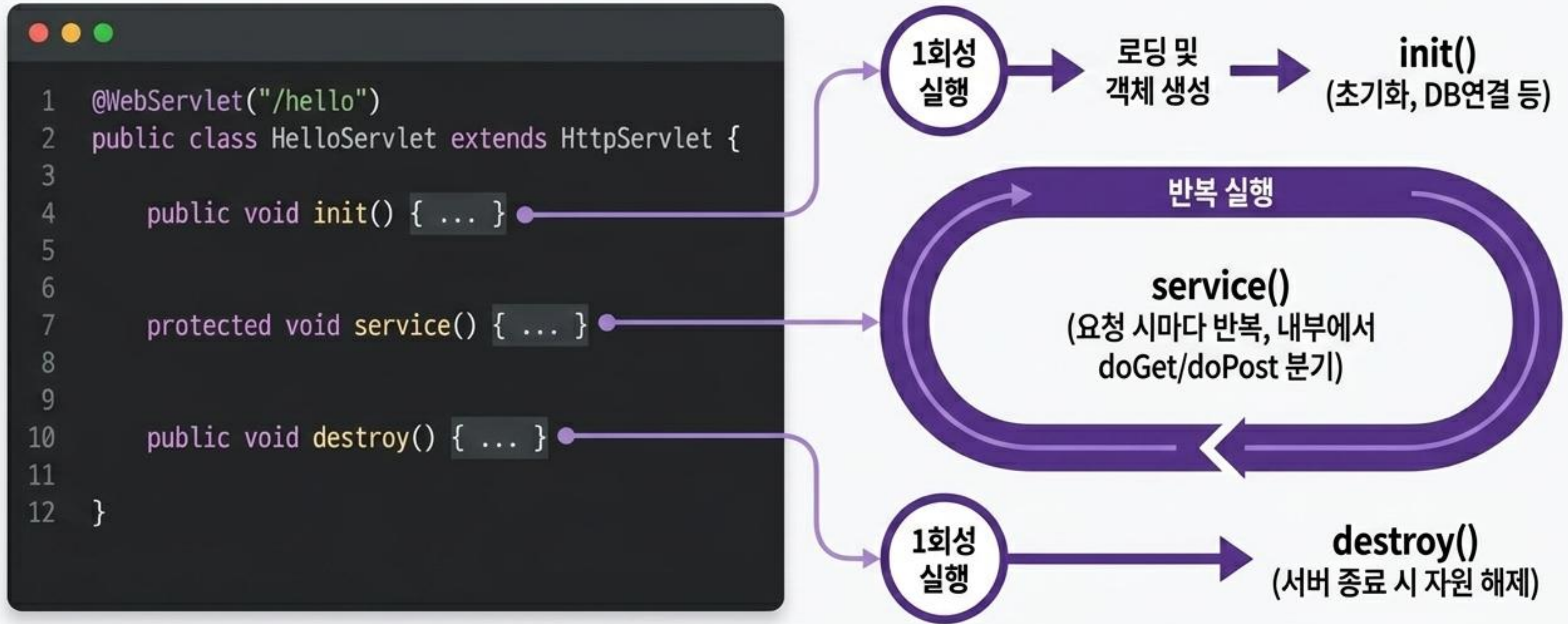
DTO

SQL

영구적인 데이터를 저장하고
비즈니스 로직과 소통하는 기반.

학습 로드맵: 각 부품의 작동 원리를 해부한 뒤, 마지막 슬라이드에서 이들을 완벽한 데이터 파이프라인으로 결합합니다.

Servlet: 클라이언트 요청을 처리하는 멀티스레드 기반의 자바 클래스입니다.



성능 최적화: Servlet은 요청마다 객체를 생성하지 않고, 하나의 객체를 재사용하는 멀티스레드 방식으로 동작하여 처리 속도가 매우 빠릅니다.

클라이언트의 두 가지 주요 요청 방식: GET vs POST

GET (조회용)



- ✓ 데이터 위치: URL에 직접 포함 (/login?id=aaa)
- ✓ 보안성: 낮음 (데이터가 노출됨)
- ✓ 길이 제한: 존재함
- ✓ 처리 속도: 빠름
- ✓ 사용 예시: 검색, 목록 조회, 단순 페이지 이동
- ✓ 메서드: doGet()

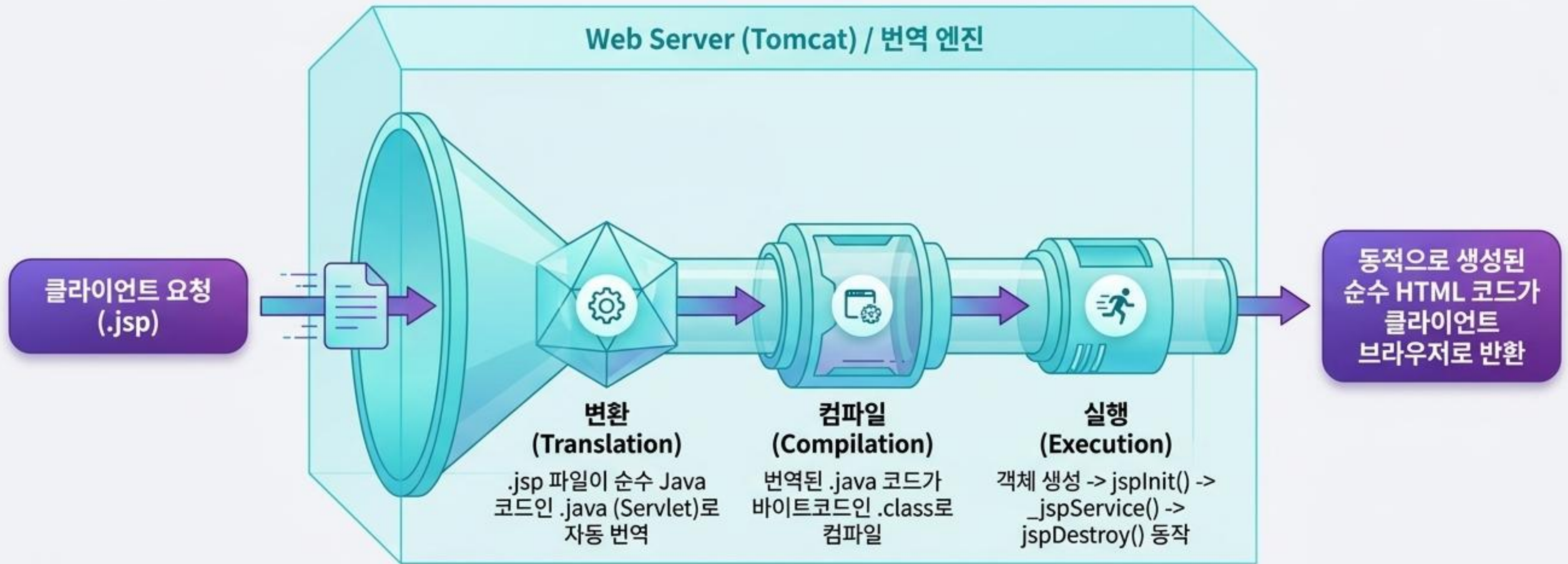
POST (저장용)



- ✓ 데이터 위치: HTTP Body에 숨겨서 전송
- ✓ 보안성: 상대적으로 높음
- ✓ 길이 제한: 거의 없음
- ✓ 처리 속도: GET에 비해 약간 느림
- ✓ 사용 예시: 로그인, 회원가입, 대용량 폼 데이터 저장
- ✓ 메서드: doPost()

service() 메서드가 클라이언트의 요청 방식을 확인하여 자동으로 해당 메서드(doGet 또는 doPost)로 분기(Routing)합니다.

JSP의 비밀: 내부적으로는 결국 Servlet으로 변환되어 실행됩니다.



JSP는 HTML 작성의 불편함을 해소하기 위해 만들어졌습니다.
HTML 뼈대 안에 Java 코드를 삽입하면, 서버가 이를 Servlet으로 자동 변환하여 처리하는 구조입니다.

JSP 해부학: 지시자, 스크립트 요소, 그리고 내장 객체

지시자 (Directive):

페이지 전체 속성, 인코딩, import 설정
(page, include, taglib)

선언문 (Declaration):

변수 및 메서드 선언. 변환 시
Servlet 클래스의 멤버로 편입됨

```
1 <%@ page contentType="text/html" import="java.sql.*" %>
2 <%! int count = 0; %>
3 <% String id = request.getParameter("id"); %>
4 <%= count %>
```

스크립틀릿 (Scriptlet):

일반적인 Java 코드 작성 영역
(_jspService 내부로 삽입됨)

표현식 (Expression):

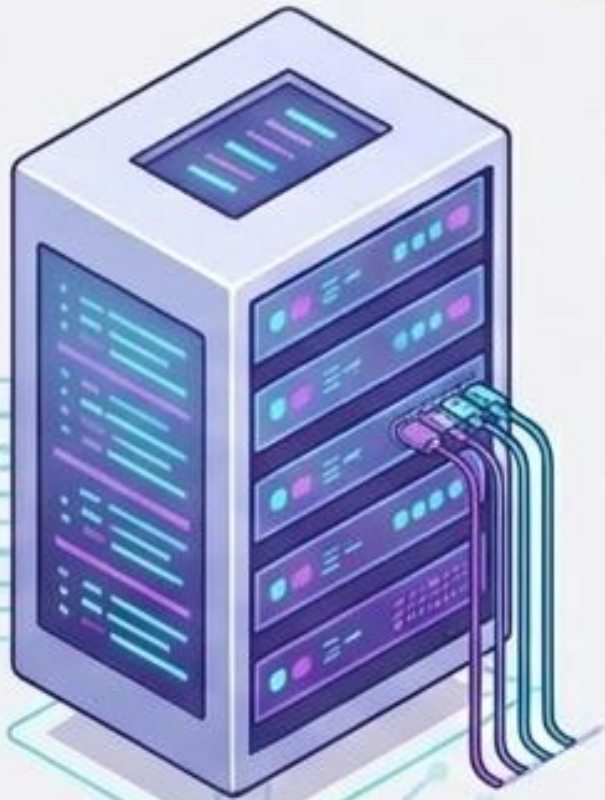
값을 화면에 출력
(out.print())와 동일한 역할

즉시 사용 가능한 내장 객체 9종사

- ✓ request (요청 정보)
- ✓ response (응답 제어)
- ✓ session (세션 관리)
- ✓ out (화면 출력)
- ✓ application (앱 전체 공유)
- ✓ pageContext (JSP 문맥 정보)
- ✓ config (설정 정보)
- ✓ page (현재 객체)
- ✓ exception (예외 처리)

상태 관리의 두 축: Session과 Cookie의 결정적 차이

Session (서버 측 저장)



-  저장 위치: 서버 메모리 공간
-  보안성: 높음
(중요 데이터 보호에 적합)
-  생명주기: 브라우저 종료 시
또는 invalidate() 호출 시
즉시 삭제
-  주요 용도: 로그인 상태 유지,
보안 연결
-  주요 메서드: `setAttribute()`,
`getAttribute()`

Cookie (클라이언트 측 저장)



-  저장 위치: 사용자 브라우저 내부
-  보안성: 낮음 (위변조 위험 존재,
민감 데이터 금지)
-  제한 사항: 저장 용량 제한 존재,
처리 속도 빠름
-  주요 용도: 자동 로그인, 장바구니,
'오늘 하루 보지 않기' 팝업
-  생성 방식: `new Cookie()`,
`response.addCookie()`

핵심 포인트: HTTP 프로토콜은 본질적으로 비연결성(Stateless)이므로,
사용자를 식별하고 연속적인 상태를 유지하기 위해 이 두 가지 기술이 필수적입니다.

왜 두 기술이 모두 필요할까요? 화면과 로직의 역할 분담

Servlet (컨트롤러 역할)

-  패러다임: 코드 중심
(Java 코드 안에 HTML 문자열이 섞임)
-  주요 역할: 클라이언트 요청 분석, 비즈니스 로직 처리, DB 통신, 데이터 가공
-  아키텍처 위치: MVC 패턴의 Controller
-  단점: 화면 UI(HTML)를 작성하고 수정하기가 매우 불편함

JSP (뷰 역할)

-  패러다임: 문서 중심
(HTML 뼈대 안에 Java 코드가 삽입됨)
-  주요 역할: Controller가 가공해준 데이터를 바탕으로 최종 사용자 화면(UI) 렌더링
-  아키텍처 위치: MVC 패턴의 View
-  단점: 로직과 디자인이 섞이면 코드가 복잡해져 유지보수가 어려움

모던 트렌드: 현재는 JSP 내부에 Java 로직을 직접 작성하기보다, 순수 View 역할로만 제한하며 Spring 프레임워크 기반의 MVC 패턴 적용이 업계 표준입니다.

데이터의 형태를 정의하는 3가지 디자인 패턴

DTO (Data Transfer Object)



역할: 계층 간(Controller ↔ DAO ↔ View)
데이터를 실어 나르는 택배 상자.

특징: 내부에 로직이 없는 단순 데이터 구조체.
Getter/Setter가 존재하며, 데이터의
변환 및 수정이 가능(Mutable).

VO (Value Object)



역할: 특정 '값' 자체를 완벽하게 표현하는
봉인된 금고.

특징: 읽기 전용(Immutable)으로 설계됨.
생성자를 통해서만 값을 주입받으며,
금액이나 날짜 등 비고유 의미를 내포함.

DAO (Data Access Object)

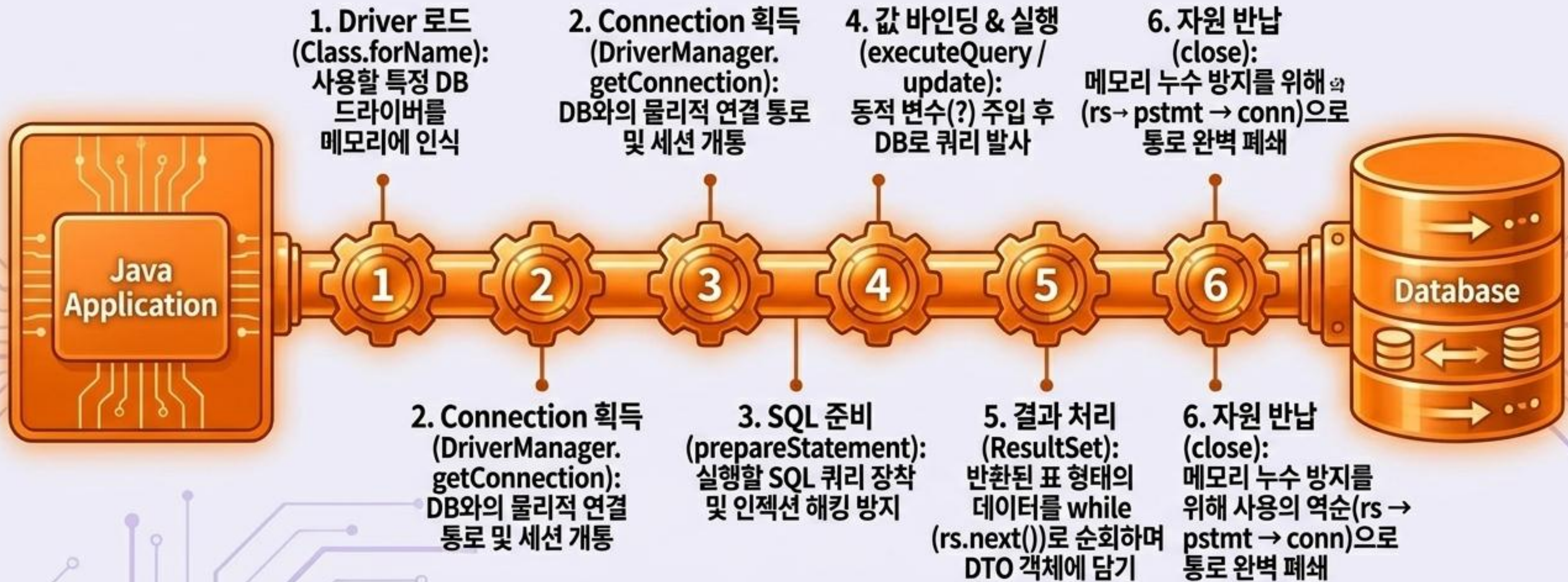


역할: DB 문을 열고 닫으며 SQL 실행을
전담하는 작업자.

특징: 데이터베이스와의 직접적인 연결 로직을
비즈니스 로직(Service)의 요청을 받아
INSERT, UPDATE, SELECT 등을 수행.

모심 트렌드: 현재트렌드와 기술은 데이터의 종의리고가 상영하는
비즈니스 로직 보직 및 종제되는 연건과 구정이 적용하도 등가 승합니다.

JDBC 파이프라인: Java 애플리케이션이 DB와 소통하는 6단계



모던 트렌드: 현재는 JDBC를 직접 다루기보다 JPA, MyBatis 같은 프레임워크를 사용하지만, 내부 동작 원리 이해를 위해 6단계 과정 숙지가 중요합니다.

관계형 DB를 다루는 언어: DML (데이터 조작어)

INSERT (데이터 추가)

```
1 INSERT INTO  
  member(name, age)  
  VALUES('kim', 20);
```

id	name	age
1	name	20
2	italy	20
3	nang	20
3	kim	20

테이블 하단에 새로운 행(Row)이 추가되며
불이 켜지는 모습.

UPDATE (데이터 수정)

```
1 UPDATE member  
  SET age=30  
  WHERE name='kim';
```

id	name	age
1	name	20
2	kim	30
3	nang	20
4	name	30

기존 행의 특정 셀(Cell) 값이 타겟팅되어
색상이 변하는 모습.

DELETE (데이터 삭제)

```
1 DELETE FROM member  
  WHERE name='kim';
```

id	name	age
1	name	20
2	italy	20
3	kim	20
4	name	20
5	name	20

테이블中间的 특정 행이 붉은색으로 변하며
소멸(Dissolve)되는 모습.

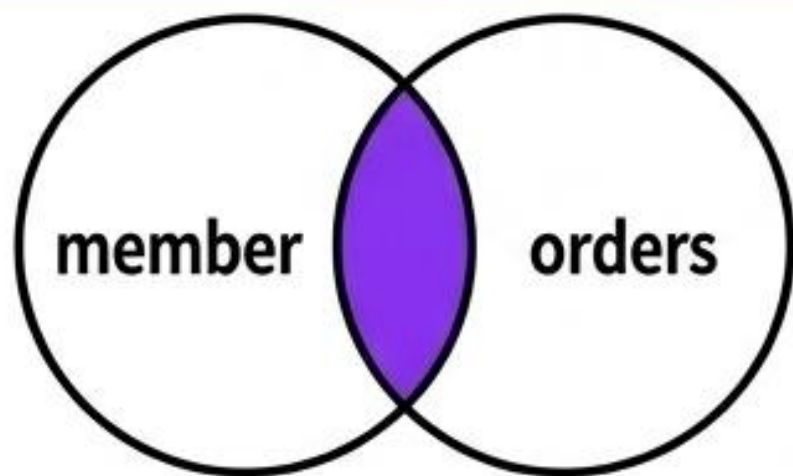
SELECT (데이터 조회)

```
1 SELECT * FROM member;
```

id	name	age
1	name	20
2	name	20
3	kim	20
4	name	20
5	name	20

돋보기 아이콘이 전체 테이블을 스캔하고
하이라이트하는 모습.

JOIN: 흠어진 테이블의 데이터를 하나의 의미 있는 결과로 결합



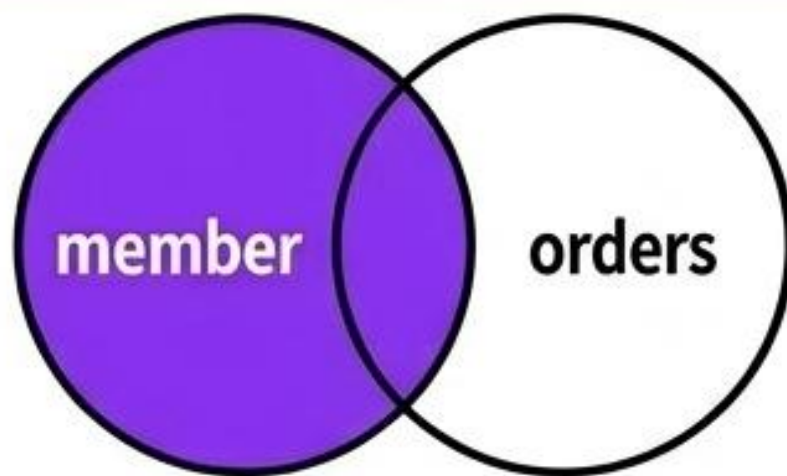
INNER JOIN (교집합)

개념: 두 테이블에 모두 공통으로 존재하는 데이터만 추출.

결과: 주문 내역이 존재하는 회원만 조회 (주문 없는 회원은 누락).

SQL: INNER JOIN ... ON

member_id	name	order_id	product
1	kim	101	Laptop
3	lee	102	Mouse
3	lee	102	Mouse



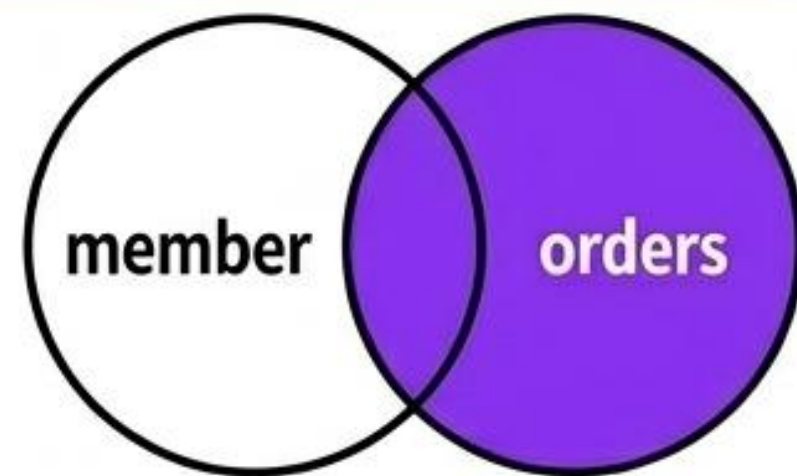
LEFT JOIN (왼쪽 기준 전체)

개념: 왼쪽 테이블(기준)의 모든 데이터와 매칭되는 오른쪽 데이터를 병합.

결과: 주문을 한 번도 하지 않은 유령 회원까지 모두 포함하여 전체 회원 목록 출력.

SQL: LEFT JOIN ... ON

member_id	name	order_id	product
1	kim	101	Laptop
2	park	NULL	NULL
3	lee	102	Mouse



RIGHT JOIN (오른쪽 기준 전체)

개념: 오른쪽 테이블을 기준으로 전체 데이터를 병합.

결과: 회원 정보가 삭제되었더라도 주문 내역이 남아있는 모든 주문 데이터 출력.

SQL: RIGHT JOIN ... ON

member_id	name	order_id	product
1	kim	101	Laptop
NULL	NULL	103	Keyboard
3	lee	102	Mouse

SubQuery: 쿼리 안에 중첩된 또 다른 쿼리



실행 순서

1. 내부(서브) 쿼리 선실행:
전체 회원의 평균 나이를
먼저 계산하여 도출 (예: 25)
2. 외부(메인) 쿼리 후실행:
도출된 결과값(25)을
조건에 대입하여, 나이가
25 초과인 회원만
최종 필터링하여 출력

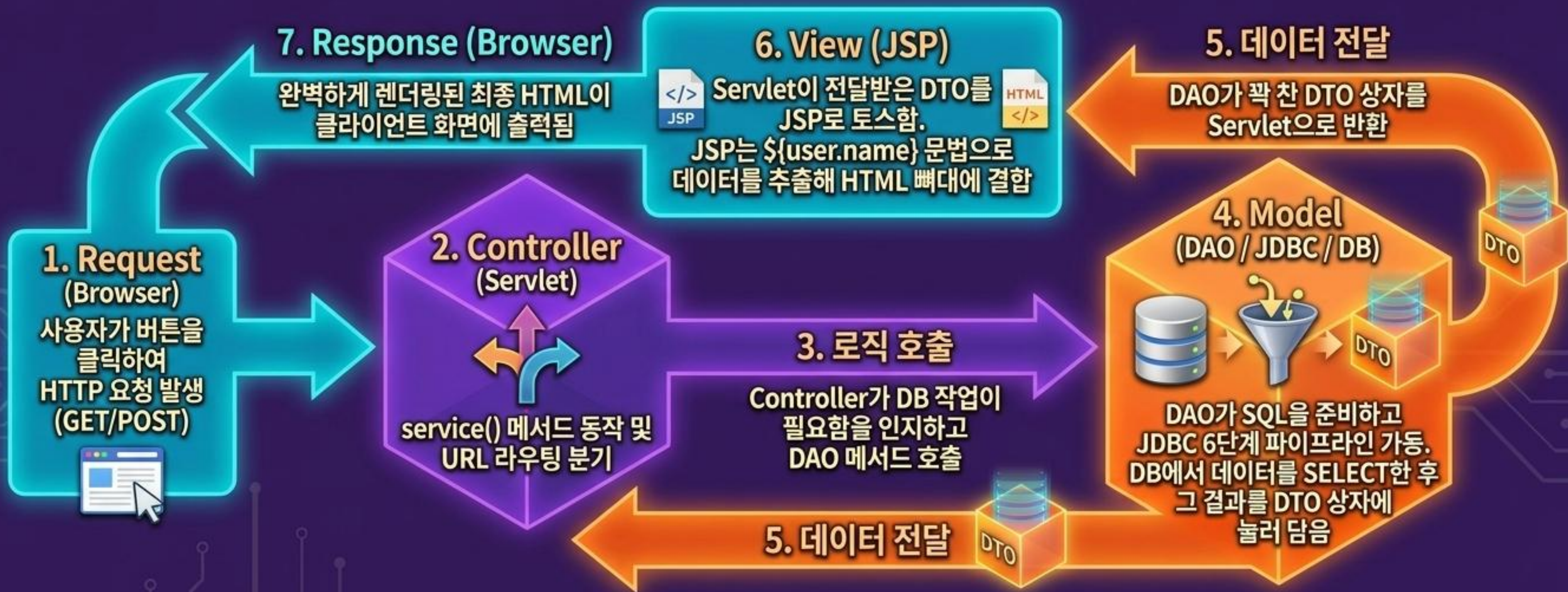
실무 팁: EXISTS 구문과 상관 서브쿼리를 결합하면,
'주문 내역이 하나라도 존재하는 회원'과 같은 복잡한
조건 필터링을 극도로 효율적으로 작성할 수 있습니다.

시스템의 조립: MVC (Model-View-Controller) 아키텍처



왜 MVC 패턴을 사용할까요? 화면 UI 레이어와 비즈니스 로직 레이어를 완벽히 분리함으로써, 스파게티 코드를 방지하고 디자이너와 백엔드 개발자의 독립적 협업을 가능하게 합니다.

실무 핵심 아키텍처 흐름 (The Complete Data Flow)



단일 부품과 문법의 암기를 넘어, 이 거대한 파이프라인 전체의 맥락과 데이터 흐름을 시각적으로 통찰하는 것. 그것이 백엔드 엔지니어링의 진정한 시작입니다.

웹 아키텍처의 진화: JSP 모놀리스에서 MVC 마스터리까지

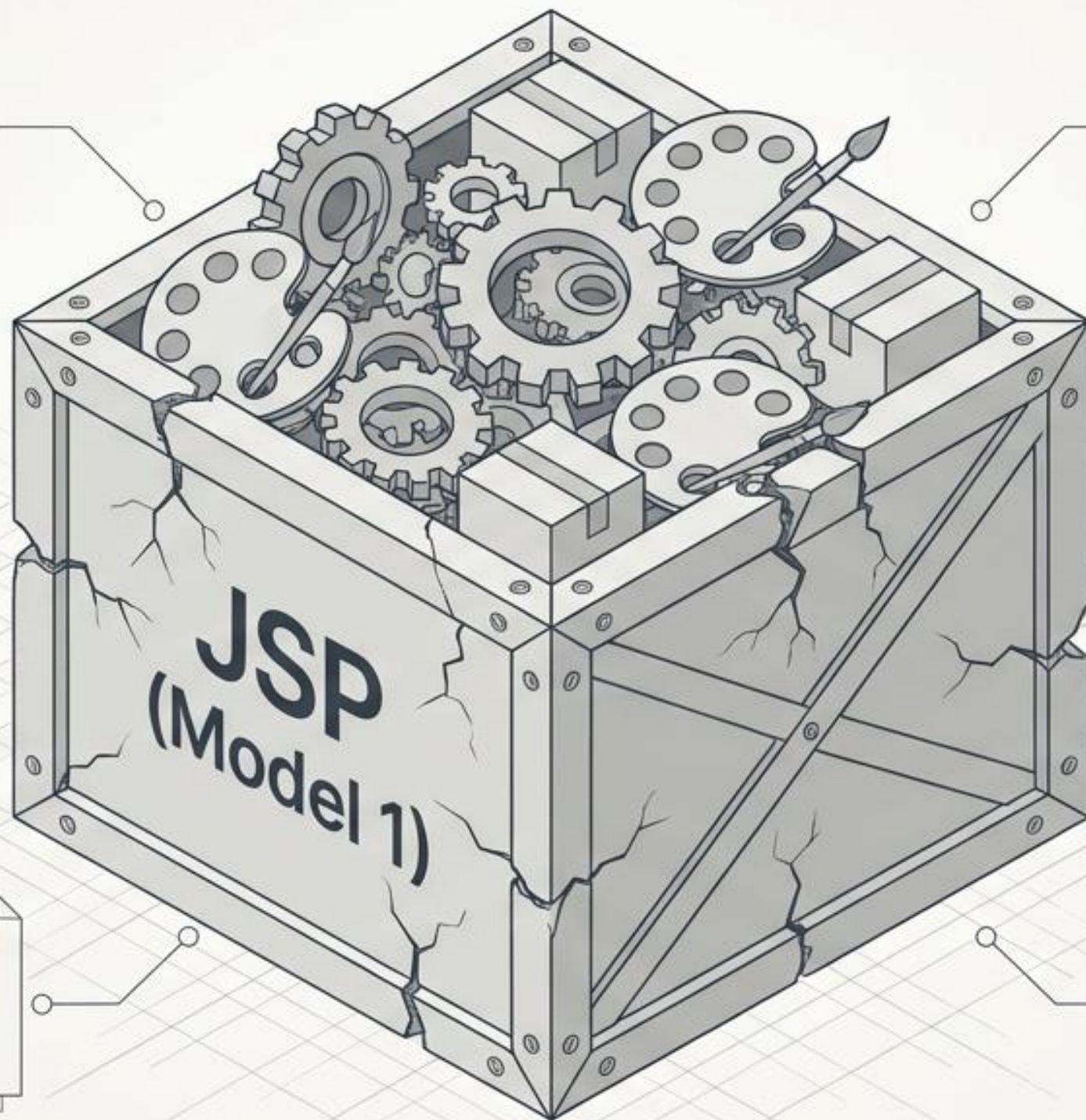
스파게티 코드에서 벗어나, 현대적이고 모듈화된
데이터 흐름과 라우팅 시스템 구축하기.



모든 것을 단일 파일에서 처리하던 과거의 방식 (Model 1)

작동 방식: JSP 파일 하나가 컨트롤러(요청 처리/자바 코드)와 뷰(화면 제공/HTML) 역할을 동시에 수행.

초기 이점: 개발 속도가 빠르고, 기술적 숙련도가 낮아도 쉽게 적용 가능.

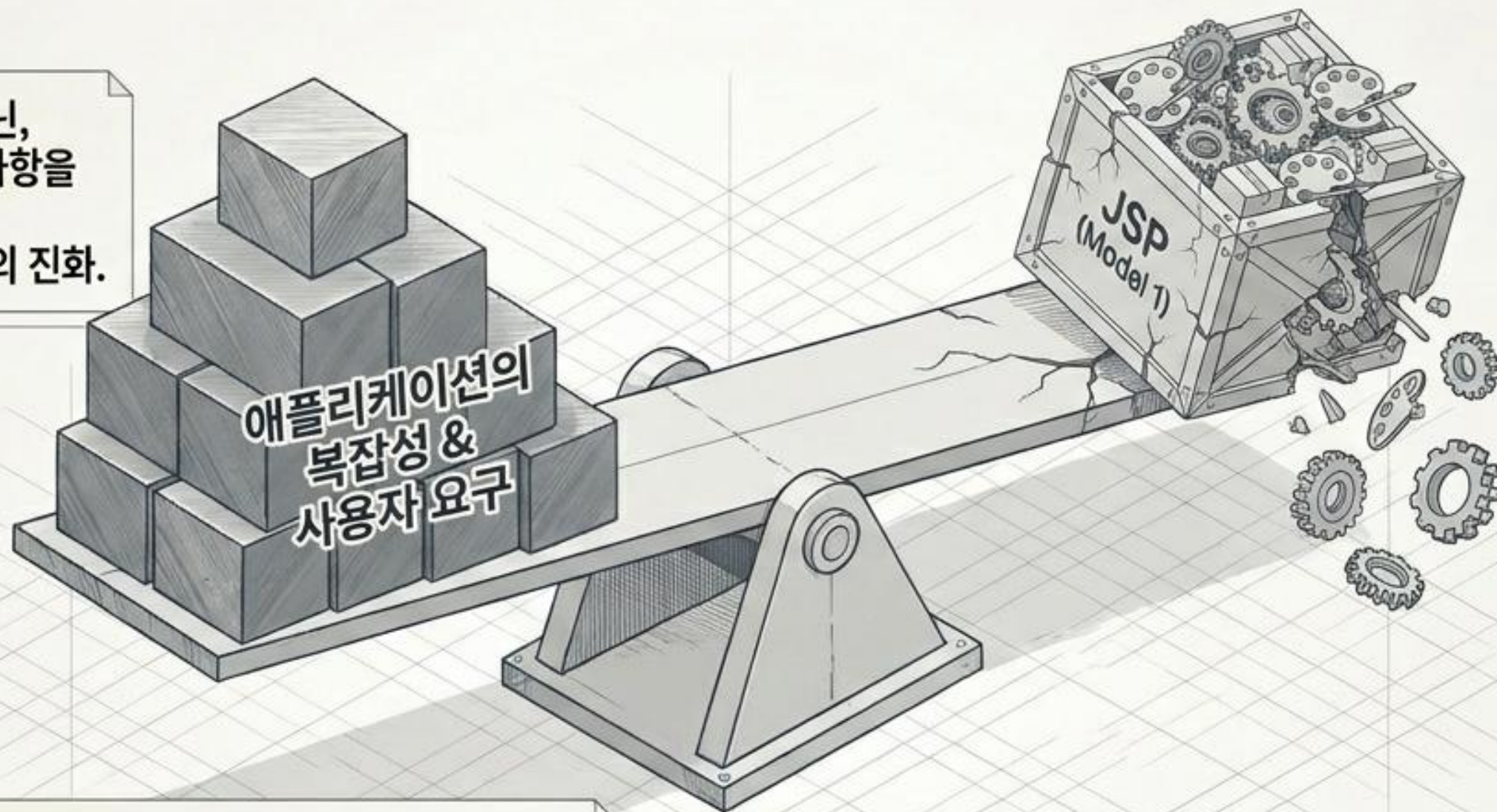


치명적 한계: 프레젠테이션 로직(UI)과 비즈니스 로직의 혼재로 인한 스파게티 코드 발생.

결과: 디자이너와 개발자의 협업이 불가능에 가깝고, 시스템이 커질수록 유지보수 난이도가 급증.

애플리케이션의 복잡성 증가가 가져온 패러다임의 전환

단순한 웹 페이지가 아닌,
복잡한 비즈니스 요구사항을
처리하는
'웹 애플리케이션'으로의 진화.

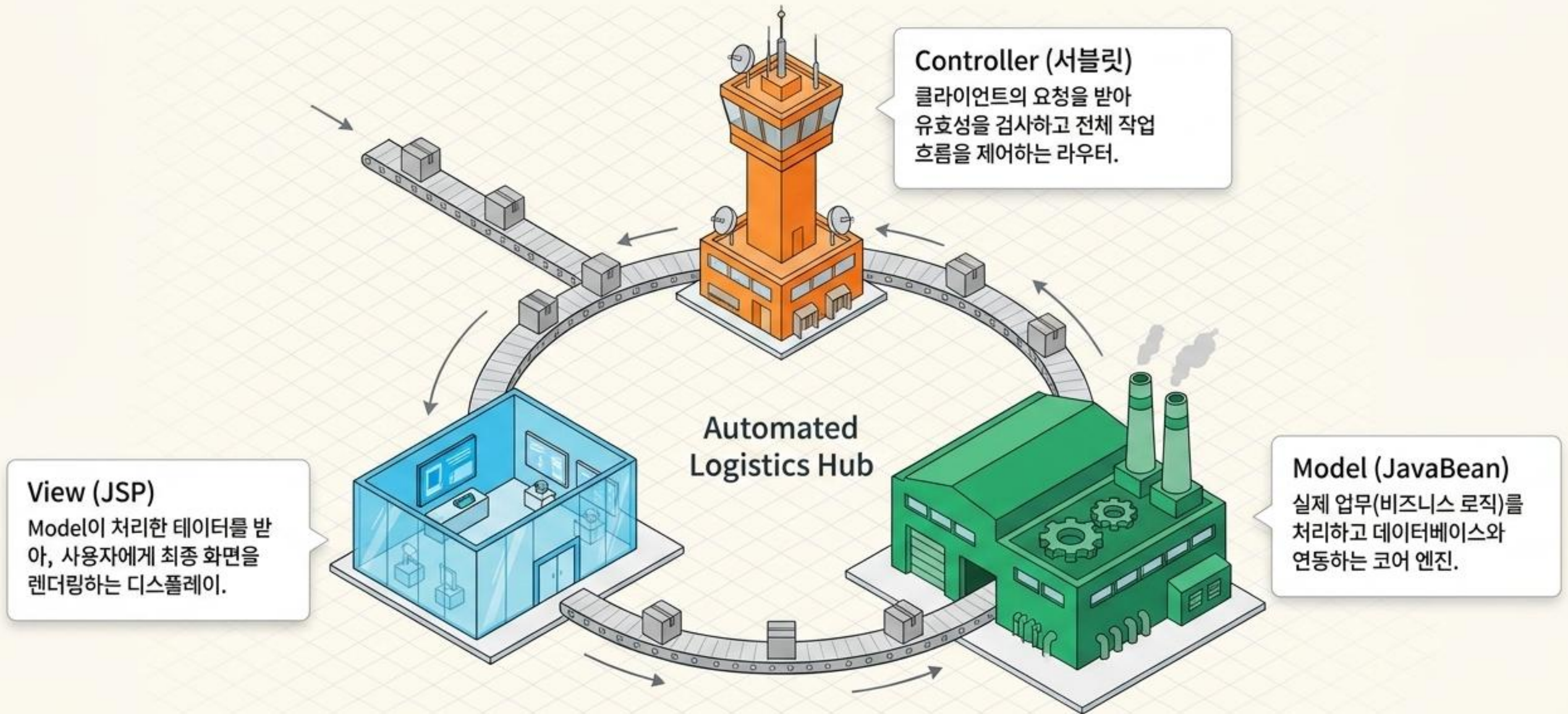


핵심 요구사항:

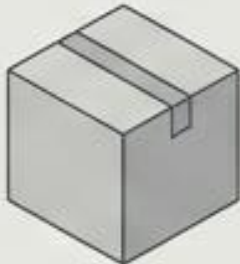
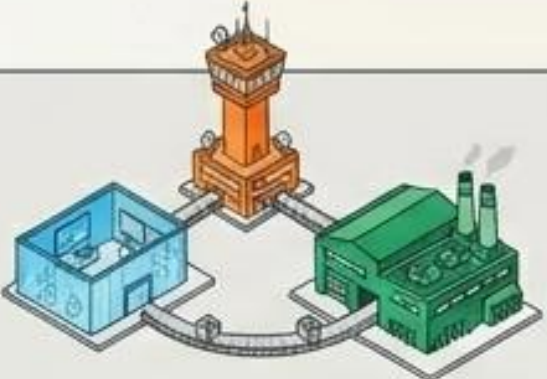
1. UI 로직과 데이터 처리 로직의 완벽한 분리.
2. 디자이너와 개발자의 병렬 작업(Parallel Work) 환경 보장.
3. 코드의 명료성과 확장성 확보.

이러한 한계를 극복하기 위해
역할을 세 가지로 분리하는
'Model 2 (MVC)' 아키텍처가 등장.

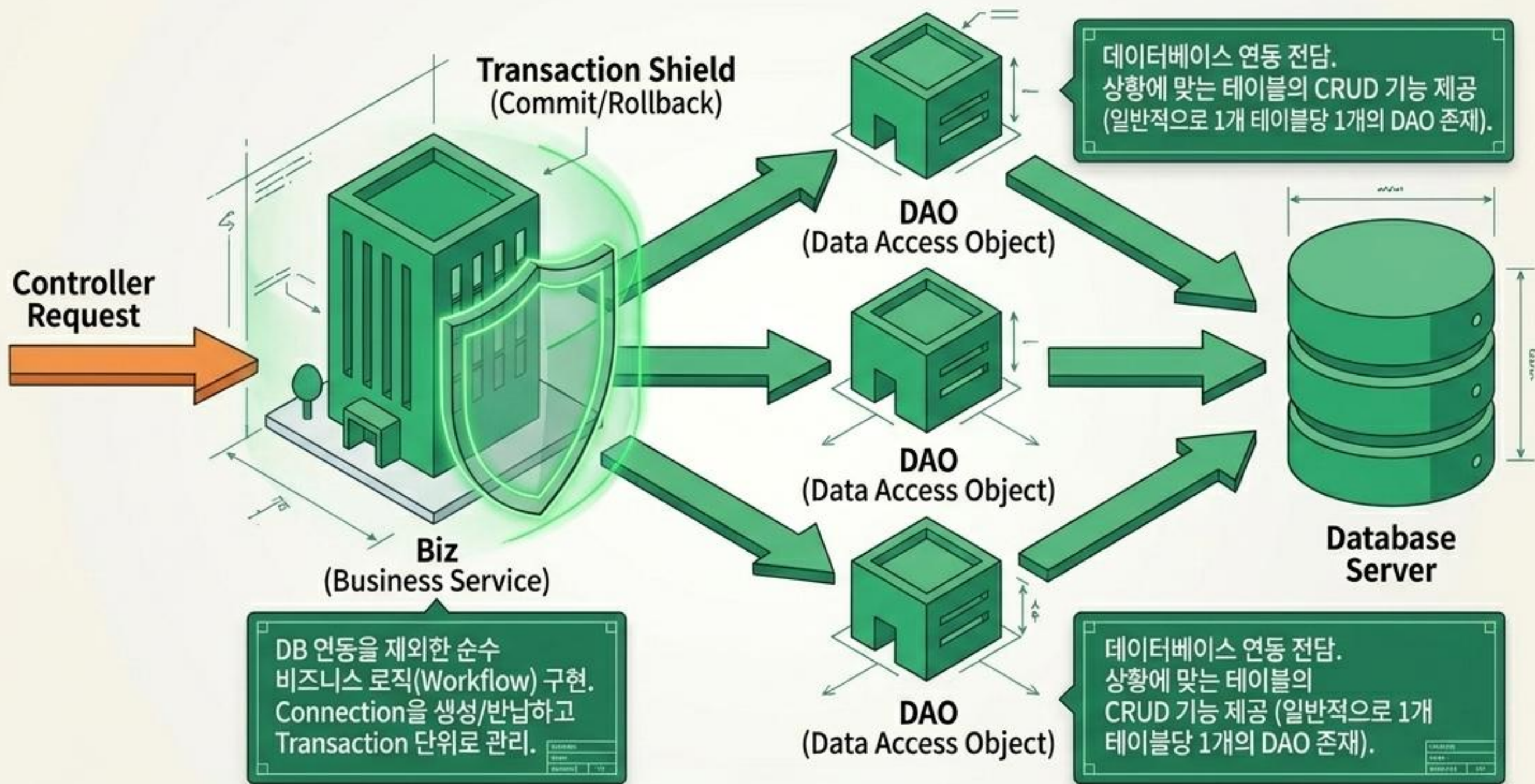
Model 2 (MVC) 아키텍처: 완벽하게 분업화된 물류 시스템



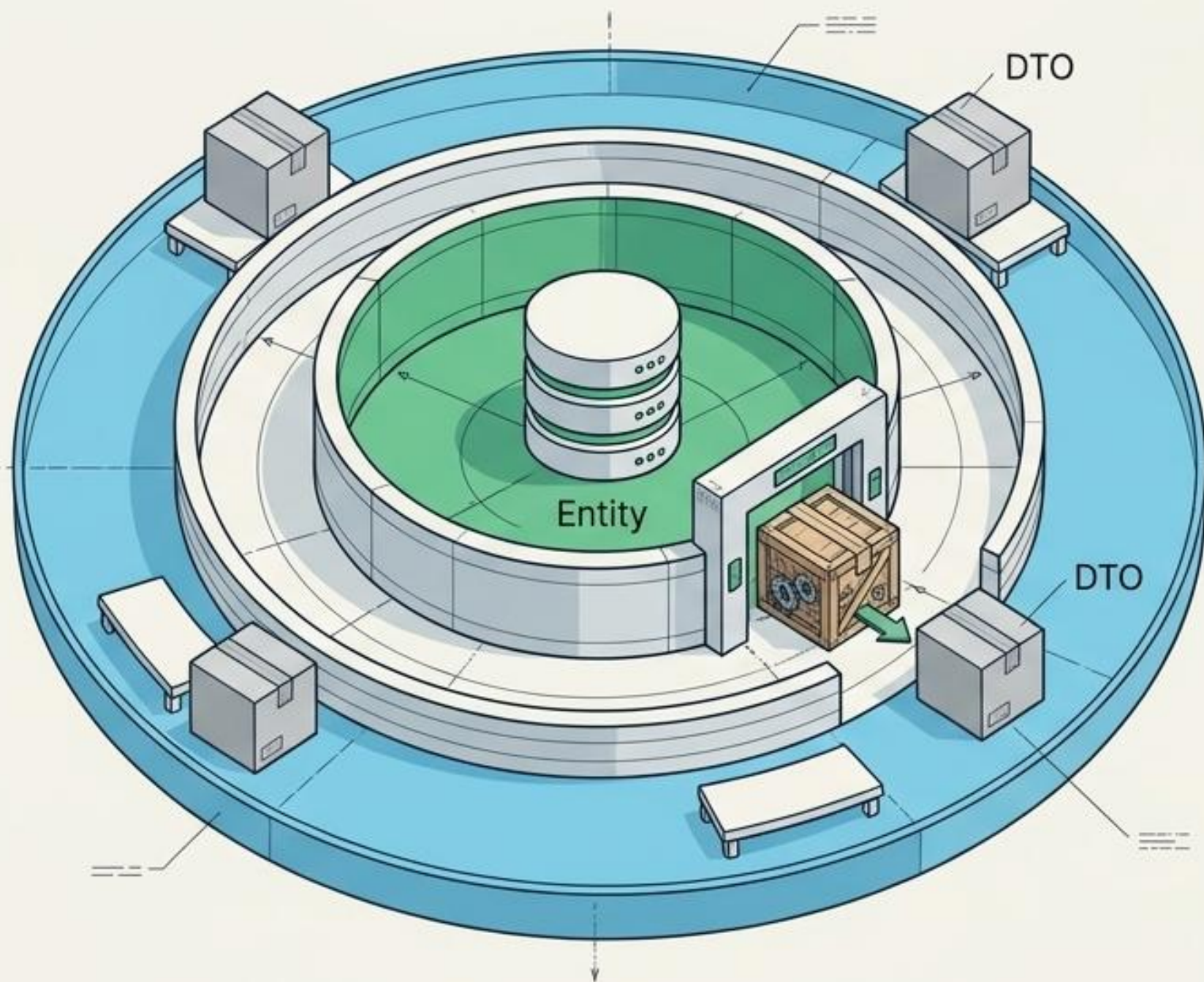
레거시 vs 모던 아키텍처 진단 매트릭스

	Model 1 (JSP Monolith) 	Model 2 (MVC) 
초기 설정 및 설계 (Setup)	빠름 (바로 개발 가능)	느림 (초기 아키텍처 디자인 시간 소요 및 이해도 요구)
역할 분리 (Role Separation)	불가능 (Java와 HTML이 뒤섞임) 	완벽함 (프레젠테이션과 비즈니스 로직 분리) 
협업 (디자이너-개발자)	어려움 (동시 작업 충돌) 	용이함 (명확한 작업 영역 분리) 
확장성 및 유지보수 (Scale)	낮음 (코드가 복잡해져 유지보수 불가) 	높음 (애플리케이션이 명료해져 확장 용이) 

모델(Model) 내부 구조: 비즈니스 로직과 데이터 접근의 분리



데이터 택배원: Entity와 DTO의 엄격한 경계선



Entity (내부 보안 구역)

DB 테이블과 동일한 구조를 가진 객체.
외부에 절대 노출하지 않으며,
프레젠테이션 로직을 포함하지 않아 변경을
최소화함.

(Biz ↔ DAO 통신에 사용)



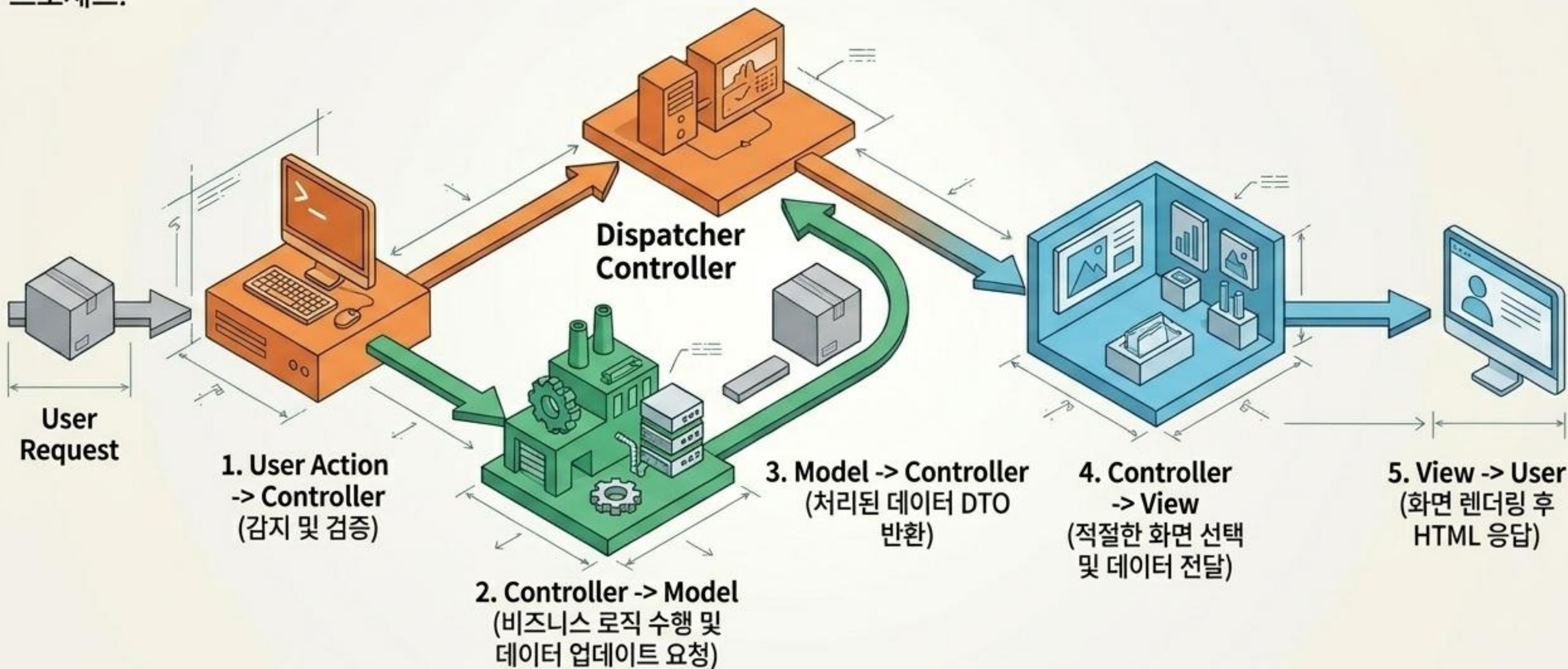
DTO (외부 전송용)

Entity와 유사하지만 외부에 노출되면 안
되는 민감한 데이터를 제외한 '데이터
교환용' 객체.

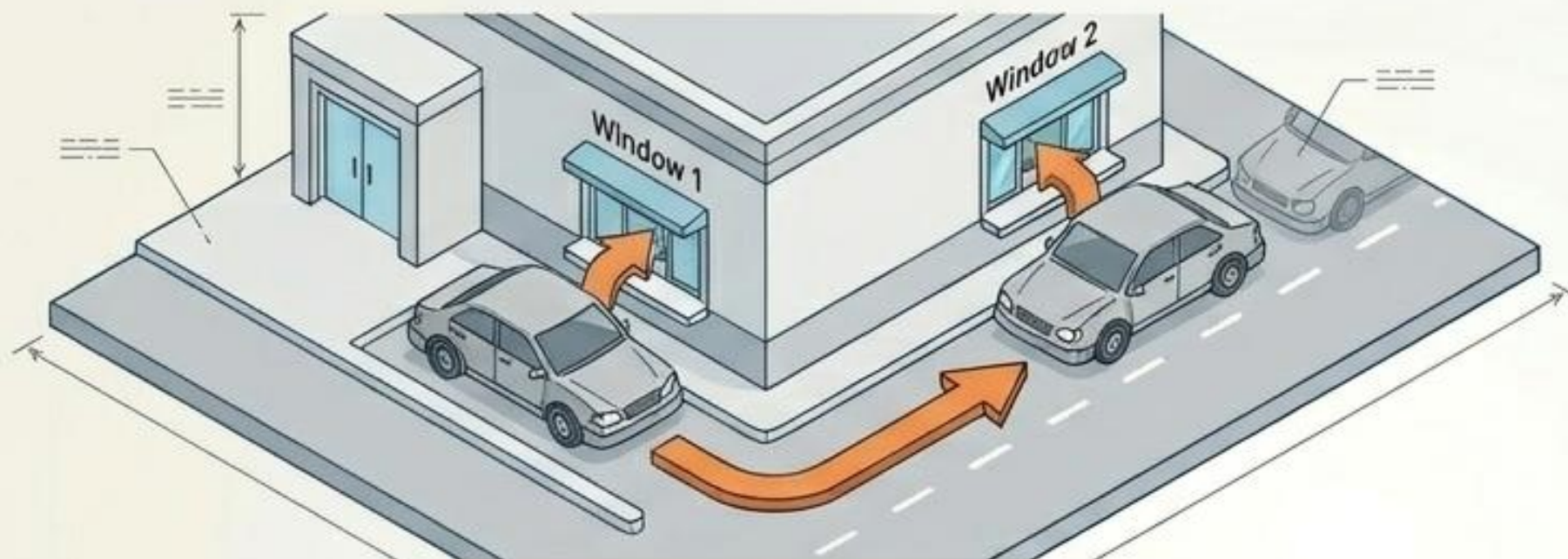
(JSP ↔ Controller ↔ Biz 계층 간 통신에 사용)

MVC 요청 수명 주기 (Request Lifecycle)

사용자 입력부터 최종 화면 렌더링까지, Controller가 중심에서 전체 흐름을 제어하고 각 영역을 라우팅하는 단방향 통신 프로세스.

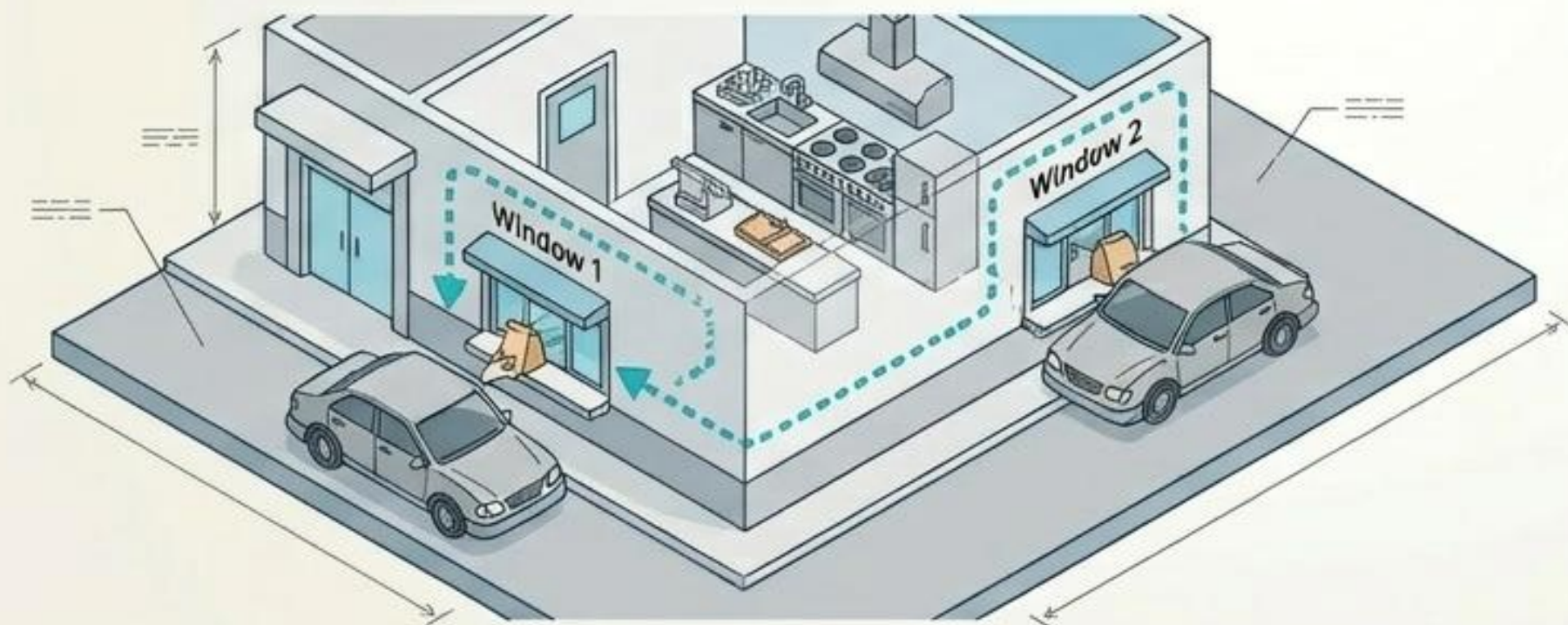


트래픽 제어: Redirect vs Forward의 결정적 차이



Redirect (경로 재지정):

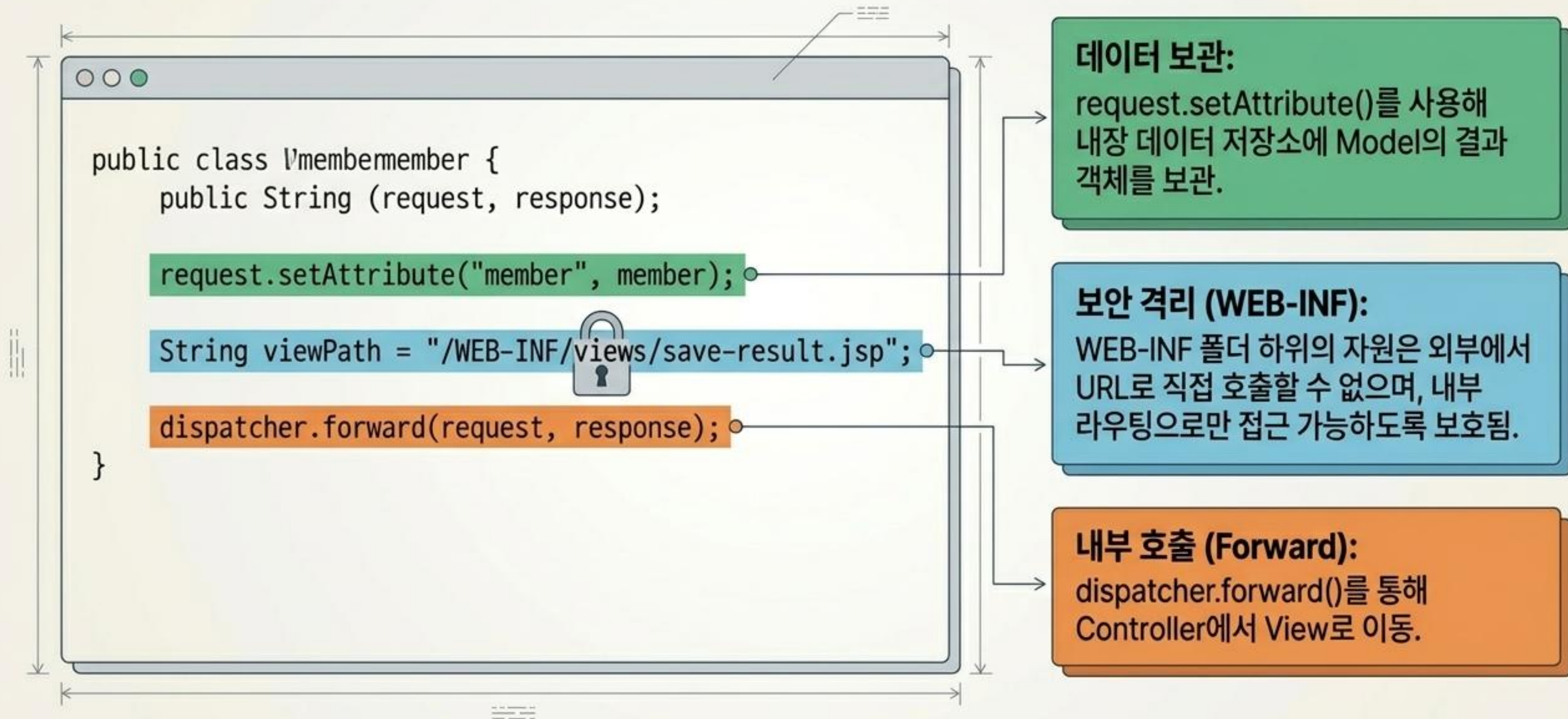
클라이언트가 인지할 수 있는 '2번'의 호출.
실제 웹 서버로 응답이 나갔다가 다시
요청하므로 브라우저의 URL 경로가 변경됨.



Forward (내부 전달):

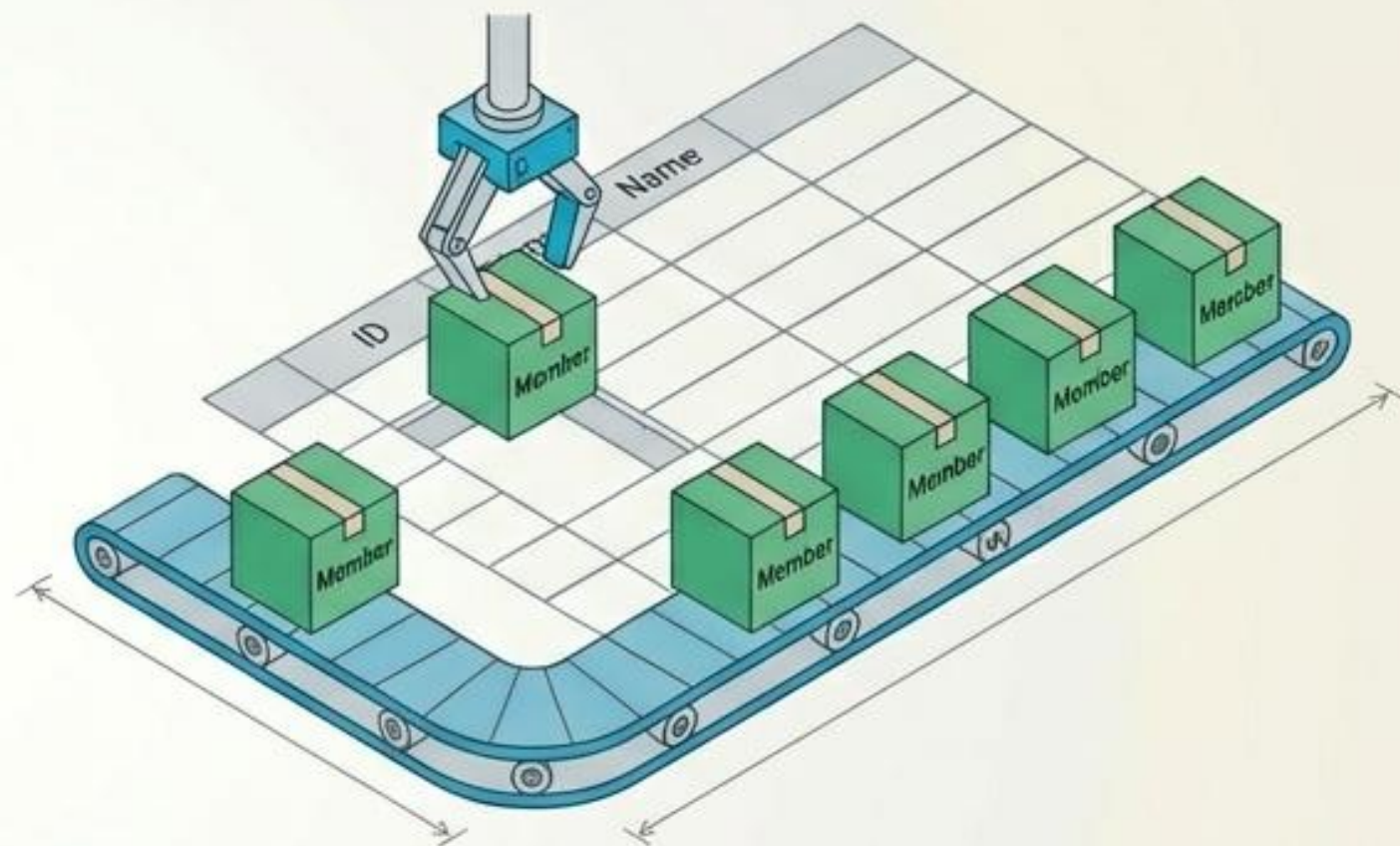
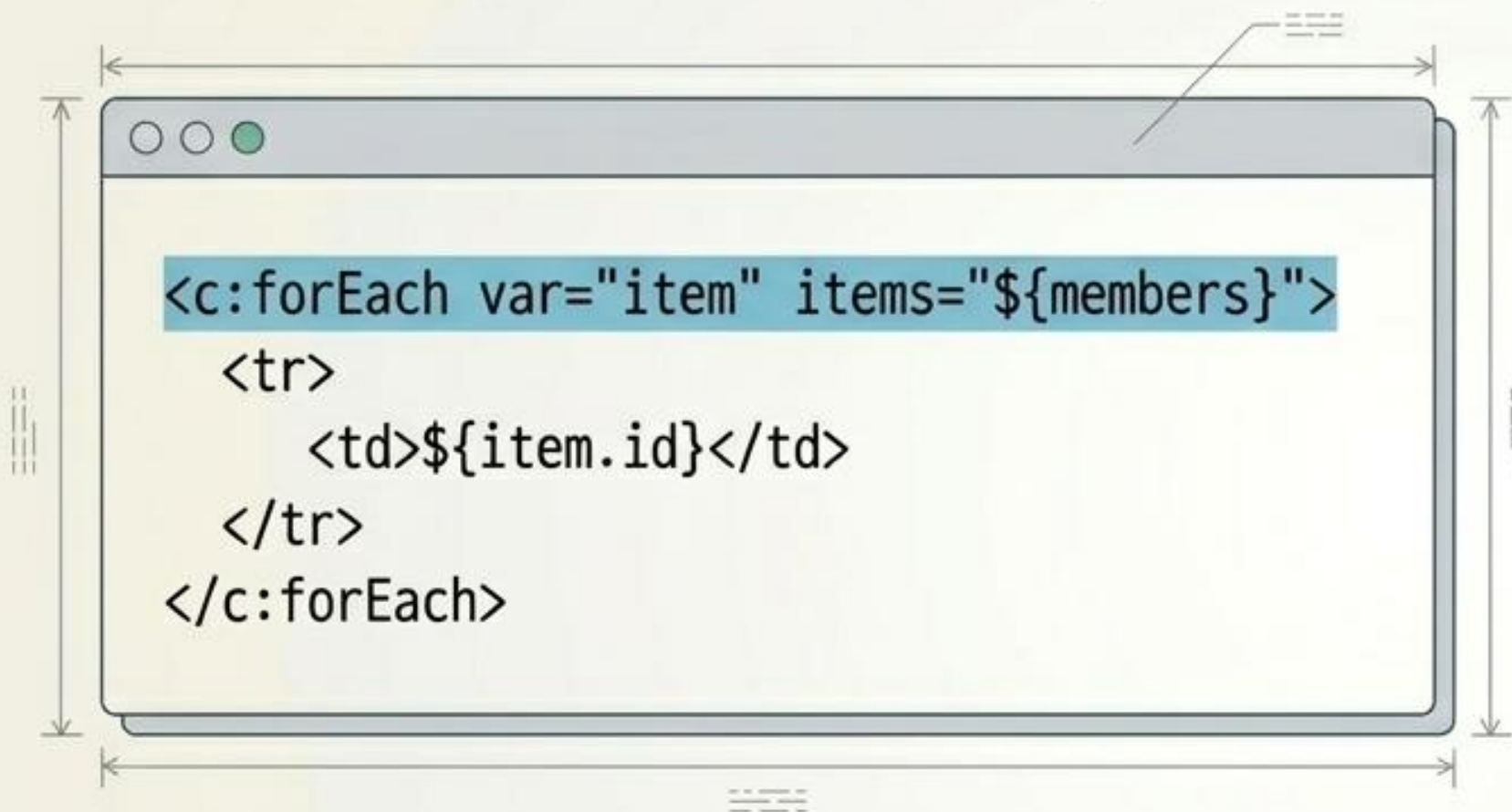
클라이언트의 1회 요청 후,
서버 내부에서만 일어나는 호출.
클라이언트는 이 위임 과정을 인지하지
못하며 URL이 유지됨.

실제 코드에 적용된 MVC: 라우팅과 데이터 보관



뷰(View) 로직의 우아함: JSTL과 프로퍼티 접근

View는 생(Raw) 자바 코드를 쓰지 않고, HTML 생성을 돕는 태그를 사용해 UI 구조를 청결하게 유지함.



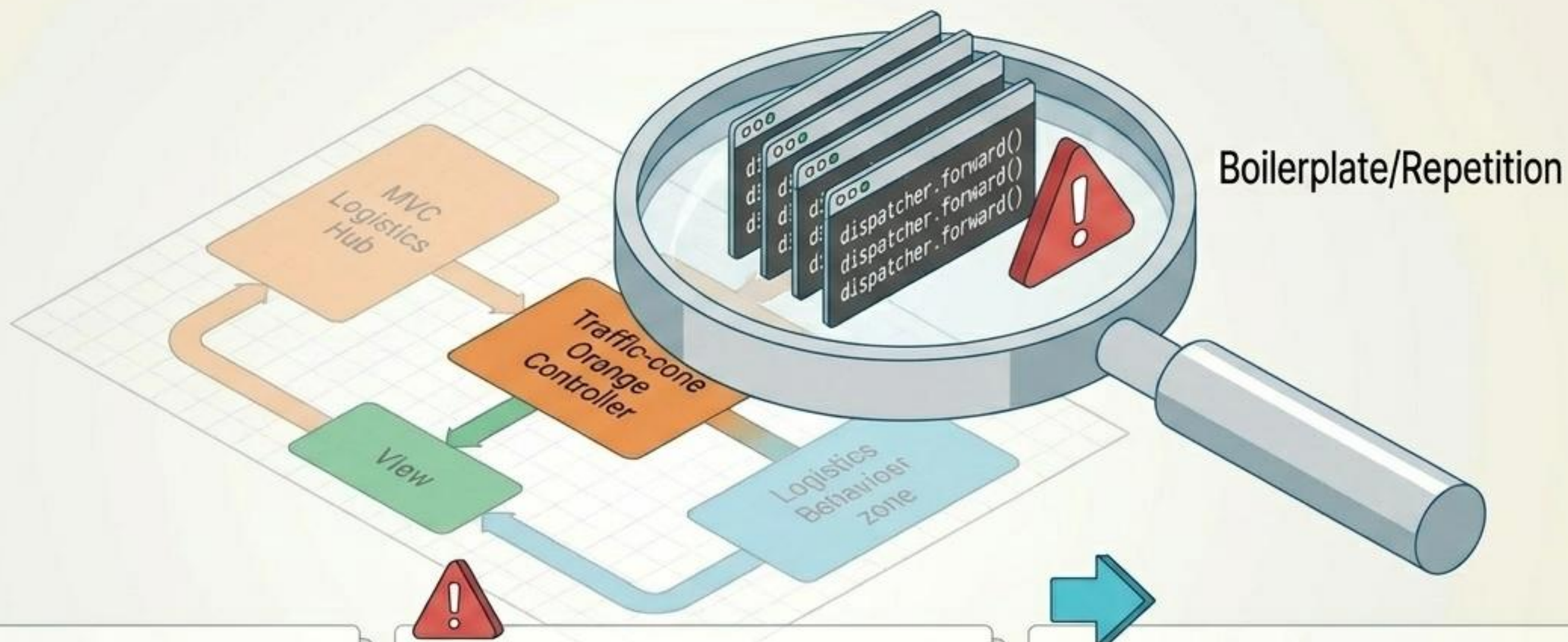
반복 렌더링:

JSTL `<c:forEach>` 태그를 사용해 Controller가 전달한 리스트 객체를 안전하게 루프 처리.

데이터 출력:

`${member.id}` 와 같은 프로퍼티 접근법을 사용.
(백그라운드에서 `request.getAttribute("member").getId()`와 동일하게 동작).

MVC의 완성, 그리고 다음 진화를 향한 과제



아키텍처의 완성:

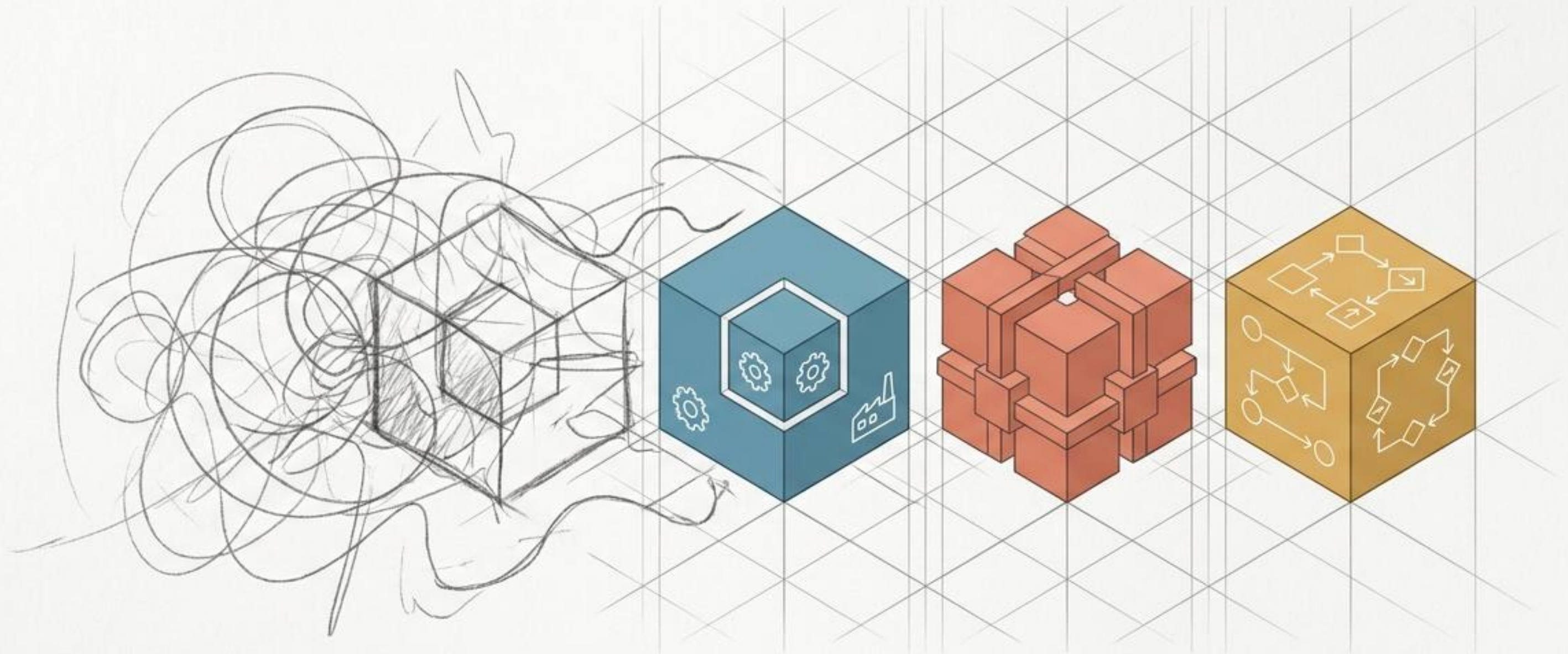
MVC 패턴은 비즈니스 로직과 화면의 라이프사이클을 완벽히 분리해 내는데 성공함.

남겨진 한계 (Boilerplate):

컨트롤러에서 뷰로 이동하는 viewPath 설정 및 dispatcher.forward() 포워딩 코드가 모든 컨트롤러마다 중복 발생.

Next Step:

이러한 반복적인 트래픽 제어 로직을 자동화하고 개발자를 비즈니스 로직에만 집중하게 만드는 프레임워크(예: Spring MVC)의 탄생 배경이 됨.

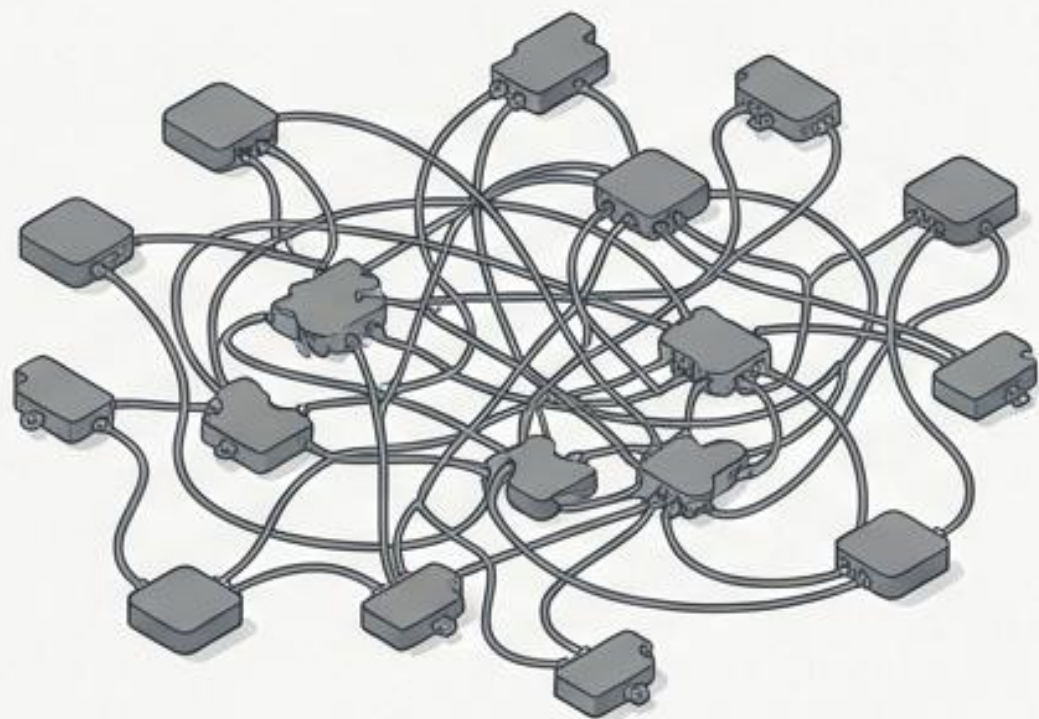


논리의 건축학: 완벽한 설계를 위한 14가지 디자인 패턴

주니어부터 아키텍트까지, 소프트웨어의 혼란을 잠재우는 구조적 청사진.

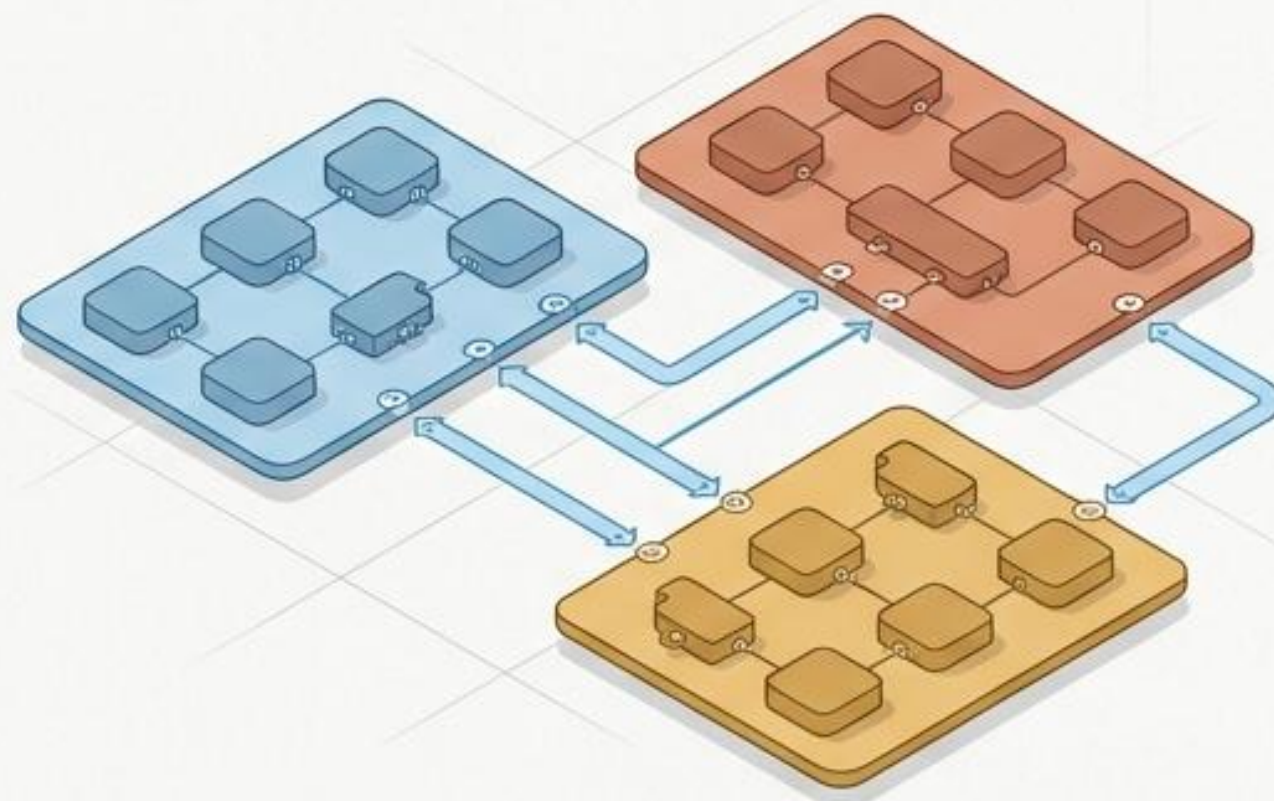
소프트웨어 설계의 혼란을 잠재우는 재사용 가능한 청사진

The Problem



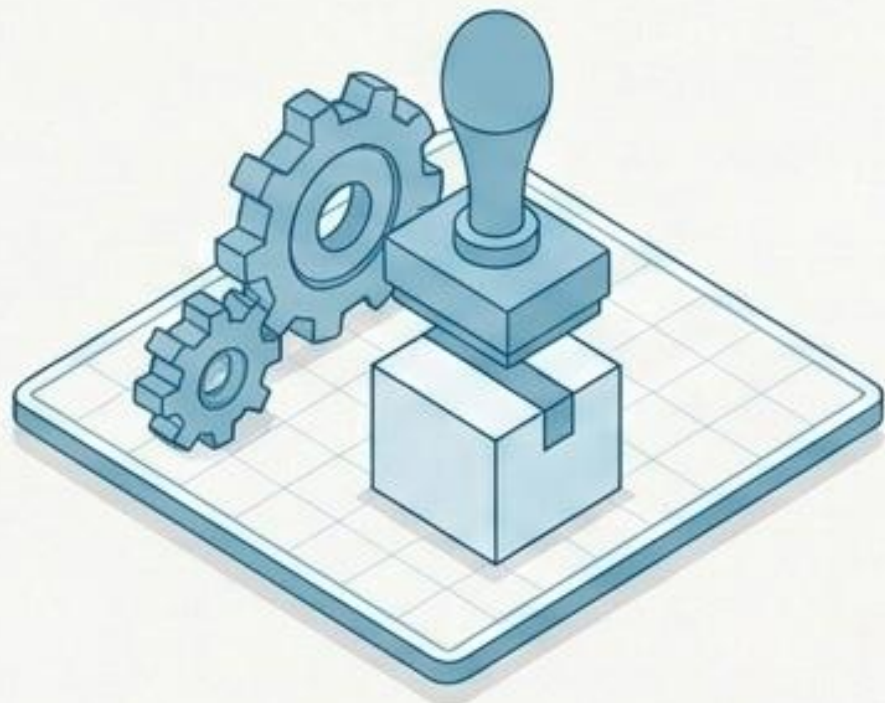
Spaghetti Code

The Solution



The Solution

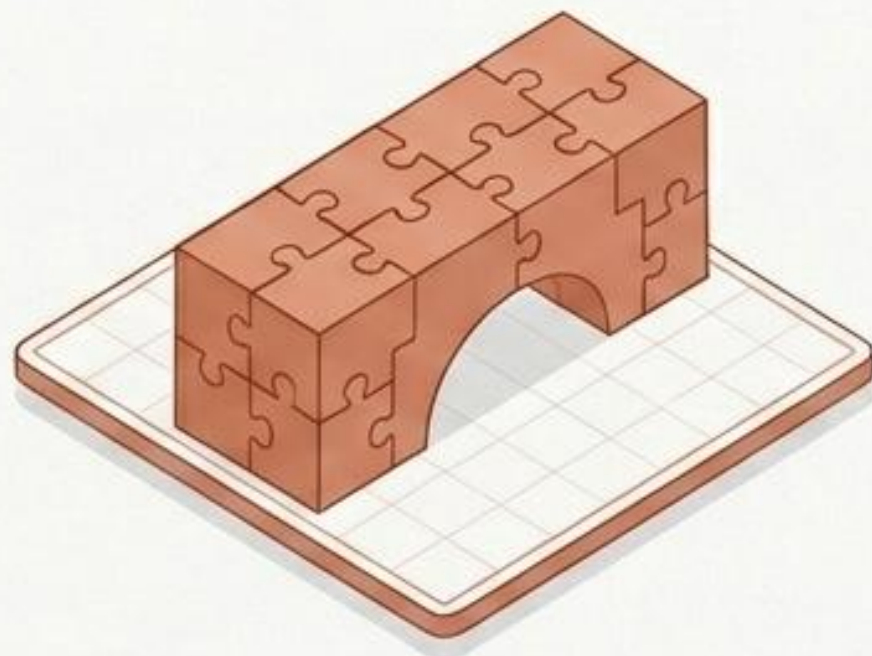
- ✓ 디자인 패턴은 소프트웨어 설계 시 자주 발생하는 문제를 해결하기 위한 검증된 설계 방법입니다.
- ✓ 바퀴를 다시 발명하지 않고, 수많은 선배 개발자들이 겪은 시행착오의 결과물을 프로젝트에 즉시 적용할 수 있습니다.
- ✓ 코드의 재사용성, 유연성, 그리고 팀원 간의 소통(공통 언어)을 극대화합니다.



생성 패턴 (Creational)

객체를 어떻게 안전하고 유연하게 만들 것인가?

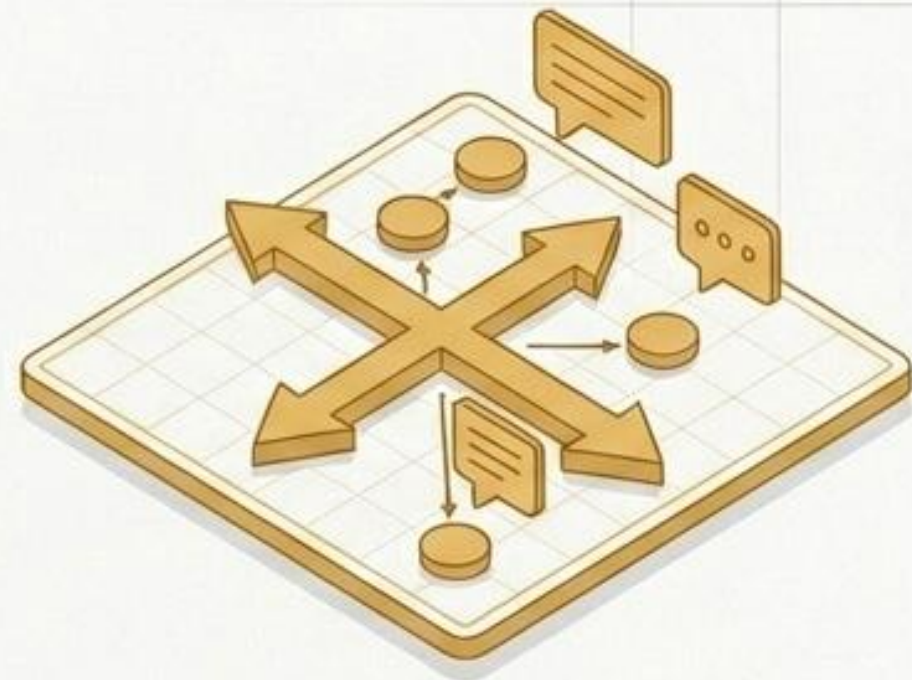
객체 생성 로직을 숨기고, 시스템과 객체 생성 방식을 분리.



구조 패턴 (Structural)

서로 다른 부품을 어떻게 조립하여 거대한 구조를 만들 것인가?

클래스와 객체를 조합하여 더 크고 효율적인 구조 형성.

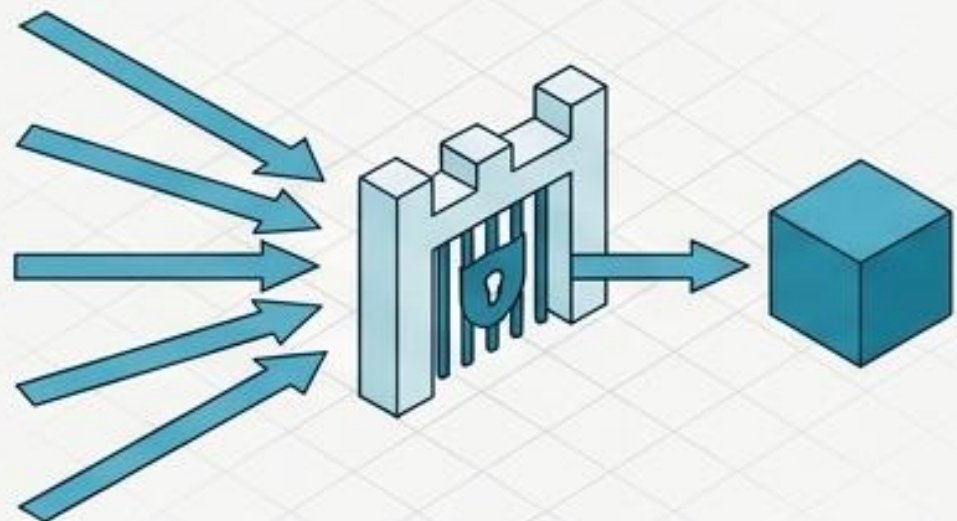


행동 패턴 (Behavioral)

객체들은 어떻게 소통하고, 책임을 나눌 것인가?

알고리즘 캡슐화 및 객체 간의 동적 메시지 교환 정의.

싱글톤 (Singleton)

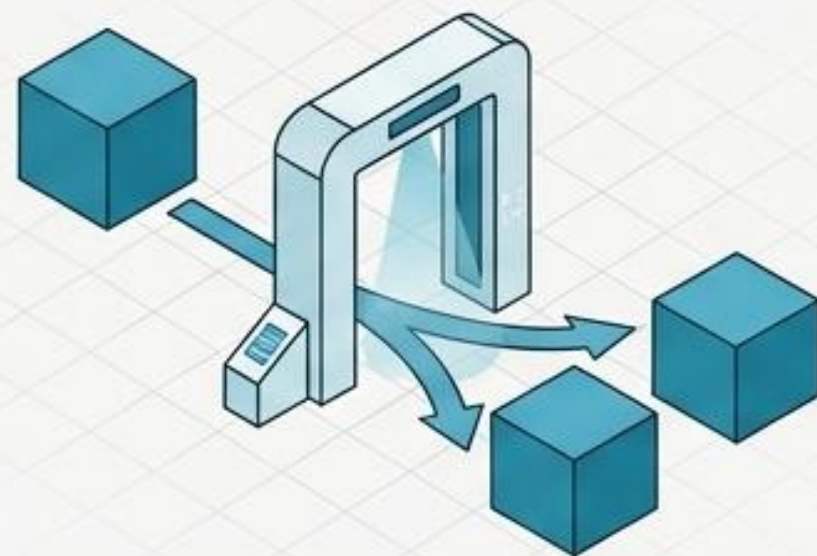


인스턴스가 단 1개만 생성되도록 보장 (메모리 절약, 공통 객체 공유).

 데이터베이스 연결  설정(Config) 관리  로그(Logger) 시스템

```
class Singleton {  
    private static Singleton instance;  
    private Singleton() {}  
    public static Singleton getInstance() {  
        if(instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

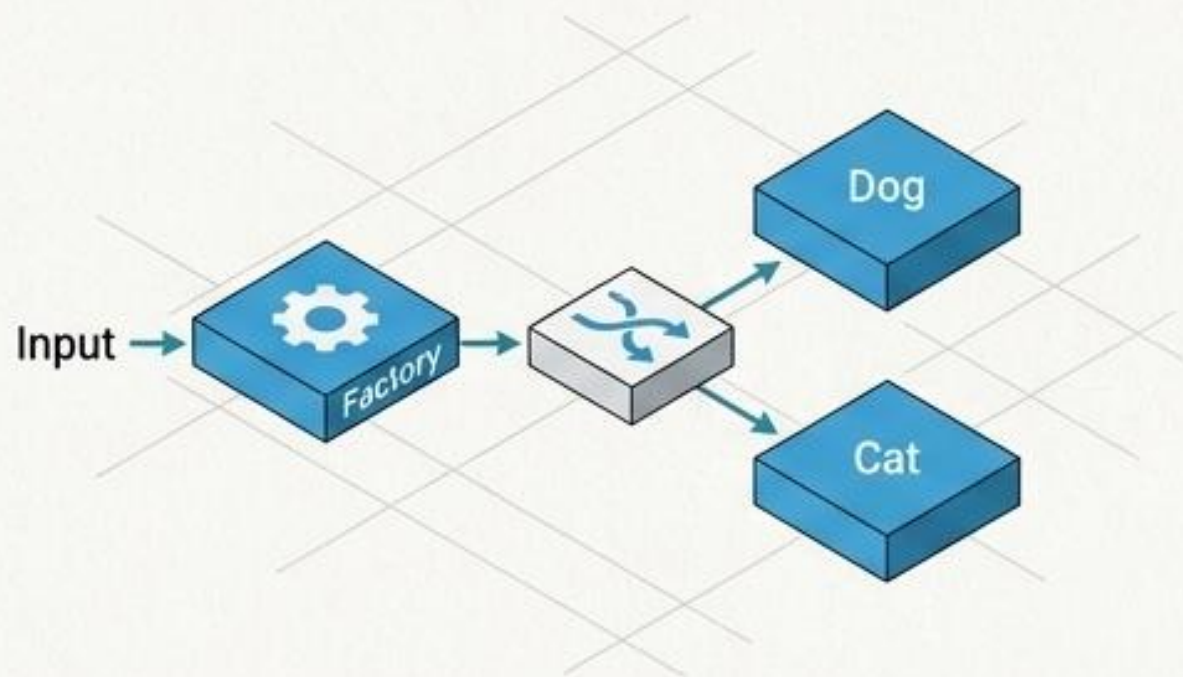
프로토타입 (Prototype)



생성 비용이 큰 기존 객체를 복사(Clone)하여 새 객체 생성.

 몬스터 복제  문서 템플릿 복사  그래픽 객체 복제

```
class Book implements Cloneable {  
    String title;  
  
    public Book clone() throws CloneNotSupportedException {  
        return (Book) super.clone();  
    }  
}
```

팩토리 메서드 (Factory Method)

객체 생성을 서브클래스(자식)에 위임.
객체의 구체적 클래스를 몰라도 생성 가능.



차량 생성

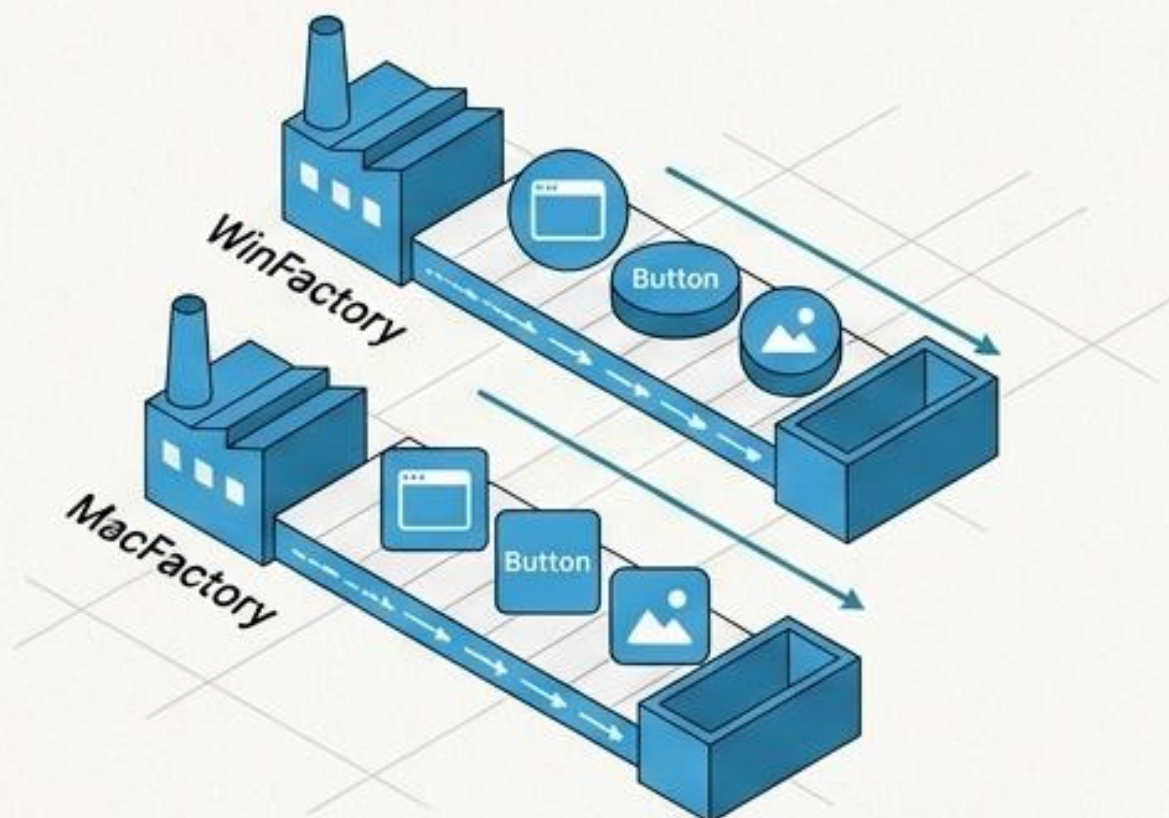


게임 캐릭터



OS별 버튼

```
class AnimalFactory {
    public Animal createAnimal() {
        return new Dog();
    }
}
```



추상 팩토리 (Abstract Factory)

서로 연관되거나 의존적인 객체들의
조합(제품군)을 통일성 있게 생성.



Win/Mac UI 세트



무기 세트



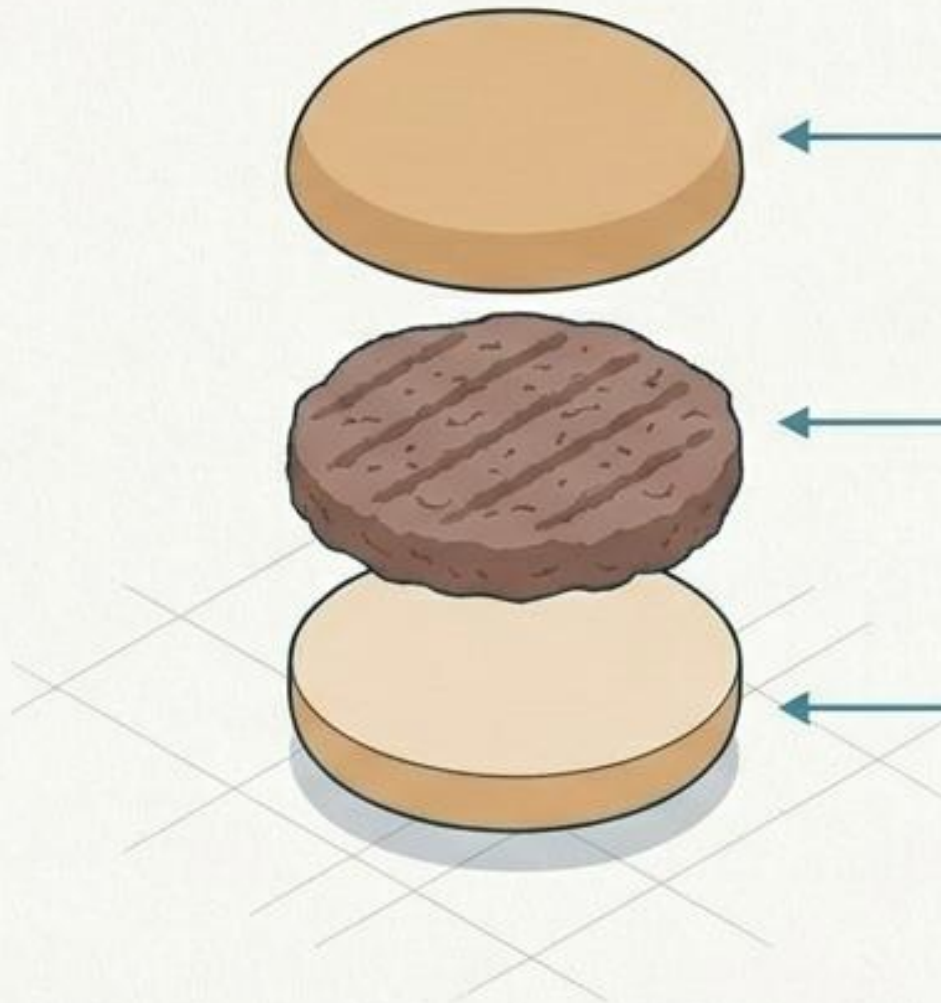
DB 종류별 객체

```
interface GUIFactory {
    Button createButton();
}

class WinFactory implements GUIFactory {
    public Button createButton() {
        return new WinButton();
    }
}
```


The Assembler: 복잡한 객체의 단계별 조립 (Builder)

복잡한 객체의 생성 과정을 단계별로 분리하여, 동일한 생성 과정으로 다양한 표현을 만들어내는 패턴 (가독성 극대화).



햄버거 주문
시스템



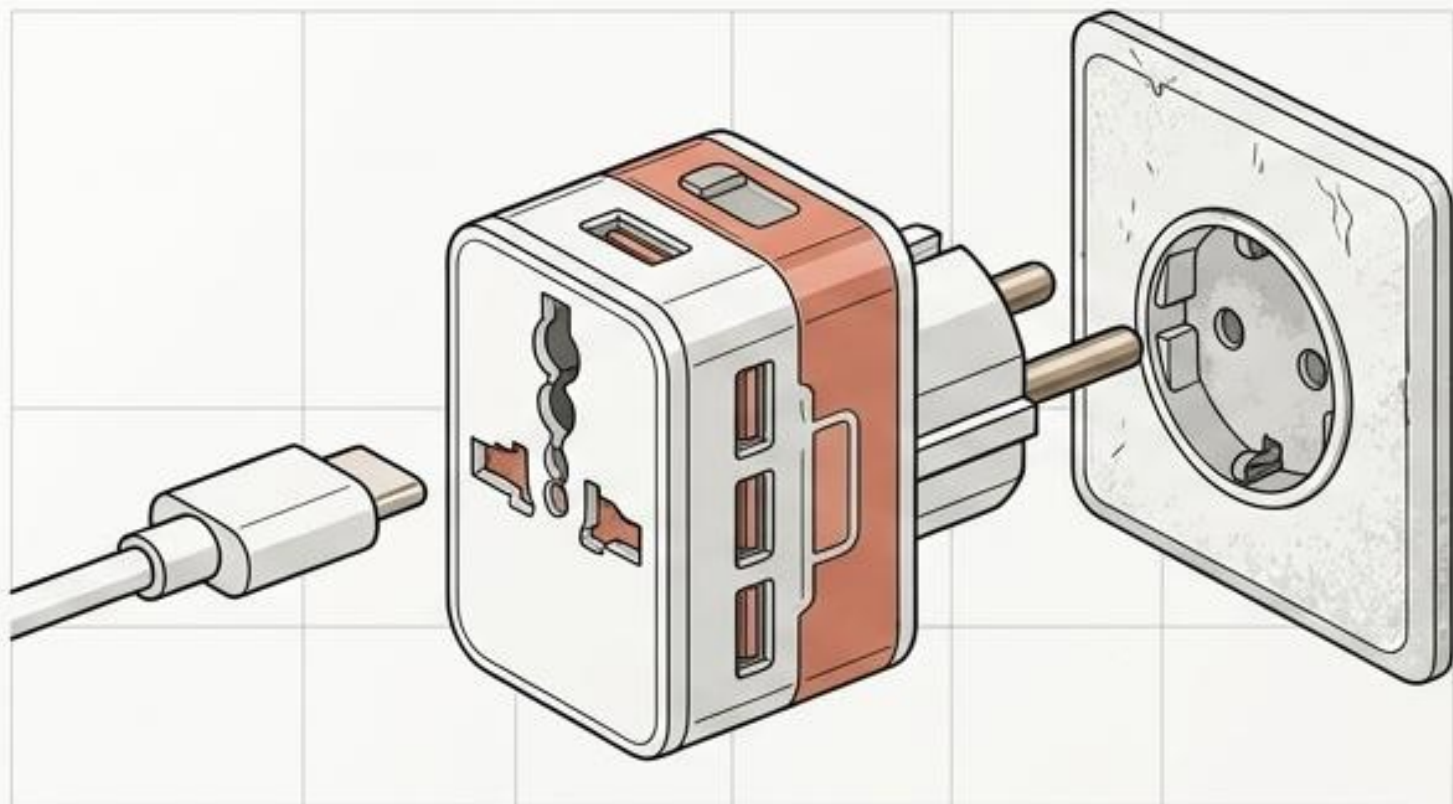
HTTP 요청
객체 생성



Lombok
@Builder

```
class User {  
    static class Builder {  
        private String name;  
        private int age;  
  
        Builder setName(String name) {  
            this.name = name;  
            return this;  
        }  
  
        Builder setAge(int age) {  
            this.age = age;  
            return this;  
        }  
  
        User build() {  
            User user = new User();  
            user.name = name;  
            user.age = age;  
            return user;  
        }  
    }  
}
```


어댑터 (Adapter)



호환되지 않는 인터페이스를
변환기처럼 연결하여
기존 코드 재사용.



USB-C → HDMI



오래된 API 연동

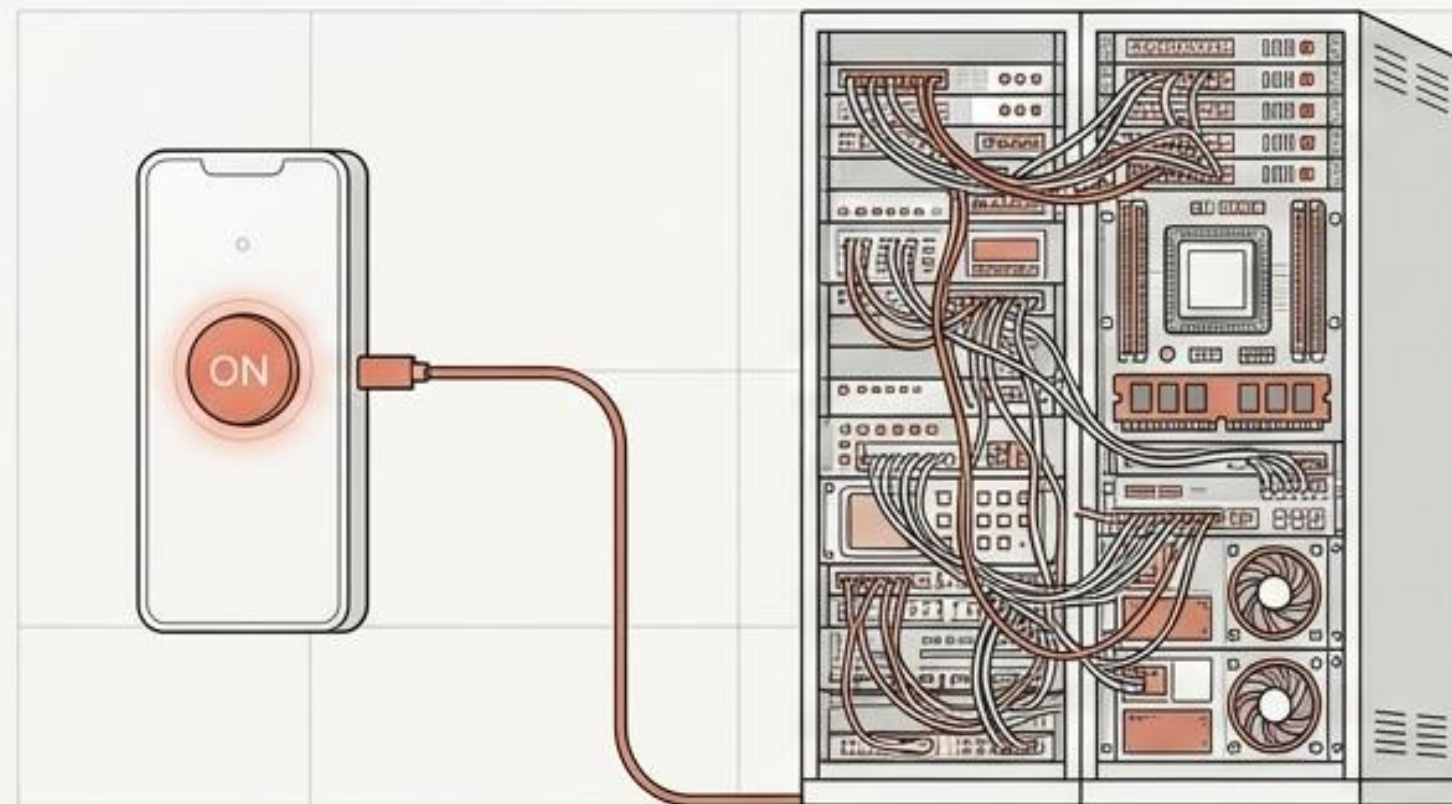


결제 시스템 통합

Adapter.java

```
class Adapter {  
  
    OldSystem old = new OldSystem();  
  
    void newMethod() {  
        old.oldMethod();  
    }  
}
```

퍼사드 (Facade)



복잡한 하위 시스템들에 대해
단순하고 통합된 상위 수준의
단일 인터페이스 제공.



컴퓨터 전원 켜기



영화관 시스템



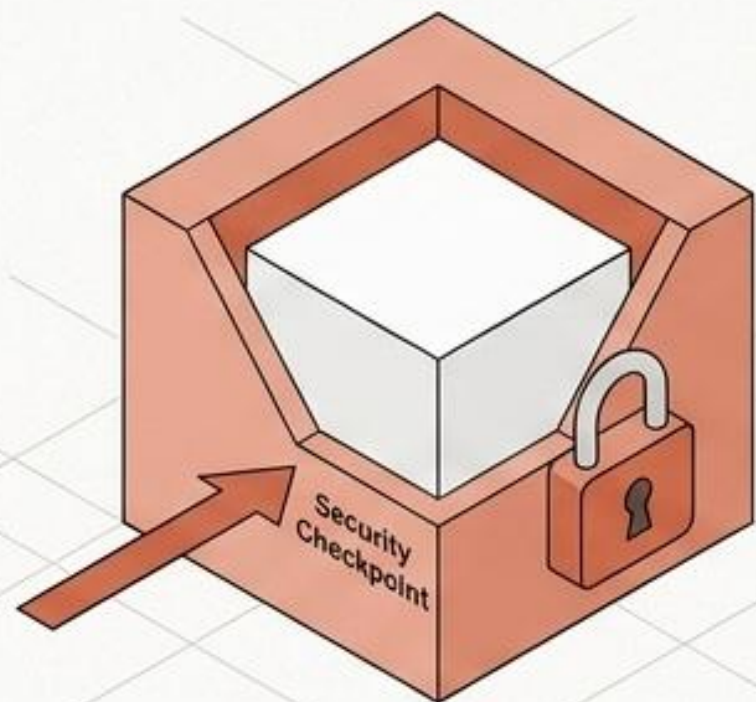
Spring Framework

ComputerFacade.java

```
class ComputerFacade {  
    CPU cpu = new CPU();  
    Memory memory = new Memory();  
  
    void start() {  
        cpu.start();  
        memory.load();  
    }  
}
```


프록시 (Proxy)

실제 객체에 대한 '접근을 제어'하거나 대신 수행하기 위한 대리자.



● ProxyService.java

```
class ProxyService implements Service {  
    private RealService service = new RealService();  
    public void run() {  
        System.out.println("권한 확인");  
        service.run();  
    }  
}
```

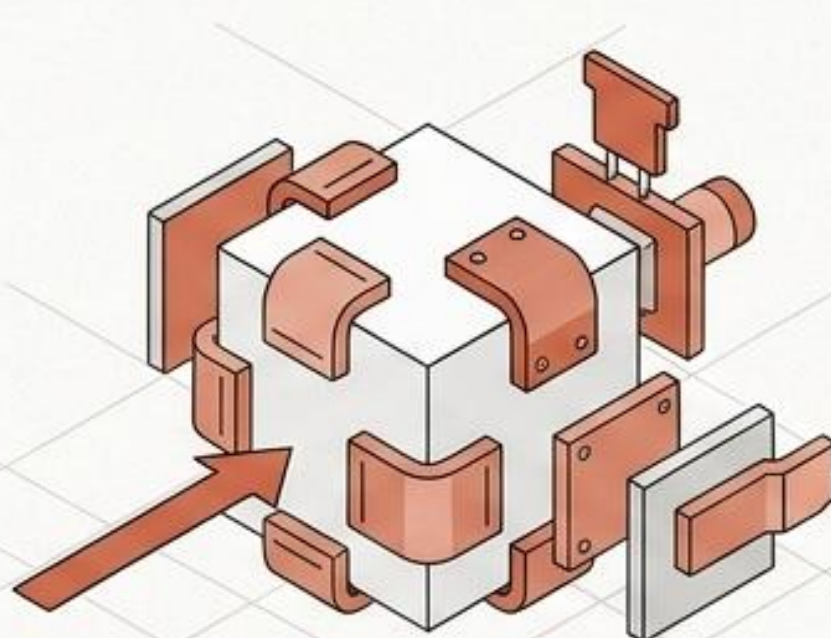
🕒 이미지 Lazy Loading

🛡️ 접근 제한

🗄️ 캐시 서버

데코레이터 (Decorator)

서브클래싱 없이 객체에 동적으로 '새로운 기능이나 책임'을 추가.



● MilkDecorator.java

```
class MilkDecorator implements Coffee {  
    Coffee coffee;  
    public String make() {  
        return coffee.make() + " + 우유";  
    }  
}
```

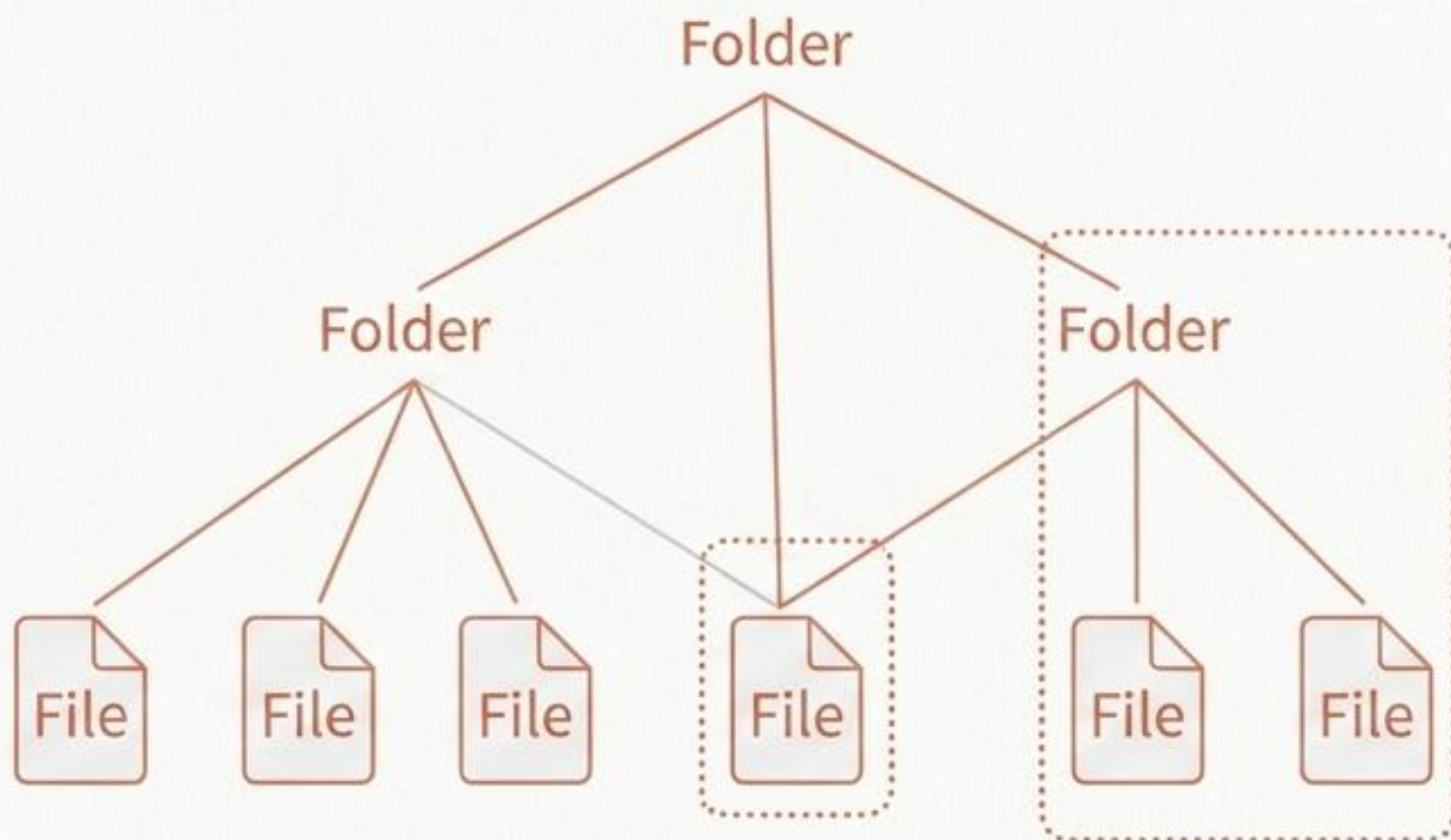
☕ 커피 옵션 추가

≈ Java IO Stream

🔔 알림 추가

The Fractal: 부분과 전체의 완벽한 동일시 (Composite)

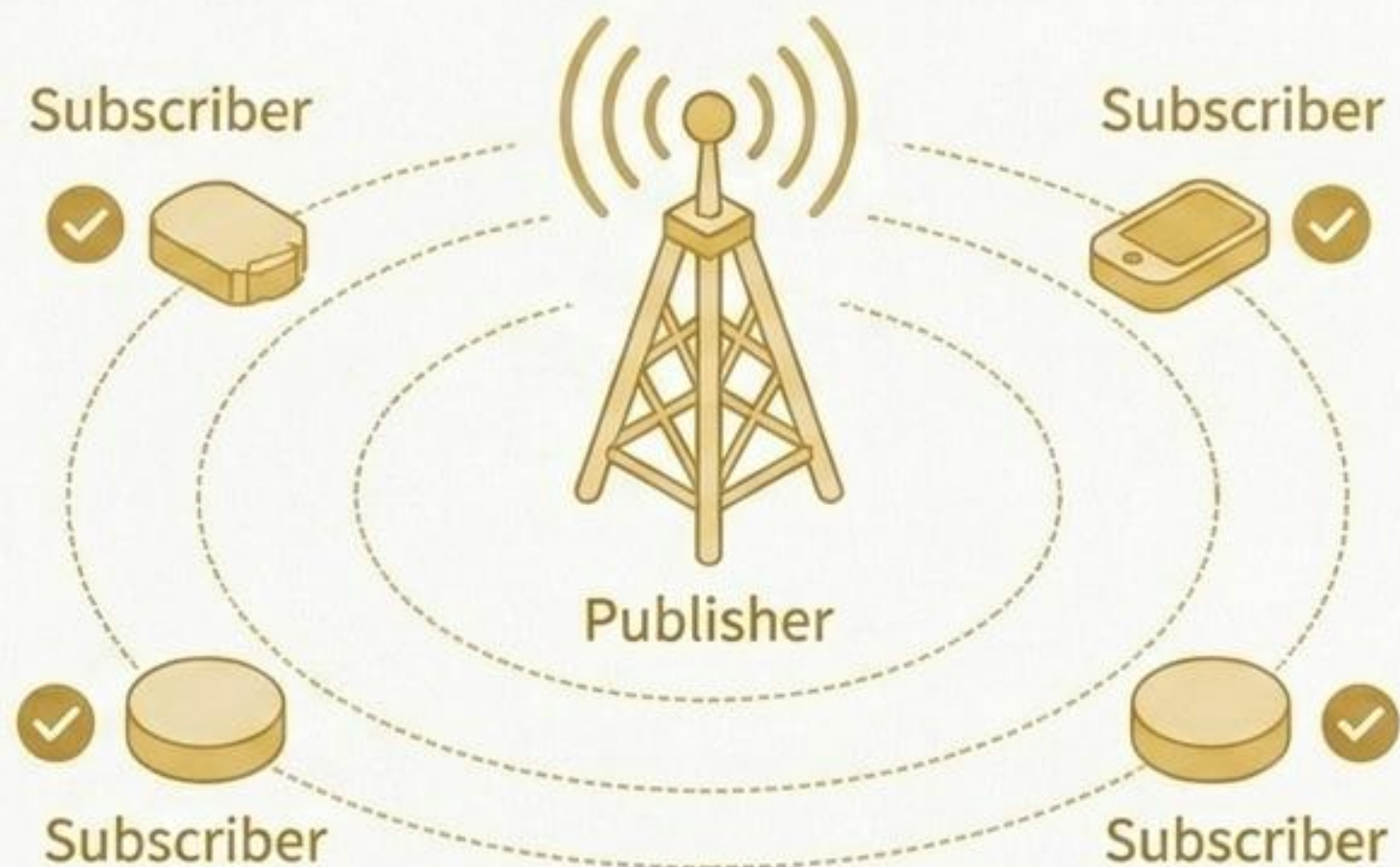
객체들의 관계를 트리 구조로 구성하여, '개별 단일 객체'와 '복합 객체 그룹'을 클라이언트가 동일한 방식으로 다루게 해주는 패턴.



```
interface FileSystem {  
    void show();  
}  
  
class File implements FileSystem {  
    public void show() {  
        System.out.println("파일");  
    }  
}
```


The Broadcaster: 상태 변화와 1:N 실시간 알림 (Observer)

중심 객체의 상태 변화가 발생할 때마다, 이를 구독(의존) 중인 여러 다른 객체들에게 변화를 자동으로 알리는 이벤트 처리 패턴.



```
interface Observer {  
    void update();  
}  
  
class Subscriber implements Observer {  
    public void update() {  
        System.out.println("알림 받음");  
    }  
}
```

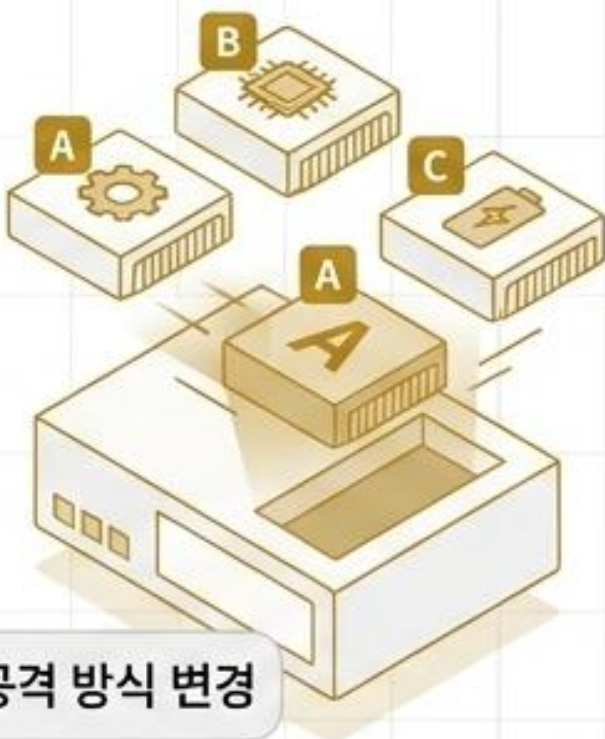
▶ 유튜브 구독 알림

📈 주식 시세 알림

💬 채팅 시스템

전략 (Strategy)

다양한 알고리즘(행동)을 캡슐화하여 런타임에 자유롭게 교체 (조건문 감소).

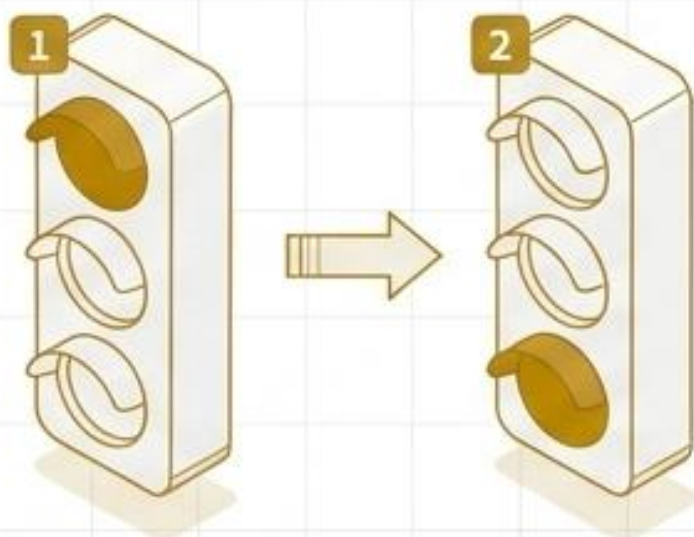


Java

```
interface PaymentStrategy {  
    void pay();  
}  
  
class CardPayment implements PaymentStrategy {  
    public void pay() {  
        System.out.println("카드 결제");  
    }  
}
```

상태 (State)

객체의 내부 '상태'에 따라 객체의 행동(메서드)이 완전히 달라지도록 캡슐화.



Java

```
interface State {  
    void action();  
}  
  
class RunningState implements State {  
    public void action() {  
        System.out.println("달리는 중");  
    }  
}
```

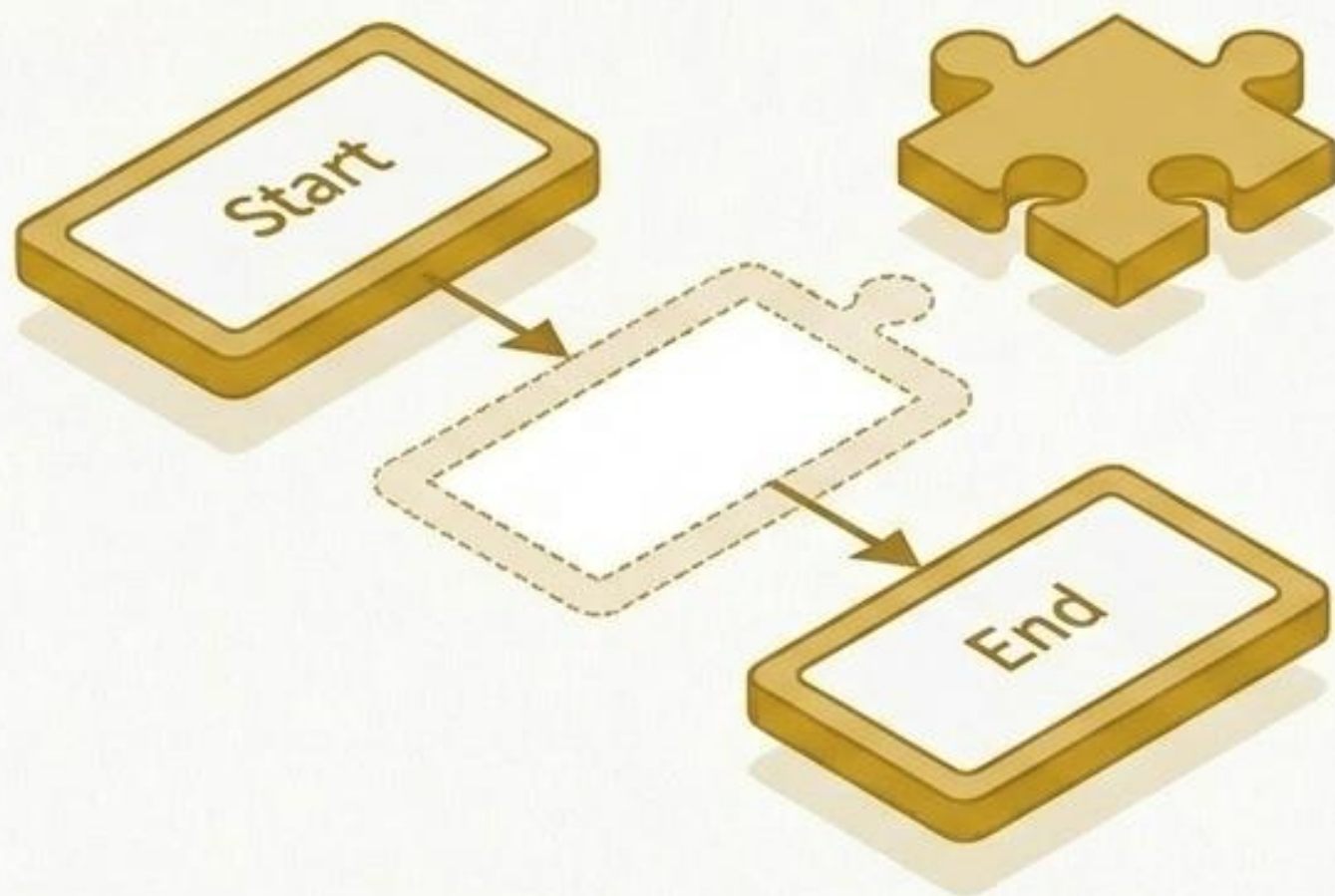
자동문 상태

자판기 상태

캐릭터 상태

The Skeleton: 알고리즘의 뼈대와 하위 클래스 재정의 (Template Method)

상위 클래스에서 알고리즘의 전체적인 뼈대(흐름)를 메서드에 정의하고, 하위 클래스에서 알고리즘의 특정 단계들만 알맞게 재정의(Override)하여 사용하는 패턴 (공통 로직 재사용 및 중복 제거).



Java

```
abstract class Game {  
    abstract void start();  
    abstract void play();  
  
    final void run() {  
        start();  
        play();  
    }  
}
```


Synthesis | The Pattern Matrix: 14 디자인 패턴 마스터 치트시트

분류	패턴	멘탈 모델	핵심 목적
생성	Singleton	절대 유일	클래스 인스턴스를 단 하나만 생성 및 공유
생성	Factory Method	단일 생성 위임	객체 생성을 자식 클래스(서브클래스)로 위임
생성	Abstract Factory	제품군 생성	호환되는 관련 객체들의 묶음을 통일성 있게 생성
생성	Builder	단계별 조립	복잡한 객체 생성 과정을 단계별로 분리
생성	Prototype	세포 분열	비용이 큰 기존 객체를 복제(Clone)하여 생성
구조	Adapter	호환용 플러그	호환되지 않는 인터페이스를 맞게 변환
구조	Proxy	검문소 (대리인)	실제 객체에 대한 접근 제어 및 대리 수행
구조	Decorator	모듈식 장갑	객체에 동적으로 새로운 행동/기능 추가
구조	Facade	만능 리모컨	복잡한 서브시스템을 단순한 단일 인터페이스로 제공
구조	Composite	프랙탈 트리	단일 객체와 복합 객체를 동일한 트리 구조로 처리
행동	Observer	방송국 구독	상태 변화 시 1:N으로 의존 객체들에게 자동 알림
행동	Strategy	카트리지 교체	알고리즘을 캡슐화하여 런타임에 동적 교체
행동	Template Method	빈칸 채우기	알고리즘 뼈대를 잡고 하위에서 세부 단계 재정의
행동	State	상태 전이	객체 의 내부 상태에 따라 행동 자체를 변경